
NOTES DE COURS

Python for Data Science

Cours, exemples, exercices et corrigés

Bechir Trabelsi

AXA France

Septembre 2026

Avant-propos

Ce cours apprend à transformer une question en programme, puis un programme en analyse défendable. Les quatre premiers chapitres installent le langage, les structures de données, le raisonnement algorithmique et la conception objet. Les cinq suivants appliquent ces idées au calcul numérique, à l'ingénierie des données, à l'incertitude statistique et à l'apprentissage automatique. Un chapitre bonus montre enfin comment construire une application d'IA générative avec l'écosystème Hugging Face sans confondre démonstration technique et système fiable.

Les notes ne sont pas un catalogue de syntaxe. Chaque notion part d'un problème, explicite la représentation choisie, suit l'état du calcul et vérifie le résultat. Les exemples inspirés des cours du MIT et de *Fluent Python* sont reformulés et adaptés à un public débutant francophone. La documentation officielle reste la référence pour le comportement des bibliothèques.

Méthode commune

1. **Explication du problème** : reformuler la question, les entrées, la sortie, les contraintes et les cas limites.
2. **Intuition** : construire un petit exemple, choisir une représentation et nommer l'état nécessaire.
3. **Solution directe ou force brute** : écrire une procédure manifestement correcte, même si elle répète du travail.
4. **Solution optimisée ou structurée** : identifier le goulot, proposer une amélioration justifiée et vérifier que le contrat ne change pas. Au chapitre 1, « améliorée » signifie surtout plus claire et plus fiable, sans notion algorithmique avancée.
5. **Réalisation** : exprimer l'algorithme en pseudo-code, puis seulement en Python ; terminer par des tests, des contrôles et une interprétation.

Table des matières

I	Fondements de Python et résolution de problèmes	1
1	Fondamentaux de Python et méthode de résolution	2
1.1	Objectifs	2
1.2	Pourquoi Python pour la science des données?	2
1.2.1	Un langage de liaison autour de calculs spécialisés	2
1.2.2	Interactivité et reproductibilité	3
1.2.3	Python complète les autres outils	3
1.2.4	Limites à connaître dès le début	3
1.3	Un programme comme suite d'évaluations	3
1.3.1	Valeurs et types	3
1.3.2	Chaînes et conversions	4
1.4	Décider avec des conditions	4
1.5	Répéter avec des boucles	5
1.5.1	Parcours et accumulateur	5
1.5.2	Compteur, filtre et meilleur candidat	5
1.5.3	Boucle <code>while</code> et terminaison	5
1.6	Fonctions simples	6
1.7	Méthode de résolution d'un petit problème	6
1.7.1	Étude guidée : résumé quotidien	6
1.8	Tester et déboguer	7
1.9	Approfondissement : lire, construire et expliquer un programme	7
1.9.1	Chaînes : accéder, transformer et découper du texte	7
1.9.2	Une spécification avant la syntaxe	8
1.9.3	La table de trace comme preuve locale	9
1.9.4	Construire par incréments vérifiables	9
1.9.5	Déboguer par catégories	10
1.10	Travail appliqué	10
1.11	À retenir	11
2	Fonctions, collections et programmes fiables	12
2.1	Objectifs	12
2.2	Décomposition et abstraction	12
2.2.1	Portée et appels	12
2.3	Séquences : chaîne, liste et tuple	13

2.3.1	Parcours expressif	13
2.3.2	Compréhensions après la boucle explicite	13
2.4	Dictionnaires et ensembles	13
2.5	Mutabilité, alias et copie	14
2.6	Erreurs, exceptions et tests	14
2.6.1	Partitions de tests	14
2.7	Étude de cas : consolider des commandes	15
2.8	Première lecture des coûts	15
2.9	Approfondissement : choisir des représentations qui portent le sens	15
2.9.1	Listes : construction, accès et tranches	15
2.9.2	Méthodes de liste : connaître effet, retour et erreur	16
2.9.3	Trier sans confondre mutation et résultat	17
2.9.4	Liste comme pile, mais pas comme file	17
2.9.5	Tuples : enregistrements, déballage et immutabilité	17
2.9.6	Dictionnaires : construction et accès explicite	17
2.9.7	Ensembles : appartenance et algèbre ensembliste	18
2.9.8	Itérer avec l'information réellement nécessaire	19
2.9.9	Partir des opérations dominantes	19
2.9.10	Références, alias et frontières de mutation	19
2.9.11	Une fonction comme frontière de raisonnement	20
2.9.12	Concevoir les tests depuis le contrat	20
2.9.13	Du Python natif aux opérations de données	20
2.10	Travail appliqué	21
2.11	À retenir	21
3	Algorithmes élémentaires et complexité	22
3.1	Objectifs	22
3.2	De la correction à l'efficacité	22
3.2.1	Temps et espace	22
3.3	Classes de croissance	23
3.3.1	Règles de composition	23
3.3.2	Cas favorable, moyen et défavorable	23
3.4	Coût des opérations Python	23
3.5	Recherche linéaire et recherche binaire	23
3.6	Hachage : supprimer une recherche répétée	24
3.6.1	Deux valeurs pour une cible	24
3.7	Motifs simples pour des données ordonnées	24
3.7.1	Deux pointeurs	24
3.7.2	Fenêtre glissante de largeur fixe	25
3.7.3	Fréquence et regroupement	25
3.8	Diagnostic d'un goulot d'étranglement	25
3.9	Étude de cas : analyser un journal d'événements	25
3.10	Approfondissement : établir et défendre un coût	26
3.10.1	Définir la taille de l'entrée	26
3.10.2	Sommes plutôt que slogans	26

3.10.3	Pire cas, cas moyen et coût amorti	26
3.10.4	Prétraiter pour répondre à plusieurs requêtes	26
3.10.5	Le tri comme investissement	27
3.10.6	Pont vers la science des données	27
3.11	Travail appliqué	27
3.12	À retenir	28
4	Conception objet pour les pipelines de données	29
4.1	Objectifs	29
4.2	Quand une classe aide	29
4.3	Classe, instance et <code>self</code>	29
4.4	Invariants et validation	30
4.5	Dataclasses et objets valeur	30
4.6	Composition plutôt qu'héritage automatique	30
4.6.1	Fonctions ou objets étapes ?	31
4.7	Frontières avec effets	31
4.8	Étude de cas : pipeline de sinistres	31
4.9	Tests de comportement	32
4.10	Approfondissement : modéliser un pipeline comme un système de contrats	32
4.10.1	Objet, état et responsabilité	32
4.10.2	Les invariants rendent les états impossibles	33
4.10.3	Dataclass, fonction ou classe ordinaire	33
4.10.4	Composition et injection des dépendances	33
4.10.5	Interface commune et sous-typage comportemental	34
4.10.6	État appris dans un pipeline de données	34
4.10.7	Tester l'interface publique	34
4.11	Travail appliqué	34
4.12	À retenir	35
II	Python appliqué à la science des données	36
5	Calcul scientifique avec NumPy	37
5.1	Question directrice	37
5.2	NumPy : un tableau numérique structuré	37
5.2.1	Vectorisation	37
5.2.2	Axes et réductions	37
5.2.3	Broadcasting comme algèbre des formes	38
5.2.4	Vues, copies et type	38
5.3	Pont vers pandas : une table avec un grain	38
5.3.1	Sélection, filtre et affectation explicite	38
5.3.2	Valeurs manquantes	39
5.3.3	Groupby : séparer, calculer, recombinaison	39
5.4	Jointures sous contrat	39
5.5	Lien avec les structures de données	40
5.6	Pipeline de préparation vérifiable	40

5.7	Approfondissement : raisonner sur formes, alignements et cardinalités	40
5.7.1	Calculer avec des formes, pas seulement avec des valeurs	40
5.7.2	Vectorisation : modèle d'exécution et limites	41
5.7.3	Vues, copies et mutation	41
5.7.4	Le grain comme invariant d'une DataFrame	41
5.7.5	Alignement pandas : puissance et piège	41
5.7.6	Groupby comme réduction contrôlée	42
5.7.7	Jointures : prédire la cardinalité	42
5.7.8	Chaîne de préparation reproductible	42
5.8	NumPy en profondeur : le tableau comme objet scientifique	42
5.8.1	Anatomie d'un ndarray	42
5.8.2	Construire sans ambiguïté	43
5.8.3	Choisir un type numérique	43
5.8.4	Reshape, transpose et invariants de taille	43
5.8.5	Indexation de base et indexation avancée	43
5.8.6	Règles complètes de broadcasting	44
5.8.7	Ufuncs, réductions et accumulation	44
5.8.8	Valeurs manquantes et valeurs non finies	44
5.8.9	Algèbre linéaire : résoudre plutôt qu'inverser	45
5.8.10	Génération aléatoire moderne	45
5.8.11	Vectoriser, fusionner ou traiter par blocs	45
5.8.12	Mesurer correctement	46
5.9	Travail appliqué	46
5.10	À retenir	47
6	Ingénierie tabulaire et analyse exploratoire avec pandas	48
6.1	Question directrice	48
6.2	Le grain et la population	48
6.3	Décrire une variable numérique	48
6.4	Forme, asymétrie et valeurs atypiques	49
6.5	Valeurs manquantes comme signal	49
6.6	Comparer des groupes	50
6.7	Relations entre variables	50
6.8	Un graphique répond à une question	50
6.9	Analyse de sensibilité	51
6.10	Étude de cas : retard de livraison	51
6.11	Approfondissement : conduire une enquête descriptive	51
6.11.1	Séparer qualité de données et phénomène étudié	51
6.11.2	Choisir un résumé pour une question	51
6.11.3	Valeurs atypiques : signaler, expliquer, décider	52
6.11.4	Comprendre les mécanismes de manque	52
6.11.5	Association, composition et paradoxe de Simpson	52
6.11.6	Construire une figure vérifiable	52
6.11.7	De la figure à la phrase	53
6.12	pandas en profondeur : transformer sous contrat	53

6.12.1	Series, DataFrame et Index	53
6.12.2	Schéma à l'entrée	53
6.12.3	Types d'extension et absence	54
6.12.4	Sélection et affectation sans ambiguïté	54
6.12.5	Tidy data et remodelage	54
6.12.6	Groupby : agrégation, transformation et filtrage	54
6.12.7	Jointures et preuves de cardinalité	55
6.12.8	Concaténation et union de schémas	55
6.12.9	Texte, dates et temps	55
6.12.10	Resample, rolling et frontières temporelles	56
6.12.11	Chaînes de méthodes et fonctions <code>pipe</code>	56
6.12.12	Performance et passage à l'échelle	56
6.13	Travail appliqué	57
6.14	À retenir	57
7	Calcul scientifique avec SciPy, probabilités et simulation	58
7.1	Question directrice	58
7.2	Expérience aléatoire et variable aléatoire	58
7.3	Génération reproductible	58
7.4	Probabilité conditionnelle et indépendance	59
7.4.1	Bayes et taux de base	59
7.5	Distributions comme modèles générateurs	59
7.5.1	Bernoulli et binomiale	59
7.5.2	Poisson et exposition	60
7.5.3	Normale et standardisation	60
7.6	Monte-Carlo : estimer par répétition	60
7.6.1	Exemple : risque de rupture	60
7.7	Loi des grands nombres et théorème central limite	61
7.8	Simulation conditionnelle	61
7.9	Approfondissement : construire un monde simulé et l'auditer	61
7.9.1	Une distribution encode un mécanisme	61
7.9.2	De la question à l'algorithme Monte-Carlo	62
7.9.3	Mesurer l'erreur Monte-Carlo	62
7.9.4	Probabilité conditionnelle et populations rares	63
7.9.5	Indépendance : hypothèse, pas absence de corrélation observée	63
7.9.6	Loi des grands nombres et TCL : deux messages distincts	63
7.9.7	Audit d'une simulation	63
7.10	SciPy en profondeur : algorithmes scientifiques spécialisés	63
7.10.1	Une bibliothèque organisée par problème	64
7.10.2	Objets distribution dans <code>scipy.stats</code>	64
7.10.3	Paramètres et unités	64
7.10.4	Ajuster une distribution sans surinterpréter	64
7.10.5	Tests statistiques comme contrats d'hypothèses	65
7.10.6	Test de permutation	65
7.10.7	Intégration numérique	65

7.10.8	Optimisation : objectif, domaine et convergence	66
7.10.9	Racines et calibration	66
7.10.10	Interpolation et extrapolation	66
7.10.11	Matrices creuses	66
7.10.12	Équations différentielles comme modèles dynamiques	67
7.11	Travail appliqué	67
7.12	À retenir	68
8	Inférence fréquentiste et bayésienne avec statsmodels et PyMC	69
8.1	Question directrice	69
8.2	Population, échantillon et estimande	69
8.3	Estimation et erreur standard	69
8.4	Bootstrap	69
8.5	Hypothèse nulle et valeur p	70
8.6	Significativité statistique et pratique	70
8.7	Conception d'une expérience A/B	70
8.7.1	Randomisation	71
8.7.2	Avant le lancement	71
8.7.3	Contrôles d'intégrité	71
8.8	Étude complète : conversion	71
8.9	Puissance et taille d'échantillon	72
8.10	Multiplicité et analyses répétées	72
8.11	Approfondissement : de l'expérience à la décision	72
8.11.1	Définir l'estimande avant la métrique	72
8.11.2	Intervalle : effet plausible et précision	72
8.11.3	Bootstrap au bon niveau	73
8.11.4	Ce que la valeur p dit et ne dit pas	73
8.11.5	Audit opérationnel avant estimation	73
8.11.6	Analyse en intention de traiter	74
8.11.7	Puissance, durée et effet minimal utile	74
8.11.8	Note de décision reproductible	74
8.12	PyMC en profondeur : modéliser l'incertitude	74
8.12.1	Le changement de question bayésien	74
8.12.2	Workflow bayésien	75
8.12.3	Premier modèle PyMC	75
8.12.4	Priors interprétables	75
8.12.5	Prior prédictif	76
8.12.6	NUTS et chaînes	76
8.12.7	Diagnostics indispensables	76
8.12.8	Posterior predictive check	76
8.12.9	Décision A/B postérieure	77
8.12.10	Lien avec la solution analytique	77
8.12.11	Modèles hiérarchiques et pooling partiel	77
8.12.12	Régression bayésienne appliquée	78
8.12.13	Sensibilité et comparaison	78

8.12.14	Reproductibilité et coût	78
8.13	Travail appliqué	78
8.14	À retenir	79
9	Systèmes de machine learning avec scikit-learn	80
9.1	Question directrice	80
9.2	Formuler la tâche	80
9.3	Séparer avant d'apprendre	80
9.4	Baseline avant modèle complexe	81
9.5	Prétraitement dans un pipeline	81
9.6	Régression et classification	82
9.6.1	Métriques de régression	82
9.6.2	Matrice de confusion	82
9.7	Seuil, capacité et coûts	82
9.8	Surapprentissage et validation	82
9.9	Analyse des erreurs	83
9.10	Interprétation et limites	84
9.11	Étude complète : attrition client	84
9.12	Approfondissement : évaluer un système de décision, pas seulement un modèle	84
9.12.1	Formulation avant sélection d'algorithme	84
9.12.2	Séparation adaptée au déploiement	84
9.12.3	Baselines qui représentent une décision réelle	84
9.12.4	Pipeline et fuite de prétraitement	85
9.12.5	Métriques comme coûts partiels	85
9.12.6	Choisir un seuil par valeur et capacité	85
9.12.7	Discrimination et calibration	86
9.12.8	Surapprentissage diagnostiqué par courbes	86
9.12.9	Analyse d'erreurs par tranches	86
9.12.10	Du test au suivi en production	86
9.13	scikit-learn en profondeur : une API pour expérimenter sans fuite	86
9.13.1	Le contrat des estimateurs	86
9.13.2	Le split est un modèle du futur	87
9.13.3	Prétraitement hétérogène avec <code>ColumnTransformer</code>	87
9.13.4	Transformateurs personnalisés	87
9.13.5	Validation croisée et distribution des scores	88
9.13.6	Recherche d'hyperparamètres	88
9.13.7	Scoring et orientation	88
9.13.8	Seuil appris séparément	89
9.13.9	Calibration	89
9.13.10	Déséquilibre de classes	89
9.13.11	Interprétation conditionnelle au modèle	89
9.13.12	Courbes d'apprentissage et de validation	89
9.13.13	Incertitude des métriques	89
9.13.14	Sérialisation et contrat de version	90
9.13.15	Monitoring en production	90

9.13.16 Boucle de rétroaction	90
9.14 Travail appliqué	90
9.15 À retenir	91
III Complément : applications d'IA générative	92
10 Bonus — Construire des applications d'IA générative avec Hugging Face	93
10.1 Question directrice	93
10.2 Cartographie de l'écosystème	93
10.2.1 Lire une model card comme une fiche de risque	93
10.3 Deux chemins d'inférence	94
10.3.1 Pipeline local	94
10.3.2 Inférence hébergée	94
10.4 Tokenisation : le texte devient des identifiants	95
10.4.1 Chat templates	95
10.5 Contrôler la génération	95
10.6 Prompts comme interfaces versionnées	96
10.7 Datasets : données reproductibles et diffusion	96
10.7.1 Prévenir la contamination	97
10.8 Embeddings et recherche augmentée	97
10.9 Architecture testable d'une application	98
10.10 Évaluer avant de démontrer	98
10.11 Résilience, sécurité et observabilité	98
10.12 Performance : batch, cache et budget	99
10.13 Étude de cas : triage d'incidents	99
10.14 Approfondissement : du prototype au système génératif	99
10.14.1 Décomposer l'incertitude	99
10.14.2 Révisions et reproductibilité	100
10.14.3 Longueur, complexité et mémoire d'attention	100
10.14.4 Récupération : mesurer séparément chaque étage	100
10.14.5 Choisir le découpage	100
10.14.6 Outils et actions	100
10.14.7 Évaluation statistique de deux versions	101
10.14.8 Une boucle de développement défendable	101
10.14.9 Quand ne pas utiliser un modèle génératif	101
10.15 Travail appliqué	101
10.16 À retenir	102
IV Corrigés raisonnés	103
Corrigés du chapitre 1	104
Corrigés du chapitre 2	108
Corrigés du chapitre 3	111

Corrigés du chapitre 4	114
Corrigés du chapitre 5	117
Corrigés du chapitre 6	122
Corrigés du chapitre 7	127
Corrigés du chapitre 8	132
Corrigés du chapitre 9	137
Corrigés du chapitre 10	142
A Références et prolongements	148

Première partie

Fondements de Python et résolution de problèmes

Fondamentaux de Python et méthode de résolution

Écrire ses premiers programmes sans mémoriser des recettes

1.1 Objectifs

À la fin du chapitre, l'étudiant doit savoir lire un programme court, prédire son résultat, utiliser les types élémentaires, écrire une condition et une boucle, encapsuler un calcul simple dans une fonction et tester plusieurs cas. Ce chapitre ne suppose aucune expérience préalable de Python.

1.2 Pourquoi Python pour la science des données ?

Python est un langage généraliste, lisible et interactif. Il permet d'exprimer un premier algorithme avec peu de mécanismes, puis de conserver le même langage pour lire des fichiers, automatiser une analyse, construire un modèle et exposer un service. Cette continuité réduit la distance entre une idée, une expérience reproductible et un programme maintenable.

Le principal avantage ne vient pas d'une syntaxe « magique ». Il vient de la combinaison de trois niveaux :

- le langage fournit conditions, boucles, fonctions, objets et modules ;
- la bibliothèque standard fournit fichiers, collections, dates, journalisation et tests ;
- l'écosystème scientifique fournit des structures spécialisées : tableaux NumPy, DataFrames pandas, algorithmes SciPy, modèles probabilistes PyMC et pipelines scikit-learn.

1.2.1 Un langage de liaison autour de calculs spécialisés

Une boucle Python pure n'est pas toujours la manière la plus rapide de traiter des millions de nombres. Les bibliothèques scientifiques déplacent les boucles intensives vers du code compilé et exposent une interface Python. L'étudiant garde un langage cohérent pour formuler le problème, préparer les données et vérifier les résultats, tout en utilisant des noyaux numériques spécialisés.

Exemple — une même question, plusieurs niveaux

Pour calculer une moyenne, une boucle explicite montre l'accumulateur et le nombre de valeurs. Plus tard, `array.mean()` exprime une réduction NumPy sur un axe, et `df.groupby(...).mean()` calcule une moyenne par groupe pandas. La question reste la même, mais la représentation et le contrat deviennent plus riches.

1.2.2 Interactivité et reproductibilité

L'interpréteur et les notebooks donnent un retour rapide, utile pour explorer un petit exemple et inspecter un résultat. Un script ou module rend ensuite l'ordre d'exécution explicite, testable et réutilisable. Le cours utilise les deux : notebook pour enquêter, fonctions et modules pour stabiliser le raisonnement. Un notebook exécuté dans le désordre peut cacher un état ancien ; il doit pouvoir être relancé depuis un noyau neuf.

1.2.3 Python complète les autres outils

SQL reste naturel pour filtrer et agréger près d'une base de données. R possède un écosystème statistique et graphique majeur. Les langages compilés conviennent lorsque contrôle bas niveau et performances strictes dominent. Choisir Python ne signifie pas ignorer ces outils. Une architecture de données sérieuse combine souvent SQL pour extraire, Python pour modéliser et automatiser, puis un service ou rapport pour restituer.

1.2.4 Limites à connaître dès le début

Python vérifie beaucoup d'erreurs de type à l'exécution. Les environnements et versions doivent être gérés pour reproduire une analyse. Les objets Python ont un coût mémoire ; une boucle pure peut être lente ; la concurrence CPU possède des contraintes selon l'implémentation. Ces limites motivent les pratiques du cours : petits contrats, tests, environnements isolés, profilage, vectorisation et choix de structure adapté.

Pourquoi l'utiliser dans ce cours

Python permet de rendre visibles les étapes du raisonnement. Nous commencerons par suivre l'état d'une boucle et justifier une structure de données. Nous passerons ensuite à des bibliothèques plus abstraites sans abandonner les questions fondamentales : quelles sont les entrées, quel objet représente le problème, quel coût paie l'algorithme et comment vérifier la sortie ?

1.3 Un programme comme suite d'évaluations

Python exécute normalement les instructions de haut en bas. Une *expression* produit une valeur. Une *affectation* associe cette valeur à un nom. Le signe = ne signifie donc pas « est égal à » au sens mathématique.

Exemple — suivre les noms

```
1 distance_km = 12
2 duration_min = 30
3 speed = distance_km / (duration_min / 60)
4 distance_km = 15
5 print(speed)          # 24.0, la valeur déjà calculée ne change pas
```

La troisième ligne utilise les valeurs disponibles à cet instant. Changer `distance_km` ensuite ne recalcule pas automatiquement `speed`.

1.3.1 Valeurs et types

Les premiers types utiles sont `int`, `float`, `bool`, `str` et la valeur spéciale `None`. Le type détermine les opérations permises : `3 + 4` produit 7, tandis que `"3" + "4"` produit la chaîne "34". Une

conversion explicite, par exemple `int("34")`, change la représentation.

Type	Exemple	Opérations fréquentes
int, float	12, 3.5	+ - * / // % **
bool	True	and, or, not
str	"Paris"	concaténation, index, tranche
NoneType	None	absence explicite de résultat

1.3.2 Chaînes et conversions

Une chaîne est une suite de caractères numérotés à partir de zéro. La tranche `text[start:stop]` inclut `start` et exclut `stop`. Cette convention demi-ouverte donne immédiatement une longueur `stop - start`.

```

1 city = "Paris"
2 print(city[0])      # P
3 print(city[1:4])    # ari
4 age_text = input("Age: ")
5 age = int(age_text)
6 print(f"Dans un an: {age + 1}")

```

Point de vigilance — l'entrée est du texte

`input` renvoie toujours une chaîne. L'expression `input(...) + 1` échoue donc. Il faut décider si l'entrée représente un entier, un réel ou un texte, puis convertir explicitement.

1.4 Décider avec des conditions

Une comparaison produit un booléen. Une structure `if/elif/else` partitionne les cas. L'ordre importe lorsque plusieurs conditions peuvent être vraies.

Exemple — tarif de musée

```

1 age = 19
2 student = True
3
4 if age < 0:
5     price = None
6 elif age < 12:
7     price = 0
8 elif student:
9     price = 8
10 else:
11     price = 12

```

On traite d'abord l'entrée invalide, puis l'enfant, puis les autres visiteurs. Pour vérifier la couverture des cas, essayer les âges `-1`, `11`, `12` et `25`.

Les comparaisons peuvent s'enchaîner : `0 <= temperature <= 40`. Les opérateurs `and`, `or` et `not` combinent des propositions. Une condition claire décrit une règle du problème ; une longue condition obscure signale souvent que les cas n'ont pas été correctement séparés.

1.5 Répéter avec des boucles

Une boucle `for` parcourt une suite connue. Une boucle `while` convient lorsque la répétition dépend d'un état qui évolue.

1.5.1 Parcours et accumulateur

```
1 readings = [18.2, 19.4, 21.0, 20.4]
2 total = 0.0
3 for value in readings:
4     total = total + value
5 mean = total / len(readings)
```

Le tableau de trace rend le calcul observable :

Tour	value	total après le tour
1	18.2	18.2
2	19.4	37.6
3	21.0	58.6
4	20.4	79.0

Après chaque tour, `total` contient la somme des valeurs déjà parcourues. Cette phrase suffit à comprendre la boucle sans invoquer encore de théorie avancée.

1.5.2 Compteur, filtre et meilleur candidat

Trois états apparaissent constamment en programmation de données :

- un compteur augmente lorsqu'une condition est satisfaite ;
- une liste de sortie reçoit les observations retenues ;
- un candidat mémorise la meilleure valeur rencontrée.

```
1 above_20 = 0
2 warm = []
3 highest = readings[0]
4 for value in readings:
5     if value > 20:
6         above_20 += 1
7         warm.append(value)
8     if value > highest:
9         highest = value
```

1.5.3 Boucle `while` et terminaison

```
1 attempts = 3
2 while attempts > 0:
3     password = input("Mot de passe: ")
4     if password == "python":
5         print("Bienvenue")
6         break
7     attempts -= 1
```

Avant d'écrire une boucle `while`, il faut nommer l'état qui change et expliquer pourquoi la condition finira par devenir fausse. Ici, `attempts` diminue.

1.6 Fonctions simples

Une fonction donne un nom à un calcul réutilisable. Les paramètres sont les entrées du calcul ; `return` fournit la sortie à l'appelant.

```
1 def celsius_to_fahrenheit(celsius):
2     """Convert a temperature from Celsius to Fahrenheit."""
3     return celsius * 9 / 5 + 32
4
5 morning = celsius_to_fahrenheit(18)
6 evening = celsius_to_fahrenheit(22)
7 print(morning, evening)
```

Contrat minimal d'une fonction

Le contrat indique ce que représente chaque paramètre, la valeur renvoyée et les cas non admis. Pour une première fonction, une phrase et deux exemples suffisent. Il faut surtout distinguer `return`, qui produit une valeur réutilisable, de `print`, qui affiche du texte.

1.7 Méthode de résolution d'un petit problème

Du texte au programme

1. Réécrire l'énoncé en une phrase sans vocabulaire Python.
2. Identifier les entrées et la sortie attendue.
3. Calculer à la main deux exemples et un cas limite.
4. Décomposer la règle en décisions et répétitions.
5. Nommer les variables selon leur rôle.
6. Écrire un pseudo-code court, puis le traduire.
7. Tracer un exemple avant d'exécuter le programme.
8. Comparer le résultat obtenu au résultat attendu.

1.7.1 Étude guidée : résumé quotidien

On reçoit des températures et un seuil d'alerte. On veut retourner la moyenne, le nombre d'alertes et la température maximale. On suppose la liste non vide.

```
1 def daily_report(values, alert_threshold):
2     total = 0.0
3     alerts = 0
4     highest = values[0]
5
6     for value in values:
7         total += value
8         if value >= alert_threshold:
9             alerts += 1
10        if value > highest:
11            highest = value
12
13    return {
14        "mean": total / len(values),
```

```
15     "alerts": alerts,  
16     "highest": highest,  
17 }
```

La représentation choisie est une liste de mesures en entrée et un dictionnaire nommé en sortie. Une seule boucle maintient trois informations indépendantes. Pour n valeurs, elle effectue un nombre de tours proportionnel à n et conserve seulement quelques nombres en plus de l'entrée et de la sortie.

1.8 Tester et déboguer

Un test associe une entrée à un résultat attendu. Il faut couvrir un cas normal, une frontière et une valeur susceptible de révéler une erreur.

```
1 assert celsius_to_fahrenheit(0) == 32  
2 assert celsius_to_fahrenheit(100) == 212  
3 assert daily_report([20], 20)["alerts"] == 1
```

Lorsqu'un résultat surprend, réduire le problème : lire le message d'erreur, trouver la première ligne où l'état devient incorrect, afficher ou tracer les variables utiles, puis corriger une seule hypothèse à la fois.

1.9 Approfondissement : lire, construire et expliquer un programme

1.9.1 Chaînes : accéder, transformer et découper du texte

Une chaîne est une séquence immuable de caractères. On la construit avec des guillemets simples ou doubles ; les triples guillemets permettent plusieurs lignes. `len(text)` donne le nombre de caractères, `text[0]` le premier, `text[-1]` le dernier. Un indice hors limites lève `IndexError`, tandis qu'une tranche est tolérante : `text[:100]` s'arrête simplement à la fin.

La tranche `text[start:stop:step]` inclut `start` et exclut `stop`. Omettre une borne signifie partir du début ou aller jusqu'à la fin. Un pas négatif parcourt en sens inverse.

```
1 label = "  Data Science  "  
2 label[2:6]      # "Data"  
3 label[-3:]     # "e "  
4 label[::-1]    # nouvelle chaine inversee
```

Comme la chaîne est immuable, `label[0] = "d"` est interdit. Les méthodes renvoient une nouvelle chaîne ; elles ne modifient pas l'original.

Méthode	Rôle	Point de vigilance
<code>strip()</code> <code>lower()</code> , <code>upper()</code>	retire espaces aux extrémités change la casse	pas ceux du milieu normalisation linguistique limitée
<code>replace(a,b)</code> <code>split(sep)</code>	remplace des sous-chaînes produit une liste de morceaux	retourne une nouvelle chaîne séparateur absent ou répété
<code>sep.join(parts)</code>	assemble des chaînes	le séparateur est l'objet appelant
<code>startswith()</code> , <code>endswith()</code> <code>find(x)</code>	teste préfixe ou suffixe position ou <code>-1</code>	renvoie un booléen <code>index(x)</code> lève une erreur si absent
<code>isdigit()</code> , <code>isalpha()</code>	classe les caractères	Unicode, pas seulement ASCII

Exemple — parser une ligne simple

Pour " alice ; 42 ; Paris ", le contrat annonce trois champs séparés par un point-virgule. On découpe, retire les espaces, vérifie le nombre de champs puis convertit l'âge.

```
1 def parse_person(line):
2     parts = [part.strip() for part in line.split(";")]
3     if len(parts) != 3:
4         raise ValueError("expected name;age;city")
5     name, age_text, city = parts
6     return name, int(age_text), city
```

Les chaînes formatées `f"{name}: {score:.1f}"` séparent la valeur de sa présentation. Le format `:.1f` arrondit pour l'affichage, mais ne modifie pas la valeur numérique stockée. Cette distinction sera importante lorsque des résultats statistiques devront être affichés sans perdre leur précision interne.

1.9.2 Une spécification avant la syntaxe

Une consigne courte cache souvent plusieurs décisions. « Calculer une remise » ne dit pas si le prix peut être négatif, si le seuil est inclus, ni à quel moment arrondir. Avant Python, écrire une fiche minimale :

Rubrique	Question à résoudre
Entrées	Quelles valeurs reçoit-on ? Dans quelles unités ?
Sortie	Quelle valeur doit être produite, et de quel type ?
Domaine	Quelles entrées sont autorisées ?
Cas limites	Zéro, chaîne vide, égalité au seuil, borne inversée ?
Exemples	Quels résultats peut-on calculer sans programme ?

Cette fiche est déjà un modèle du problème. Elle sépare les décisions métier de la façon de les écrire. Un étudiant qui commence par le code risque de laisser Python choisir implicitement ce que l'énoncé n'a pas précisé.

Exemple — tarif de cinéma

Un billet coûte 12 euros, 8 euros si l'âge est strictement inférieur à 18 ans, et 10 euros à partir de 65 ans. Les catégories se chevauchent-elles ? Non, si l'on teste d'abord l'âge invalide, puis la jeunesse, puis l'âge senior. Les cas 17, 18, 64 et 65 révèlent les frontières mieux qu'un seul exemple « normal ».

```

1 def ticket_price(age):
2     if age < 0:
3         raise ValueError("age must be non-negative")
4     if age < 18:
5         return 8
6     if age >= 65:
7         return 10
8     return 12

```

1.9.3 La table de trace comme preuve locale

Lire une boucle signifie suivre son état, instruction après instruction. Pour calculer la plus grande température, on choisit d'abord une valeur initiale valide puis on énonce l'invariant informel : après chaque itération, `highest` est le maximum de la partie déjà lue.

```

1 temperatures = [18, 21, 19, 24]
2 highest = temperatures[0]
3 for value in temperatures[1:]:
4     if value > highest:
5         highest = value

```

itération	valeur lue	highest avant	highest après
1	21	18	21
2	19	21	21
3	24	21	24

La table révèle aussi le problème de la liste vide : il n'existe pas de première valeur. Ce n'est pas un détail de Python mais une décision de contrat. On peut refuser la liste vide, retourner `None`, ou demander une valeur par défaut ; chacune de ces options exprime une sémantique différente.

1.9.4 Construire par incréments vérifiables

Pour un petit programme, une progression fiable est : faire fonctionner un exemple, nommer l'état, traiter une branche, ajouter une répétition, extraire une fonction, puis tester les frontières. Supposons que l'on compte les jours au-dessus d'un seuil. Commencer par la comparaison d'une seule valeur ; l'étendre ensuite à une liste avec un compteur ; seulement alors créer une fonction qui retourne le compteur.

```

1 def count_above(values, threshold):
2     count = 0
3     for value in values:
4         if value > threshold:
5             count += 1
6     return count

```

L'égalité mérite un test : « au-dessus » signifie `>` et non `>=`. Un nom précis, un exemple calculé à la main et un test de frontière protègent mieux qu'un long commentaire ajouté après coup.

1.9.5 Déboguer par catégories

Une erreur de syntaxe empêche Python de construire le programme ; une erreur à l'exécution survient sur un chemin particulier ; une erreur logique produit une valeur mais viole le contrat. La stratégie diffère :

1. lire le premier message d'erreur pertinent et sa ligne ;
2. réduire l'entrée jusqu'au plus petit cas qui échoue ;
3. annoncer la valeur attendue avant d'ajouter un affichage temporaire ;
4. inspecter l'état juste avant la première divergence ;
5. transformer le cas corrigé en test de non-régression.

L'objectif n'est pas d'essayer des modifications au hasard, mais de localiser la première hypothèse fausse. Cette discipline restera la même avec pandas ou un modèle de machine learning : on réduit, on mesure, on compare au contrat.

Point de vigilance — confondre sortie et affichage

`print` rend une valeur visible pour une personne ; `return` transmet une valeur à l'appelant. Une fonction de calcul qui ne fait qu'imprimer devient difficile à tester et impossible à composer dans un calcul plus grand.

Sources principales : Documentation officielle des types intégrés et tutoriel Python ; cours Python Education de Google pour la progression par opérations.

1.10 Travail appliqué

Sans exécuter Python, donner la valeur et le type de huit expressions : `7 / 2`, `7 // 2`, `7 % 2`, `"7" * 2`, `3 < 5`, `not True`, `int("12") + 1` et `len("data")`. Vérifier ensuite dans l'interpréteur et expliquer chaque différence.

Écrire `shipping_cost(weight, express)`. Le tarif comprend 4 euros fixes et 1,50 euro par kilogramme. L'option express ajoute 6 euros. Un poids nul ou négatif doit être refusé. Calculer les résultats attendus avant d'écrire le code.

Écrire une fonction qui renvoie le nombre de lettres, chiffres et espaces d'un texte. La chaîne vide doit produire trois zéros.

Écrire `sum_between(start, stop)` qui additionne tous les entiers de `start` à `stop` inclus. Traiter le cas où `start > stop`.

À partir d'une liste non vide, produire le nombre de mesures, la moyenne, le minimum, le maximum et le nombre de valeurs supérieures à un seuil. Construire d'abord une table de trace sur [18, 21, 19, 24].

1.11 À retenir

Un programme exécute des instructions dans un ordre précis. Les conditions séparent des cas ; les boucles font évoluer un état ; les fonctions donnent une interface à un calcul. La première preuve de compréhension reste la capacité à prédire et à expliquer l'état du programme sur un exemple.

Sources principales : [1], [4], [6]

Fonctions, collections et programmes fiables

Décomposer, représenter, parcourir et tester

2.1 Objectifs

Ce chapitre approfondit les fonctions et introduit les collections qui structurent la majorité des programmes de données. L'étudiant devra choisir une représentation, expliquer les effets d'une modification et construire des tests à partir d'un contrat.

2.2 Décomposition et abstraction

Une grande fonction mêlant lecture, calcul, affichage et écriture devient difficile à vérifier. La décomposition sépare des sous-problèmes. L'abstraction permet ensuite d'utiliser une fonction à partir de son contrat, sans relire son implémentation.

```
1 def net_amount(amount, tax_rate):
2     return amount * (1 - tax_rate)
3
4 def summarize_amounts(amounts, tax_rate):
5     cleaned = []
6     for amount in amounts:
7         if amount >= 0:
8             cleaned.append(net_amount(amount, tax_rate))
9     return cleaned
```

2.2.1 Portée et appels

Les paramètres sont des noms locaux. Une variable créée dans une fonction n'est pas directement disponible ailleurs. Cette règle permet de raisonner sur une fonction sans suivre tout le programme.

Exemple — retourner plusieurs résultats

```
1 def min_max(values):
2     return min(values), max(values)
3
4 low, high = min_max([4, 9, 2, 7])
```

La fonction renvoie un tuple. L'affectation décompose le résultat en deux noms.

2.3 Séquences : chaîne, liste et tuple

Une séquence possède un ordre, une longueur et des positions. La liste est mutable ; le tuple exprime une collection de positions qui ne seront pas remplacées. Un tuple peut représenter un enregistrement court, par exemple (`latitude`, `longitude`).

Structure	Bon usage	Propriété centrale	Exemple
Liste	collection ordonnée qui évolue	mutable	observations nettoyées
Tuple	enregistrement ou ordre fixe	positions non remplaçables	coordonnées
Chaîne	texte	séquence immuable	identifiant brut

2.3.1 Parcours expressif

`enumerate` fournit position et valeur. `zip` avance dans plusieurs itérables en parallèle. Le déballage donne un nom à chaque composante.

```

1 names = ["Ada", "Linus", "Guido"]
2 scores = [18, 15, 17]
3
4 for rank, (name, score) in enumerate(zip(names, scores), start=1):
5     print(rank, name, score)
```

2.3.2 Compréhensions après la boucle explicite

Une compréhension convient à une transformation ou un filtre simple. Elle ne doit pas cacher plusieurs décisions ni des effets de bord.

```

1 squares_loop = []
2 for x in range(6):
3     if x % 2 == 0:
4         squares_loop.append(x * x)
5
6 squares = [x * x for x in range(6) if x % 2 == 0]
```

2.4 Dictionnaires et ensembles

Un dictionnaire associe une clé unique à une valeur. Il modélise naturellement un enregistrement nommé, un index ou un compteur. Un ensemble modélise l'appartenance et l'unicité.

Exemple — compter des catégories

```

1 events = ["open", "click", "open", "purchase", "click", "open"]
2 counts = {}
3 for event in events:
4     counts[event] = counts.get(event, 0) + 1
```

Après chaque tour, `counts` contient les fréquences exactes du préfixe déjà lu. Le même raisonnement réapparaîtra dans `groupby` au chapitre 5.

Une clé doit être hachable et conserver une valeur stable. Les nombres, chaînes et tuples de valeurs hachables conviennent ; une liste ne convient pas comme clé. `collections.Counter` et

`defaultdict` fournissent des versions spécialisées de motifs fréquents, mais comprendre la boucle explicite reste indispensable.

2.5 Mutabilité, alias et copie

L'affectation d'une liste à un second nom ne copie pas la liste.

```
1 original = [[1, 2], [3, 4]]
2 alias = original
3 shallow = original.copy()
4
5 alias.append([5, 6])      # change original
6 shallow[0].append(99)    # change aussi original[0]
```

La copie superficielle recrée seulement le conteneur externe. Les listes internes restent partagées. Il faut dessiner les relations entre noms et collections avant de raisonner sur une structure imbriquée.

Point de vigilance — modifier pendant un parcours

Ajouter ou supprimer des éléments dans la collection parcourue peut sauter des éléments ou prolonger la boucle. Construire une nouvelle liste rend souvent le contrat plus clair. Une modification en place doit être annoncée.

2.6 Erreurs, exceptions et tests

Une exception distingue un résultat valide d'une situation où le contrat ne peut pas être respecté. On lève `ValueError` pour une valeur hors domaine et on intercepte seulement les erreurs que l'on sait traiter.

```
1 def parse_positive_int(text):
2     try:
3         value = int(text)
4     except ValueError as error:
5         raise ValueError("expected an integer") from error
6     if value <= 0:
7         raise ValueError("expected a positive integer")
8     return value
```

2.6.1 Partitions de tests

Le test boîte noire vient du contrat. Pour `parse_positive_int`, les partitions sont : entier positif, zéro, entier négatif, texte non numérique. Une frontière est souvent plus informative qu'une grande collection de valeurs aléatoires.

```
1 assert parse_positive_int("7") == 7
2 for bad in ["0", "-2", "seven"]:
3     try:
4         parse_positive_int(bad)
5     except ValueError:
6         pass
7     else:
8         raise AssertionError("an invalid value was accepted")
```

2.7 Étude de cas : consolider des commandes

Chaque commande possède un client, une catégorie et un montant. On veut ignorer les lignes annulées, calculer le montant par client et signaler les identifiants répétés.

```
1 def consolidate_orders(rows):
2     totals = {}
3     seen_ids = set()
4     duplicate_ids = []
5
6     for row in rows:
7         order_id = row["order_id"]
8         if order_id in seen_ids:
9             duplicate_ids.append(order_id)
10        else:
11            seen_ids.add(order_id)
12
13        if row["status"] != "cancelled":
14            customer = row["customer"]
15            totals[customer] = totals.get(customer, 0.0) + row["amount"]
16
17    return totals, duplicate_ids
```

La liste conserve les lignes et leur ordre. L'ensemble répond à la question « déjà vu ? ». Le dictionnaire agrège les montants. Choisir une seule liste pour tout obligerait à rechercher plusieurs fois les clients et les identifiants.

2.8 Première lecture des coûts

Sans formaliser encore toutes les classes de complexité, on distingue :

- l'accès à une position de liste, qui ne parcourt pas la liste ;
- la recherche d'une valeur dans une liste, qui peut examiner tous les éléments ;
- l'appartenance à un ensemble ou la recherche d'une clé, rapide en moyenne ;
- la mémoire supplémentaire des collections construites pendant le calcul.

Le chapitre 3 transformera ces observations en analyse asymptotique rigoureuse.

2.9 Approfondissement : choisir des représentations qui portent le sens

2.9.1 Listes : construction, accès et tranches

Une liste est une séquence ordonnée et mutable. Elle peut contenir des objets de types différents, même si des éléments homogènes donnent généralement un modèle plus simple. On la construit par littéral, par conversion d'un itérable ou progressivement.

```
1 empty = []
2 scores = [12, 18, 15]
3 letters = list("data")
4 zeros = [0] * 4
5 copy_of_scores = list(scores)
```

Les indices commencent à zéro ; les indices négatifs partent de la fin. Lire `scores[3]` lorsque la longueur vaut trois lève `IndexError`. Une tranche `scores[1:3]` construit une nouvelle liste contenant les positions 1 et 2. La forme générale `start:stop:step` suit la même convention que `range` : la borne `stop` est exclue.

Les opérateurs de séquence sont également disponibles : `+` concatène sans modifier les opérandes, `*` répète, `in` teste l'appartenance, `len` mesure la longueur, `min/max/sum` réduisent des valeurs compatibles. Attention à `[[0] * 3] * 4` : la répétition duplique des références vers la même liste interne. Une compréhension crée au contraire quatre listes indépendantes.

```
1 bad_grid = [[0] * 3] * 4
2 bad_grid[0][0] = 9      # change la première case de chaque ligne
3
4 good_grid = [[0] * 3 for _ in range(4)]
5 good_grid[0][0] = 9     # change une seule ligne
```

2.9.2 Méthodes de liste : connaître effet, retour et erreur

Les méthodes principales se répartissent en ajout, retrait, recherche, ordre et copie. Celles qui modifient seulement la liste retournent `None` par convention. Ainsi, `scores = scores.append(20)` perd la liste : après l'appel, `scores` vaut `None`. Le motif correct appelle la méthode puis relit la même liste.

Opération	Effet ou résultat	Erreur / remarque
<code>append(x)</code>	ajoute un objet en fin	mutation, retourne <code>None</code>
<code>extend(iterable)</code>	ajoute chaque élément de l'itérable	différent de <code>append(list)</code>
<code>insert(i,x)</code>	insère avant la position <i>i</i>	décale les éléments suivants
<code>pop()</code>	retire et retourne le dernier	<code>IndexError</code> si vide
<code>pop(i)</code>	retire et retourne la position <i>i</i>	décalage après <i>i</i>
<code>remove(x)</code>	retire la première valeur égale à <i>x</i>	<code>ValueError</code> si absente
<code>clear()</code>	retire tous les éléments	conserve le même objet liste
<code>index(x[,a,b])</code>	indice de la première occurrence	<code>ValueError</code> si absente
<code>count(x)</code>	nombre d'occurrences	parcourt la liste
<code>sort(key=..., reverse=...)</code>	trie sur place	retourne <code>None</code>
<code>reverse()</code>	inverse sur place	retourne <code>None</code>
<code>copy()</code>	nouvelle liste superficielle	éléments internes partagés

`append(x)` ajoute un seul objet : si *x* est une liste, elle devient un élément imbriqué. `extend(x)` parcourt *x* et ajoute ses éléments. Cette différence est visible dans la longueur et le grain de la collection.

```
1 values = [1, 2]
2 values.append([3, 4])    # [1, 2, [3, 4]]
3 values = [1, 2]
4 values.extend([3, 4])    # [1, 2, 3, 4]
```

L'affectation de tranche peut remplacer, supprimer ou insérer plusieurs éléments. Elle modifie la liste existante et peut changer sa longueur.

```
1 letters = ["a", "b", "c", "d"]
2 letters[1:3] = ["B", "C", "X"]
3 del letters[-1]
```

Utiliser `del` lorsque l'on connaît une position et que la valeur retirée n'est pas nécessaire ; `pop` lorsque la valeur doit être récupérée ; `remove` lorsque la première valeur égale est connue mais pas sa position.

2.9.3 Trier sans confondre mutation et résultat

`list.sort()` modifie la liste ; `sorted(iterable)` retourne une nouvelle liste et accepte tout itérable. Les deux offrent `key` et `reverse`. La clé est calculée pour chaque élément et définit la valeur de comparaison.

```
1 rows = [
2     {"city": "Lyon", "delay": 12},
3     {"city": "Paris", "delay": 5},
4     {"city": "Lyon", "delay": 3},
5 ]
6 ordered = sorted(rows, key=lambda row: (row["city"], row["delay"]))
```

Le tri Python est stable : deux éléments ayant la même clé conservent leur ordre relatif. Cette propriété autorise des tris successifs du critère secondaire vers le critère principal, même si une clé tuple est souvent plus lisible.

2.9.4 Liste comme pile, mais pas comme file

Avec `append` et `pop()`, une liste implémente une pile dernier entré-premier sorti. En revanche, `pop(0)` ou `insert(0,x)` déplacent les autres éléments. Pour une file premier entré-premier sorti, `collections.deque` fournit `append`, `appendleft`, `pop` et `popleft` aux extrémités. Le choix de structure vient ici directement des opérations dominantes.

2.9.5 Tuples : enregistrements, déballage et immutabilité

Le tuple se construit par des valeurs séparées par des virgules. Les parenthèses aident la lecture, mais la virgule crée réellement le tuple. Le tuple vide vaut `()` et le singleton exige une virgule : `("Paris",)`. Sans elle, `("Paris")` est une chaîne entre parenthèses.

```
1 point = (48.8566, 2.3522)
2 latitude, longitude = point
3 first, *middle, last = [10, 20, 30, 40]
```

Le déballage vérifie le nombre de positions, sauf lorsqu'une cible étoilée capture le reste. Il rend les rôles plus explicites que `point[0]` et `point[1]`.

Un tuple interdit le remplacement de ses positions, mais peut contenir un objet mutable. Dans `([1,2], "A")`, on ne peut pas remplacer la première position, mais on peut modifier la liste qu'elle désigne. Un tuple est hachable seulement si tous ses éléments pertinents le sont. Les méthodes `count` et `index` sont disponibles comme pour les séquences, mais aucun ajout ou retrait n'existe.

2.9.6 Dictionnaires : construction et accès explicite

Un dictionnaire associe des clés uniques à des valeurs. Les clés doivent être hachables ; les valeurs peuvent être quelconques. Depuis Python 3.7, l'ordre d'insertion fait partie du comportement

du langage, mais un dictionnaire reste choisi pour l'association par clé, pas comme substitut universel à une liste.

```

1 customer = {"id": 17, "city": "Lille"}
2 customer["segment"] = "retail"      # ajout
3 customer["city"] = "Roubaix"       # remplacement
4 city = customer["city"]             # exige la cle
5 country = customer.get("country", "FR")

```

`d[key]` lève `KeyError` si l'absence viole le contrat. `d.get(key, default)` convient lorsque l'absence est normale et qu'une valeur par défaut a un sens. Tester `key in d` porte sur les clés. Écraser une clé existante ne crée pas une seconde paire ; cela remplace sa valeur.

Opération	Rôle	Remarque
<code>keys()</code> , <code>values()</code> , <code>items()</code>	vues dynamiques des clés, valeurs, paires	itérables
<code>get(k, default)</code>	valeur ou défaut sans insertion	pas de <code>KeyError</code>
<code>setdefault(k, default)</code>	retourne la valeur et insère le défaut si absent	mutation possible
<code>update(other)</code>	ajoute ou remplace plusieurs paires	collision écrasée
<code>pop(k[, default])</code>	retire une clé et retourne sa valeur	erreur sans défaut
<code>popitem()</code>	retire la dernière paire insérée	erreur si vide
<code>copy()</code>	copie superficielle	valeurs partagées
<code>clear()</code>	retire toutes les paires	même objet

Les vues de dictionnaire ne sont pas des listes figées : si le dictionnaire change, la vue reflète le changement. Pour conserver un instantané, utiliser `list(d.items())` ou une copie adaptée. Pendant un parcours, éviter de changer le nombre de clés du même dictionnaire ; parcourir une copie des clés si une suppression est nécessaire.

Exemple — regrouper des lignes

`setdefault` permet d'initialiser une liste au premier passage. `defaultdict` exprime le même motif plus directement lorsque l'initialisation est systématique.

```

1 by_city = {}
2 for row in rows:
3     by_city.setdefault(row["city"], []).append(row)
4
5 from collections import defaultdict
6 by_city = defaultdict(list)
7 for row in rows:
8     by_city[row["city"]].append(row)

```

2.9.7 Ensembles : appartenance et algèbre ensembliste

Un ensemble mutable se construit avec `set()` ou des accolades non vides. `{}` crée un dictionnaire vide, pas un ensemble. Les doublons sont éliminés et aucun ordre métier ne doit être déduit de l'affichage ou du parcours.

```

1 active = {"A", "B", "C"}
2 contacted = {"B", "D"}
3
4 active & contacted      # intersection: {"B"}

```

```

5 active | contacted      # union
6 active - contacted     # difference: actifs non contactés
7 active ^ contacted     # difference symétrique

```

`add(x)` ajoute un élément ; `update(iterable)` en ajoute plusieurs. `remove(x)` lève `KeyError` si `x` est absent, tandis que `discard(x)` ne fait rien. `pop()` retire un élément arbitraire et ne doit pas être utilisé pour simuler une file ordonnée. `clear()` vide l'ensemble.

Les relations `a <= b`, `a < b`, `a >= b` testent sous-ensemble et sur-ensemble ; `isdisjoint` vérifie qu'aucun élément n'est commun. `frozenset` est immuable et hachable : il peut servir de clé lorsque l'ordre n'a pas de sens, par exemple pour identifier une combinaison de catégories.

2.9.8 Itérer avec l'information réellement nécessaire

Parcourir directement les valeurs évite un indice artificiel. Utiliser `enumerate` si la position est une donnée, `zip` pour aligner plusieurs itérables, et `dict.items()` pour obtenir clé et valeur. Par défaut, parcourir un dictionnaire produit ses clés.

```

1 for position, score in enumerate(scores, start=1):
2     print(position, score)
3
4 for customer_id, total in totals.items():
5     print(customer_id, total)
6
7 for name, score in zip(names, scores, strict=True):
8     print(name, score)

```

`zip(..., strict=True)` transforme une différence de longueur en erreur plutôt que de tronquer silencieusement. C'est un bon choix lorsque les séquences sont supposées alignées par contrat.

2.9.9 Partir des opérations dominantes

Une collection se choisit moins par habitude que par les questions que le programme lui pose. Une liste répond bien à « quel élément est en position *i* ? » ; un ensemble à « cette valeur a-t-elle déjà été vue ? » ; un dictionnaire à « quelle information est associée à cette clé ? ». Un tuple signale qu'un petit enregistrement positionnel ne doit pas être modifié. Cette sémantique précède les performances.

Besoin	Représentation	Opération centrale	Risque à surveiller
ordre et doublons	liste	parcours, index	recherche répétée
unicité	ensemble	appartenance	ordre métier perdu
association	dictionnaire	accès par clé	clé absente ou dupli- quée
enregistrement fixe	tuple	décomposition	positions peu expli- cites

Le même jeu de données peut nécessiter plusieurs représentations simultanées. Une déduplication stable utilise une liste pour la sortie et un ensemble pour la mémoire des valeurs déjà rencontrées. Ce n'est pas une redondance accidentelle : chaque structure porte un invariant différent.

2.9.10 Références, alias et frontières de mutation

En Python, une variable désigne un objet. L'affectation d'une liste à un second nom ne copie pas son contenu : les deux noms désignent le même objet. Une copie superficielle crée un nouveau conteneur, mais les objets internes restent partagés.

```

1 original = [{"id": 1, "tags": ["new"]}
2 shallow = original.copy()
3 shallow[0]["tags"].append("priority")
4 assert original[0]["tags"] == ["new", "priority"]

```

Pour raisonner correctement, dessiner les objets et les flèches plutôt que deux « boîtes de variables » indépendantes. La question utile est : quelle opération crée un nouvel objet, et laquelle modifie l'objet existant ? Cette distinction explique de nombreux bugs de préparation de données.

Une fonction qui mute son argument doit l'annoncer dans son contrat et, idéalement, porter un nom qui le suggère. Pour une transformation, retourner une nouvelle valeur rend souvent la chaîne d'opérations plus facile à tester. Cela ne signifie pas que toute mutation est mauvaise : une accumulation locale dans une fonction courte est claire et efficace. Le danger vient surtout d'une mutation partagée et invisible.

2.9.11 Une fonction comme frontière de raisonnement

Une bonne fonction réduit ce que l'on doit garder en tête. Son contrat précise les préconditions, la valeur retournée, les erreurs attendues et les effets. Les annotations de type documentent ce contrat sans remplacer les contrôles à l'exécution.

```

1 def normalize_labels(labels: list[str]) -> list[str]:
2     """Return non-empty, stripped, lowercase labels in input order."""
3     result = []
4     for label in labels:
5         cleaned = label.strip().lower()
6         if cleaned:
7             result.append(cleaned)
8     return result

```

Cette fonction possède une propriété simple : la longueur du résultat ne dépasse jamais celle de l'entrée et chaque valeur retournée est non vide. Les propriétés sont souvent plus puissantes qu'une liste d'exemples, car elles décrivent toute une famille de cas.

2.9.12 Concevoir les tests depuis le contrat

Un test n'est pas seulement une paire entrée-sortie heureuse. Une partition utile pour `normalize_labels` contient : entrée vide, une valeur normale, espaces seuls, variantes de casse, accents, ordre et absence de mutation de l'entrée. Pour un dictionnaire indexé par identifiant, tester aussi une clé manquante et un doublon.

```

1 def test_normalize_labels_preserves_input():
2     labels = [" New ", "", "VIP"]
3     before = labels.copy()
4     assert normalize_labels(labels) == ["new", "vip"]
5     assert labels == before

```

La seconde assertion vérifie un effet absent, donc une partie importante du contrat. Plus tard, les contrôles de cardinalité d'une jointure pandas jouent le même rôle : ils vérifient ce qui ne doit pas changer.

2.9.13 Du Python natif aux opérations de données

Une agrégation par dictionnaire est le modèle mental d'un `groupby` : une clé identifie un groupe, un état résume les lignes déjà vues. Un ensemble matérialise une clé unique ; un dictionnaire maté-

rialise un index ; `zip` aligne des séquences sous une hypothèse de position commune. Comprendre ces mécanismes permet de ne pas traiter pandas comme une suite de recettes opaques.

Exemple — moyenne par produit sans bibliothèque

Deux dictionnaires suffisent : l'un pour les sommes, l'autre pour les effectifs. La moyenne n'est calculée qu'à la fin afin de ne pas perdre l'information nécessaire à la mise à jour.

```
1 totals, counts = {}, {}  
2 for product, amount in sales:  
3     totals[product] = totals.get(product, 0.0) + amount  
4     counts[product] = counts.get(product, 0) + 1  
5 means = {p: totals[p] / counts[p] for p in totals}
```

Sources principales : Tutoriel et types intégrés de Python ; Google Python Education, « Python Lists » ; *Fluent Python*, chapitres sur séquences et mappings.

2.10 Travail appliqué

À partir d'une liste de chaînes, retirer les espaces, ignorer les chaînes vides et renvoyer les valeurs dans l'ordre d'origine. Écrire d'abord une boucle, puis une compréhension lisible.

Normaliser une phrase en minuscules, découper les mots et produire un dictionnaire de fréquences. Définir précisément le traitement de la ponctuation.

Renvoyer la première occurrence de chaque identifiant en conservant l'ordre. Comparer une solution fondée uniquement sur une liste à une solution utilisant un ensemble des valeurs vues.

Construire un index `customer_id -> position` pour une liste d'enregistrements. Refuser une clé dupliquée et écrire les tests des cas vide, normal et dupliqué.

2.11 À retenir

Les collections ne sont pas interchangeables : leur sémantique et leurs opérations déterminent la qualité du programme. Une fonction fiable possède un contrat testable, limite ses effets et rend ses erreurs explicites.

Sources principales : [1], [4], [3], [5], [6], [7]

Algorithmes élémentaires et complexité

Évaluer le temps, la mémoire et les compromis de conception

3.1 Objectifs

L'étudiant devra définir la taille d'une entrée, analyser temps et espace auxiliaire, identifier l'opération dominante, comparer une solution directe à une solution mieux indexée et reconnaître quelques motifs simples utiles aux données.

3.2 De la correction à l'efficacité

La correction vient en premier : un programme rapide qui répond à une autre question reste faux. Une fois le contrat satisfait, on étudie sa capacité à traiter des entrées plus grandes. Chronométrer dépend de la machine, de l'interpréteur et des données. Le comptage d'opérations et l'ordre de croissance donnent un modèle plus transportable.

Taille de l'entrée

La variable de complexité doit décrire ce qui grandit. Pour une liste, on utilise souvent $n = \text{len}(L)$. Pour deux listes indépendantes, on conserve n et m . Pour une matrice, les dimensions r et c peuvent toutes deux compter.

3.2.1 Temps et espace

La complexité temporelle compte les opérations dominantes. La complexité spatiale auxiliaire mesure la mémoire supplémentaire au pic d'exécution. Elle exclut l'entrée déjà reçue et distingue la sortie obligatoire des structures créées pour accélérer le calcul.

Exemple — sortie contre mémoire auxiliaire

Une fonction qui copie les n nombres positifs produit une sortie pouvant atteindre $O(n)$. Si elle n'utilise qu'un compteur en plus de cette sortie, son espace *auxiliaire* est $O(1)$, tandis que son empreinte totale supplémentaire incluant la sortie est $O(n)$. Il faut annoncer la convention utilisée.

3.3 Classes de croissance

Classe	Nom	Exemple	Conséquence quand n double
$O(1)$	constante	accès <code>values[i]</code>	coût similaire
$O(\log n)$	logarithmique	recherche binaire	une étape supplémentaire environ
$O(n)$	linéaire	somme d'une liste	travail doublé
$O(n \log n)$	quasi linéaire	tri par comparaison	un peu plus que doublé
$O(n^2)$	quadratique	toutes les paires	travail multiplié par quatre

Les constantes comptent en pratique, mais elles ne changent pas la classe de croissance. $3n + 20$ est $O(n)$. Une analyse doit cependant conserver des paramètres indépendants : parcourir deux entrées coûte $O(n + m)$, pas automatiquement $O(n)$.

3.3.1 Règles de composition

Des blocs successifs additionnent leurs coûts et le terme dominant reste. Deux boucles successives sur n éléments donnent $O(n) + O(n) = O(n)$. Deux boucles imbriquées qui parcourent chacune tous les éléments donnent $O(n^2)$. Une boucle interne qui avance un pointeur sans jamais revenir en arrière peut, elle, rester linéaire.

3.3.2 Cas favorable, moyen et défavorable

Une recherche linéaire trouve parfois la cible au premier emplacement, mais peut examiner toute la liste. On annonce généralement le pire cas, sauf si une autre convention est explicitement pertinente. Pour dictionnaires et ensembles, on parle de coût moyen $O(1)$ sous le fonctionnement habituel du hachage.

3.4 Coût des opérations Python

Opération	Temps usuel	Remarque
<code>L[i], len(L)</code>	$O(1)$	accès direct
<code>x in L</code>	$O(n)$	scan possible de toute la liste
<code>L.append(x)</code>	$O(1)$ amorti	réallocation occasionnelle
<code>L.insert(0, x)</code>	$O(n)$	déplacement des références
<code>sorted(L)</code>	$O(n \log n)$	nouvelle liste
<code>k in d, x in s</code>	$O(1)$ moyen	mémoire de table de hachage

L'analyse amortie de `append` répartit le coût de rares agrandissements sur une suite d'ajouts. Pour une file, `collections.deque` évite les insertions coûteuses au début d'une liste.

3.5 Recherche linéaire et recherche binaire

La recherche linéaire n'impose aucun ordre. Son pire cas est $O(n)$ et son espace auxiliaire $O(1)$. La recherche binaire exige une séquence triée et élimine la moitié de l'intervalle à chaque étape.

```

1 def binary_search(values, target):
2     left = 0
3     right = len(values) - 1
4     while left <= right:
```

```

5     middle = (left + right) // 2
6     if values[middle] == target:
7         return middle
8     if values[middle] < target:
9         left = middle + 1
10    else:
11        right = middle - 1
12    return None

```

L'intervalle encore possible est `[left, right]`. Chaque tour le réduit. Le temps est $O(\log n)$ et l'espace auxiliaire $O(1)$ dans cette version itérative. Trier une seule fois peut être rentable si de nombreuses recherches suivent : $O(n \log n + q \log n)$ remplace alors $O(qn)$.

3.6 Hachage : supprimer une recherche répétée

Considérons la détection d'un doublon. Comparer toutes les paires utilise peu de mémoire mais coûte $O(n^2)$. Mémoriser les valeurs déjà rencontrées transforme la recherche intérieure en appartenance moyenne constante.

```

1 def contains_duplicate(values):
2     seen = set()
3     for value in values:
4         if value in seen:
5             return True
6         seen.add(value)
7     return False

```

Temps : $O(n)$ en moyenne. **Espace auxiliaire :** $O(n)$ au pire pour `seen`.

3.6.1 Deux valeurs pour une cible

Pour chaque valeur x , on cherche $target - x$. La solution exhaustive compare les paires. La solution indexée mémorise valeurs et positions déjà vues.

```

1 def two_sum(values, target):
2     position = {}
3     for index, value in enumerate(values):
4         complement = target - value
5         if complement in position:
6             return position[complement], index
7         position[value] = index
8     return None

```

Après le traitement de la position i , `position` indexe les valeurs du préfixe $0 \dots i$. L'ordre de la vérification évite de réutiliser le même élément. **Temps :** $O(n)$ moyen. **Espace auxiliaire :** $O(n)$.

3.7 Motifs simples pour des données ordonnées

3.7.1 Deux pointeurs

Sur une liste triée, deux indices peuvent encadrer une somme. Si la somme est trop petite, avancer l'indice gauche est la seule décision susceptible de l'augmenter. Chaque indice avance au plus n fois : le coût total reste $O(n)$ après le tri.

3.7.2 Fenêtre glissante de largeur fixe

Deux fenêtres consécutives de largeur k partagent $k - 1$ valeurs. Recalculer chaque somme coûte $O(nk)$. Soustraire la valeur sortante et ajouter la nouvelle donne $O(n)$ avec $O(1)$ d'espace auxiliaire hors sortie.

```

1 def moving_average(values, width):
2     if width <= 0 or width > len(values):
3         raise ValueError("invalid width")
4     total = sum(values[:width])
5     result = [total / width]
6     for right in range(width, len(values)):
7         total += values[right] - values[right - width]
8         result.append(total / width)
9     return result

```

3.7.3 Fréquence et regroupement

Une carte de fréquences réduit une séquence à un dictionnaire valeur $\rightarrow$ nombre. Un dictionnaire de listes réalise un regroupement. Ces motifs sont les mécanismes explicites derrière les agrégations pandas étudiées plus tard.

3.8 Diagnostic d'un goulot d'étranglement

Procédure d'analyse

1. Définir la taille de chaque entrée.
2. Repérer les opérations qui se répètent avec cette taille.
3. Calculer le temps et le pic de mémoire de chaque bloc.
4. Identifier le terme dominant et l'opération qui le cause.
5. Proposer un index, un pré-calcul, un tri unique ou un flux borné.
6. Vérifier que le nouveau contrat et les résultats restent identiques.

3.9 Étude de cas : analyser un journal d'événements

On reçoit des couples (user_id, action). On veut les fréquences d'actions, les utilisateurs uniques et la première position de chaque achat. Un seul parcours peut maintenir ces trois résumés.

```

1 def index_events(events):
2     action_counts = {}
3     users = set()
4     first_purchase = {}
5     for position, (user_id, action) in enumerate(events):
6         users.add(user_id)
7         action_counts[action] = action_counts.get(action, 0) + 1
8         if action == "purchase" and user_id not in first_purchase:
9             first_purchase[user_id] = position
10    return action_counts, users, first_purchase

```

Pour n événements et u utilisateurs, le temps moyen est $O(n)$ et la mémoire auxiliaire $O(u + a)$, où a est le nombre d'actions distinctes.

3.10 Approfondissement : établir et défendre un coût

3.10.1 Définir la taille de l'entrée

Une complexité n'a de sens qu'après avoir nommé les paramètres qui grandissent. Pour une liste, n peut être le nombre d'éléments. Pour une table, il faut souvent distinguer n lignes et p colonnes. Pour une jointure, n et m désignent les tailles des deux tables. Pour q requêtes sur un catalogue de n identifiants, le coût total peut être plus informatif que le coût d'une requête isolée.

Le temps mesure un nombre d'opérations abstraites ; l'espace auxiliaire mesure la mémoire créée par l'algorithme, en séparant si possible la sortie obligatoire. Une fonction qui retourne une copie de n valeurs exige déjà $O(n)$ pour sa sortie ; un second tableau temporaire de taille n ajoute $O(n)$ auxiliaire. Cette convention doit être annoncée, car deux analyses correctes peuvent sinon sembler contradictoires.

3.10.2 Sommes plutôt que slogans

Deux boucles ne signifient pas automatiquement $O(n^2)$. Deux parcours successifs coûtent $n + n = 2n$, donc $O(n)$. Une boucle de taille n contenant une boucle de taille n effectue environ n^2 opérations. Une boucle triangulaire effectue

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2},$$

donc reste quadratique. Une boucle qui divise la zone de recherche par deux effectue au plus environ $\log_2 n$ étapes.

Exemple — maximum puis nombre d'occurrences

Un premier parcours trouve le maximum ; un second compte ses occurrences. Le temps est $O(n) + O(n) = O(n)$ et l'espace auxiliaire $O(1)$. Fusionner les deux parcours peut réduire une constante, mais ne change pas la classe de croissance. Le bon choix dépend de la lisibilité et du volume réel, pas d'un réflexe « une seule boucle ».

3.10.3 Pire cas, cas moyen et coût amorti

La recherche d'une clé dans une table de hachage est $O(1)$ en moyenne, pas une promesse absolue. Les collisions et le redimensionnement existent. L'ajout en fin de liste est également amorti $O(1)$: certaines opérations déclenchent une réallocation coûteuse, mais le coût moyen d'une longue séquence d'ajouts reste constant par ajout. Une analyse sérieuse précise donc le régime utilisé.

Le pire cas reste essentiel lorsqu'une latence maximale ou un adversaire compte. Le cas moyen est utile lorsque le mécanisme de hachage et les données ordinaires rendent les pathologies improbables. Les mesures empiriques complètent l'analyse, mais ne la remplacent pas : un chronomètre sur $n = 1\,000$ ne révèle pas nécessairement le comportement à $n = 10^8$.

3.10.4 Prétraiter pour répondre à plusieurs requêtes

Supposons q questions d'appartenance sur n identifiants.

- scans répétés : $O(qn)$ temps, $O(1)$ espace auxiliaire ;
- tri unique puis dichotomies : $O(n \log n + q \log n)$ temps, copie éventuelle $O(n)$;
- ensemble : $O(n + q)$ temps moyen, espace $O(n)$.

Si $q = 1$ et la mémoire est rare, le scan peut être rationnel. Si les données sont déjà triées, la deuxième option devient attractive. Si les mises à jour sont fréquentes ou l'ordre doit être conservé, les contraintes changent encore. « Optimisé » signifie donc optimisé pour un usage et un budget précis.

3.10.5 Le tri comme investissement

Trier coûte généralement $O(n \log n)$, mais crée une structure exploitable par la recherche binaire, les deux pointeurs, les détections de doublons adjacents et certaines jointures. Le tri peut cependant muter l'entrée, consommer une copie et perdre l'ordre métier. Une solution par dictionnaire conserve parfois l'ordre et obtient un temps moyen linéaire au prix d'un index supplémentaire.

Fiche d'audit d'une solution

1. Nommer toutes les tailles : n , m , p , q , taille de fenêtre k .
2. Compter les parcours, les recherches internes, les tris et les allocations.
3. Écrire le temps et le pic mémoire, avec pire cas ou cas moyen explicite.
4. Isoler le terme dominant et le relier à une ligne ou une opération concrète.
5. Proposer une représentation qui supprime ce travail répété.
6. Vérifier correction, ordre, doublons, mutation et cas limites après modification.

3.10.6 Pont vers la science des données

Les mêmes motifs réapparaissent à plus grande échelle. Un `groupby` construit des groupes et maintient des résumés ; une jointure crée ou exploite un index ; une moyenne mobile réutilise une fenêtre ; une matrice dense occupe une mémoire proportionnelle à np ; une validation croisée répète plusieurs entraînements. L'utilisateur d'une bibliothèque ne voit pas toutes les boucles, mais reste responsable de la croissance du travail et de la mémoire.

Exemple — goulot caché dans une boucle pandas

Filtrer toute une table une fois par client ressemble à une boucle courte, mais répète un scan de n lignes pour chacun des u clients. Un regroupement construit les groupes une fois. Le diagnostic DSA fournit ici le modèle mental qui justifie `groupby`, au-delà de la simple mémorisation de sa syntaxe.

3.11 Travail appliqué

Résoudre *Contains Duplicate*. Présenter la comparaison de toutes les paires, le tri suivi d'un scan et l'ensemble des valeurs vues. Comparer temps, espace et mutation de l'entrée.

Étant donnés des prix et un budget, renvoyer les indices de deux prix distincts dont la somme vaut le budget. Fournir la solution exhaustive et la solution par dictionnaire.

Produire toutes les moyennes contiguës de largeur k . Comparer le recalcul des tranches à la mise à jour de la somme courante.

Un million d'identifiants reçoivent dix mille requêtes. Comparer scans répétés, tri et recherches binaires, puis construction d'un ensemble. Identifier les hypothèses qui feraient préférer chaque solution.

3.12 À retenir

Une optimisation défendable part d'un goulot identifié. Elle annonce le temps, la mémoire auxiliaire et les hypothèses. Les tables de hachage échangent souvent de la mémoire contre du temps ; l'ordre permet la recherche binaire et les pointeurs ; une fenêtre évite de recalculer des données contiguës.

Sources principales : [6], [3], [7], [8]

Conception objet pour les pipelines de données

Encapsuler un état, définir une interface et composer des étapes

4.1 Objectifs

L'étudiant devra distinguer classe et instance, définir un état valide, écrire des méthodes simples, utiliser une dataclass comme objet valeur et composer chargeurs, validateurs et transformateurs sans créer une hiérarchie inutile.

4.2 Quand une classe aide

Une fonction convient à une transformation sans état durable. Une classe devient utile lorsqu'un concept possède des données liées, des règles de validité et plusieurs opérations qui partagent cet état. Tout dictionnaire ne mérite pas une classe.

Objet comme abstraction de données

Un objet associe une représentation interne à une interface. La représentation répond à « quelles données décrivent l'objet ? ». L'interface répond à « quelles opérations les autres parties du programme peuvent-elles demander ? ».

4.3 Classe, instance et `self`

La classe est le plan d'un nouveau type. Chaque appel au constructeur crée une instance distincte. `self` désigne l'instance qui reçoit l'appel.

```
1 class TemperatureScale:
2     def __init__(self, offset, factor):
3         self.offset = offset
4         self.factor = factor
5
6     def convert(self, value):
7         return value * self.factor + self.offset
8
9 celsius_to_fahrenheit = TemperatureScale(offset=32, factor=9 / 5)
10 print(celsius_to_fahrenheit.convert(20))
```

Les attributs `offset` et `factor` forment l'état. La méthode `convert` utilise cet état. Une seconde instance peut représenter une autre conversion sans modifier la première.

4.4 Invariants et validation

Un invariant d'objet doit rester vrai après la construction et après chaque méthode. Pour un intervalle, `start <= end`. Pour un schéma, les noms de colonnes sont uniques. Il vaut mieux refuser tôt un état invalide que propager un objet incohérent.

```

1 class Interval:
2     def __init__(self, start, end):
3         if start > end:
4             raise ValueError("start must be <= end")
5         self.start = start
6         self.end = end
7
8     def duration(self):
9         return self.end - self.start
10
11    def overlaps(self, other):
12        return self.start <= other.end and other.start <= self.end

```

4.5 Dataclasses et objets valeur

Une dataclass réduit le code répétitif des objets principalement décrits par leurs champs. Elle produit notamment un constructeur, une représentation et une égalité.

```

1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class RecordIssue:
5     row: int
6     column: str
7     message: str

```

`frozen=True` empêche le remplacement normal des champs et rend l'intention d'objet valeur explicite. Les annotations décrivent l'interface pour les lecteurs et les outils ; elles ne valident pas les valeurs à l'exécution.

Point de vigilance — valeurs mutables par défaut

Un champ liste doit être créé pour chaque instance. Dans une dataclass, utiliser `field(default_factory=list)`. Dans une fonction ou un constructeur, utiliser `None` puis créer la liste localement. Une liste créée une seule fois comme valeur par défaut serait partagée.

4.6 Composition plutôt qu'héritage automatique

Un pipeline peut contenir des étapes qui partagent un petit protocole `transform`. La composition assemble des objets indépendants. Chaque étape reste testable seule.

```

1 class Pipeline:
2     def __init__(self, steps):
3         self.steps = list(steps)
4
5     def run(self, rows):
6         current = rows
7         for step in self.steps:

```

```

8         current = step.transform(current)
9     return current

```

Le pipeline ne demande pas la classe concrète d’une étape. Il demande seulement une méthode compatible. Ce polymorphisme structurel correspond au style naturel de Python. Une classe abstraite peut formaliser l’interface lorsque plusieurs équipes partagent une bibliothèque, mais elle n’est pas obligatoire dans un petit programme.

4.6.1 Fonctions ou objets étapes ?

Une fonction pure suffit lorsque l’étape n’apprend rien et n’a pas de configuration complexe. Un objet convient lorsqu’il possède une configuration, des statistiques apprises ou des dépendances. Cette distinction prépare les transformateurs scikit-learn, qui séparent `fit` et `transform`.

4.7 Frontières avec effets

La lecture de fichiers, la journalisation et l’heure courante sont des effets. On les place à la frontière. Le coeur transforme des données reçues en mémoire.

```

1 from pathlib import Path
2 import csv
3
4 class CsvLoader:
5     def load(self, path):
6         with Path(path).open(encoding="utf-8", newline="") as stream:
7             return list(csv.DictReader(stream))
8
9 def normalize_rows(rows):
10     return [
11         {key.strip().lower(): value.strip() for key, value in row.items()}
12         for row in rows
13     ]

```

Le gestionnaire de contexte ferme le fichier même en cas d’erreur. La fonction `normalize_rows` ne dépend pas du système de fichiers et se teste avec une liste construite dans le test.

4.8 Étude de cas : pipeline de sinistres

Le contrat : charger des lignes, vérifier les champs obligatoires, convertir le montant et conserver les lignes valides avec leurs erreurs.

```

1 from dataclasses import dataclass
2
3 @dataclass
4 class ValidationResult:
5     valid_rows: list
6     issues: list[RecordIssue]
7
8 class ClaimValidator:
9     required = {"claim_id", "amount", "country"}
10
11     def validate(self, rows):
12         valid = []
13         issues = []
14         for index, row in enumerate(rows, start=2):

```

```

15         missing = self.required - row.keys()
16         if missing:
17             issues.append(RecordIssue(index, "*", f"missing {sorted(missing)}"))
18             continue
19         try:
20             amount = float(row["amount"])
21         except ValueError:
22             issues.append(RecordIssue(index, "amount", "not numeric"))
23             continue
24         if amount < 0:
25             issues.append(RecordIssue(index, "amount", "negative"))
26             continue
27         valid.append(**row, "amount": amount)
28     return ValidationResult(valid, issues)

```

Cette classe a une responsabilité observable : elle transforme des lignes en résultat de validation. Elle ne lit pas le fichier et n'imprime pas les erreurs. Son test peut vérifier le nombre de lignes valides, les positions et les messages.

4.9 Tests de comportement

Tester une classe signifie tester son contrat public, pas recopier son implémentation.

```

1 def test_interval_rejects_reverse_bounds():
2     try:
3         Interval(5, 2)
4     except ValueError:
5         pass
6     else:
7         raise AssertionError("invalid interval accepted")
8
9 def test_pipeline_applies_steps_in_order():
10     class AddOne:
11         def transform(self, values):
12             return [value + 1 for value in values]
13
14     assert Pipeline([AddOne(), AddOne()]).run([1, 2]) == [3, 4]

```

4.10 Approfondissement : modéliser un pipeline comme un système de contrats

4.10.1 Objet, état et responsabilité

Un objet est utile lorsqu'une identité, un état valide et plusieurs opérations cohérentes doivent évoluer ensemble. Une classe ne sert pas à « ranger du code » : un module et des fonctions remplissent déjà ce rôle. Elle doit permettre de formuler une abstraction plus courte que sa mise en œuvre. Par exemple, un `SchemaValidator` peut promettre `validate(row) -> list[Issue]` sans exposer comment chaque règle est stockée.

Une responsabilité claire aide à reconnaître les mauvaises frontières. Si une classe charge un fichier, normalise des chaînes, entraîne un modèle, envoie un courriel et écrit des graphiques, chaque test exige tout le système. La décomposition recherche des composants qui changent pour des raisons différentes : accès aux données, règles de validation, transformation et orchestration.

4.10.2 Les invariants rendent les états impossibles

Un invariant est une propriété vraie après la construction et après chaque opération publique. Pour un compte : solde non négatif. Pour une spécification de colonne : nom non vide et type parmi un ensemble autorisé. Pour un modèle ajusté : paramètres appris présents avant `transform`.

```

1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class ColumnSpec:
5     name: str
6     dtype: str
7     nullable: bool = False
8
9     def __post_init__(self):
10         if not self.name.strip():
11             raise ValueError("column name must not be empty")
12         if self.dtype not in {"number", "string", "date"}:
13             raise ValueError("unsupported dtype")

```

Le constructeur refuse immédiatement un objet invalide. Cela réduit le nombre d'états que le reste du programme doit considérer. Une validation tardive déplace au contraire l'erreur loin de sa cause.

4.10.3 Dataclass, fonction ou classe ordinaire

Une dataclass convient à une donnée nommée dont l'égalité et la représentation sont importantes. Une fonction convient à un calcul sans mémoire entre appels. Une classe ordinaire convient lorsqu'il faut protéger des transitions d'état ou offrir plusieurs méthodes autour du même concept. Le choix n'est pas hiérarchique : la solution la plus petite qui exprime le contrat est généralement préférable.

Forme	Bon signal	Signal d'alerte
fonction	entrée vers sortie, pas d'état durable	variables globales cachées
dataclass	objet valeur, champs explicites	logique métier dispersée ailleurs
classe	invariants et transitions cohérentes	une seule méthode sans état
composition	étapes substituables	dépendances créées à l'intérieur

4.10.4 Composition et injection des dépendances

La composition construit un service à partir de collaborateurs. Un pipeline peut recevoir un chargeur et une liste de validateurs plutôt que de choisir lui-même un chemin de fichier et des classes concrètes. Cette injection rend les dépendances visibles et remplaçables en test.

```

1 class Pipeline:
2     def __init__(self, loader, validators, transform):
3         self.loader = loader
4         self.validators = list(validators)
5         self.transform = transform
6
7     def run(self, source):
8         rows = self.loader.load(source)
9         issues = [issue for row in rows
10                  for validator in self.validators
11                  for issue in validator.validate(row)]

```

```

12     if issues:
13         return {"data": None, "issues": issues}
14     return {"data": self.transform(rows), "issues": []}

```

Un faux chargeur en mémoire suffit alors pour tester l'orchestration. Le système réel peut utiliser CSV, base de données ou API sans modifier la logique de validation.

4.10.5 Interface commune et sous-typage comportemental

Deux composants sont substituables s'ils respectent le même contrat, pas simplement s'ils possèdent une méthode du même nom. Un validateur ne doit pas parfois lever une exception et parfois retourner une liste sans convention. Une interface peut rester informelle par duck typing, être documentée par un protocole de type, ou imposée par une classe abstraite. Introduire abc seulement lorsque plusieurs implémentations et une frontière stable justifient ce coût.

4.10.6 État appris dans un pipeline de données

La distinction entre `fit` et `transform` est un excellent exemple d'objet avec état. `fit` calcule une moyenne, un vocabulaire ou des paramètres sur les données d'entraînement; `transform` applique cet état sans le réapprendre.

```

1 class MeanImputer:
2     def __init__(self):
3         self.mean_ = None
4
5     def fit(self, values):
6         clean = [x for x in values if x is not None]
7         if not clean:
8             raise ValueError("cannot fit on empty data")
9         self.mean_ = sum(clean) / len(clean)
10        return self
11
12    def transform(self, values):
13        if self.mean_ is None:
14            raise RuntimeError("fit must be called first")
15        return [self.mean_ if x is None else x for x in values]

```

L'invariant change après `fit`. Cette séparation prépare directement la notion de fuite de données : réajuster l'imputeur sur le test ferait entrer le futur dans l'état.

4.10.7 Tester l'interface publique

Les tests doivent couvrir construction valide et invalide, transitions d'état, erreurs attendues, indépendance entre instances et substitution des collaborateurs. Éviter de tester un attribut privé ou l'ordre exact d'appels internes si le comportement public suffit. Un bon test survit à une réorganisation interne et échoue lorsque le contrat change réellement.

4.11 Travail appliqué

Concevoir une classe `Account` avec titulaire, solde, dépôt et retrait. Refuser un montant négatif et un retrait supérieur au solde. Énoncer les invariants.

Créer une dataclass immuable `ColumnSpec` contenant nom, type attendu et indicateur de valeur manquante autorisée. Comparer cette représentation à un tuple.

Définir des validateurs possédant `validate(row)` puis un validateur composé qui retourne toutes les erreurs. Aucun validateur ne doit lire ni écrire de fichier.

Séparer lecture CSV, normalisation des clés et validation. Écrire un test unitaire de chaque transformation et un test d'intégration sur un petit fichier temporaire.

4.12 À retenir

Une classe sert une abstraction, pas une préférence de style. Son état respecte des invariants et son interface limite ce que l'appelant doit connaître. Pour les pipelines, des composants courts, composés et testables sont généralement plus robustes qu'une hiérarchie profonde.

Sources principales : [2], [6], [7]

Deuxième partie

Python appliqué à la science des données

Calcul scientifique avec NumPy

Formes, types, mémoire, vectorisation et algèbre numérique

5.1 Question directrice

Un fichier de transactions contient montants, quantités, régions, dates et valeurs manquantes. Comment le transformer en table analytique sans perdre des lignes, dupliquer le chiffre d'affaires ou mélanger les unités ? La syntaxe de la bibliothèque vient après la définition du grain et des contrôles.

5.2 NumPy : un tableau numérique structuré

Un `ndarray` possède une forme, un nombre de dimensions et un type homogène. Contrairement à une liste de listes, il stocke les nombres dans une représentation compacte et expose des opérations portant sur des axes entiers.

```
1 import numpy as np
2
3 amounts = np.array([12.5, 8.0, 15.5, 4.0], dtype=np.float64)
4 print(amounts.shape)      # (4,)
5 print(amounts.dtype)     # float64
6 print(amounts.mean())
```

5.2.1 Vectorisation

Une opération vectorisée décrit le calcul sur le tableau complet. La boucle existe toujours, mais NumPy l'exécute dans du code compilé optimisé.

```
1 net = np.array([100.0, 120.0, 80.0])
2 tax_rate = 0.20
3 gross = net * (1 + tax_rate)
4 eligible = gross >= 100
5 selected = gross[eligible]
```

La vectorisation ne corrige pas une mauvaise formulation. Il faut toujours vérifier forme, type, unité et masque booléen.

5.2.2 Axes et réductions

Pour une matrice de forme `(n_rows, n_columns)`, `axis=0` réduit les lignes et conserve une valeur par colonne ; `axis=1` réduit les colonnes et conserve une valeur par ligne.

```

1 weekly = np.array([
2     [12, 15, 11],
3     [10, 14, 13],
4     [16, 12, 15],
5 ])
6 column_means = weekly.mean(axis=0) # trois produits
7 row_totals = weekly.sum(axis=1)    # trois semaines

```

5.2.3 Broadcasting comme algèbre des formes

Le broadcasting aligne les dimensions en partant de la droite. Deux dimensions sont compatibles si elles sont égales ou si l'une vaut 1.

```

1 sales = np.array([[10, 12, 8], [9, 15, 11]]) # (2, 3)
2 prices = np.array([4.0, 2.5, 6.0])          # (3,)
3 revenue = sales * prices                     # (2, 3)

```

La seconde dimension correspond ici aux produits. Une forme compatible peut produire un résultat sémantiquement faux si les axes ne représentent pas la même chose.

5.2.4 Vues, copies et type

Une tranche simple est souvent une vue qui partage la mémoire du tableau. La modifier peut changer le tableau d'origine. `copy()` est nécessaire lorsque l'indépendance fait partie du contrat. Le `dtype` contrôle précision, mémoire et valeurs possibles.

```

1 base = np.array([10, 20, 30, 40])
2 view = base[1:3]
3 view[0] = 999
4 assert base[1] == 999
5 independent = base[1:3].copy()

```

5.3 Pont vers pandas : une table avec un grain

Un `DataFrame` associe des colonnes nommées à un index. Avant toute opération, écrire : « une ligne représente ... ». Le grain détermine l'unicité attendue et la signification des agrégations.

```

1 import pandas as pd
2
3 orders = pd.DataFrame({
4     "order_id": [101, 102, 103, 104],
5     "customer": ["A", "B", "A", "C"],
6     "quantity": [2, 1, 3, 1],
7     "unit_price": [8.0, 12.0, 8.0, None],
8 })
9 print(orders.shape)
10 print(orders.dtypes)
11 print(orders.isna().sum())

```

5.3.1 Sélection, filtre et affectation explicite

`loc` sélectionne avec les étiquettes ; `iloc` avec les positions. Un masque booléen doit avoir un index compatible avec la table filtrée.

```

1 valid = orders.loc[orders["unit_price"].notna()].copy()
2 valid.loc[:, "revenue"] = valid["quantity"] * valid["unit_price"]
3 large = valid.loc[valid["revenue"] >= 20, ["order_id", "revenue"]]

```

Le `copy()` rend ici l'intention d'une table indépendante explicite. Après une transformation, contrôler le nombre de lignes, les types et quelques valeurs connues.

5.3.2 Valeurs manquantes

Une absence peut signifier non mesuré, non applicable, non reçu ou réellement nul. Supprimer ou imputer sans cette distinction change la question. L'audit commence par les taux d'absence globaux et par groupe.

```

1 missing_by_customer = (
2     orders.assign(missing_price=orders["unit_price"].isna())
3     .groupby("customer", as_index=False)
4     .agg(rows=("order_id", "size"), missing=("missing_price", "sum"))
5 )
6 missing_by_customer["rate"] = (
7     missing_by_customer["missing"] / missing_by_customer["rows"]
8 )

```

5.3.3 Groupby : séparer, calculer, recombinaison

Une agrégation doit nommer la mesure, la fonction et le niveau de sortie.

```

1 customer_summary = (
2     valid.groupby("customer", as_index=False)
3     .agg(
4         orders=("order_id", "nunique"),
5         revenue=("revenue", "sum"),
6         mean_basket=("revenue", "mean"),
7     )
8 )

```

`size` compte les lignes, `count` ignore les valeurs manquantes et `nunique` compte les valeurs distinctes. Les confondre produit des indicateurs différents.

5.4 Jointures sous contrat

Une jointure peut augmenter le nombre de lignes. Si une table de référence contient plusieurs lignes par clé, une jointure plusieurs-à-plusieurs multiplie les observations.

```

1 customers = pd.DataFrame({
2     "customer": ["A", "B", "C"],
3     "region": ["North", "South", "North"],
4 })
5
6 enriched = valid.merge(
7     customers,
8     on="customer",
9     how="left",
10    validate="many_to_one",
11    indicator=True,
12 )

```

```

13 assert len(enriched) == len(valid)
14 assert enriched["_merge"].eq("both").all()

```

Le contrat `many_to_one` exprime que plusieurs commandes peuvent appartenir à un client, mais qu'un client n'apparaît qu'une fois dans la dimension. L'indicateur révèle les clés non appariées.

5.5 Lien avec les structures de données

Un index pandas joue le rôle d'un index algorithmique. Une agrégation construit une carte clé-vers-résumé. Une jointure utilise des stratégies de hachage ou de tri. La vectorisation traite un bloc contigu plutôt qu'une succession d'objets Python. Les outils changent, mais les questions de coût et de représentation restent les mêmes.

5.6 Pipeline de préparation vérifiable

```

1 def prepare_orders(orders, customers):
2     required = {"order_id", "customer", "quantity", "unit_price"}
3     missing = required - set(orders.columns)
4     if missing:
5         raise ValueError(f"missing columns: {sorted(missing)}")
6
7     out = orders.copy()
8     out["unit_price"] = pd.to_numeric(out["unit_price"], errors="coerce")
9     out = out.loc[out["unit_price"].notna() & out["quantity"].gt(0)].copy()
10    out["revenue"] = out["quantity"] * out["unit_price"]
11    out = out.merge(customers, on="customer", how="left",
12                   validate="many_to_one", indicator=True)
13    if not out["_merge"].eq("both").all():
14        raise ValueError("unmatched customers")
15    return out.drop(columns="_merge")

```

Les contrôles entourent les transformations : schéma avant, domaine pendant, cardinalité de jointure et réconciliation après. Un pipeline robuste rend les hypothèses exécutables.

5.7 Approfondissement : raisonner sur formes, alignements et cardinalités

5.7.1 Calculer avec des formes, pas seulement avec des valeurs

Dans NumPy, la forme fait partie du type mental de l'objet. Si X contient n observations et p variables, alors $X \in \mathbb{R}^{n \times p}$. Un vecteur de poids $w \in \mathbb{R}^p$ produit $Xw \in \mathbb{R}^n$. Écrire ces formes avant le code permet de prédire les opérations valides et leur sortie.

```

1 X = np.array([[2.0, 1.0, 0.0],
2              [1.0, 3.0, 2.0]])    # (2, 3)
3 w = np.array([0.5, -1.0, 2.0])   # (3,)
4 b = 0.25                         # scalaire
5 scores = X @ w + b               # (2,)

```

Le broadcasting aligne les dimensions depuis la droite. Une matrice (n, p) accepte un vecteur $(p,)$ pour une opération colonne par colonne ; un vecteur $(n,)$ n'exprime pas automatiquement une opération ligne par ligne. Pour ce dernier besoin, une forme $(n, 1)$ rend l'axe explicite.

5.7.2 Vectorisation : modèle d'exécution et limites

La vectorisation déplace les boucles dans du code compilé et opère sur des blocs homogènes. Elle réduit le coût d'interprétation Python et permet des accès mémoire plus compacts, sans supprimer le travail mathématique. Xw reste proportionnel à np .

Créer de grands temporaires peut annuler le bénéfice. L'expression `((X - mean) / std) ** 2` crée potentiellement plusieurs tableaux. Pour des données qui ne tiennent pas en mémoire, traiter par lots ou utiliser une opération plus directe est parfois préférable. L'analyse temps-espace du chapitre 3 reste donc indispensable.

Les types numériques comptent également. Un million de valeurs `float64` occupe environ huit mégaoctets hors surcoûts ; en `float32`, environ quatre. Réduire le type sans vérifier domaine et précision peut toutefois provoquer débordement ou perte d'information. Toujours inspecter `dtype`, unités et valeurs extrêmes.

5.7.3 Vues, copies et mutation

De nombreuses tranches NumPy sont des vues sur la même mémoire. Modifier la tranche peut modifier l'original. Les indexations avancées produisent souvent une copie. Plutôt que mémoriser une liste de cas, vérifier `np.shares_memory` lors d'un doute et documenter les frontières où une copie est intentionnelle.

```
1 a = np.arange(6)
2 view = a[1:4]
3 view[0] = 99
4 assert a[1] == 99
5
6 copy = a[[1, 2, 3]]
7 copy[0] = -1
8 assert a[1] == 99
```

Le lien avec les alias Python est direct, mais le partage porte ici sur un tampon numérique potentiellement massif.

5.7.4 Le grain comme invariant d'une DataFrame

Une DataFrame ne signifie pas « une table » tant que chaque ligne n'a pas de sens précis. Une ligne peut représenter une commande, un article de commande, un client-jour ou une mesure de capteur. Le grain détermine la clé attendue, les agrégations légitimes et le sens des effectifs.

Avant transformation, écrire une phrase : « une ligne représente un article au sein d'une commande ». Dans ce grain, `len(df)` compte des articles, pas des commandes. Le nombre de commandes exige `nunique("order_id")`. Un panier moyen doit d'abord agréger les articles au niveau commande ; prendre la moyenne du montant des lignes répond à une autre question.

5.7.5 Alignement pandas : puissance et piège

pandas aligne les opérations par étiquette d'index. C'est utile pour combiner des séries dont l'ordre diffère, mais dangereux si l'on croit à un alignement purement positionnel.

```
1 price = pd.Series({"A": 10, "B": 20})
2 quantity = pd.Series({"B": 2, "A": 3})
3 revenue = price * quantity
4 # A -> 30, B -> 40 : alignement par étiquette
```

Après un filtre ou une concaténation, inspecter l'index. Utiliser `reset_index` seulement lorsque l'index précédent n'a plus de sens ; ne pas l'employer pour masquer un désalignement incompris.

5.7.6 Groupby comme réduction contrôlée

Une agrégation transforme un grain fin en grain plus grossier. Les clés de groupement définissent la sortie ; chaque mesure doit annoncer sa règle : somme, nombre de lignes, nombre d'entités distinctes, moyenne pondérée ou quantile. Les agrégations nommées rendent ce contrat visible.

```

1 order_level = (lines.assign(revenue=lines["price"] * lines["quantity"])
2                  .groupby(["region", "order_id"], as_index=False)
3                  .agg(order_revenue=("revenue", "sum"),
4                      customer_id=("customer_id", "first")))
5
6 summary = (order_level.groupby("region", as_index=False)
7            .agg(orders=("order_id", "nunique"),
8                customers=("customer_id", "nunique"),
9                mean_basket=("order_revenue", "mean")))
```

Les deux étapes empêchent de pondérer implicitement les commandes par leur nombre de lignes.

5.7.7 Jointures : prédire la cardinalité

Une jointure doit être précédée d'une hypothèse : un-à-un, plusieurs-à-un, un-à-plusieurs ou plusieurs-à-plusieurs. Si des transactions sont enrichies par une dimension client, la clé client doit être unique à droite. L'option `validate` transforme cette hypothèse en contrôle exécutable ; `indicator=True` rend visibles les lignes sans correspondance.

Réconciliation d'une transformation

Comparer avant et après : nombre de lignes, clés distinctes, taux de valeurs manquantes, sommes métier, domaines des catégories et quelques cas suivis par identifiant. Une différence n'est pas nécessairement une erreur, mais elle doit être expliquée par le contrat de l'opération.

5.7.8 Chaîne de préparation reproductible

Une préparation défendable conserve les données brutes, applique des étapes nommées et retourne à la fois données et rapport de contrôle. Chaque étape reçoit une table et produit une table au grain annoncé. Les dépendances temporelles sont explicites ; aucun nettoyage manuel dans un tableur ne se glisse entre deux cellules du notebook.

5.8 NumPy en profondeur : le tableau comme objet scientifique

5.8.1 Anatomie d'un ndarray

Un tableau NumPy sépare un tampon de données de ses métadonnées. Les éléments vivent dans une zone mémoire typée ; `shape` décrit la longueur de chaque axe, `strides` le nombre d'octets à sauter pour avancer d'une position sur chaque axe, et `dtype` interprète les bits. Deux tableaux peuvent ainsi exposer des formes différentes tout en partageant le même tampon.

```

1 a = np.arange(12, dtype=np.float64).reshape(3, 4)
2 print(a.ndim)      # 2
3 print(a.shape)     # (3, 4)
4 print(a.size)      # 12
5 print(a.itemsize)  # 8 bytes
6 print(a.nbytes)    # 96 bytes
7 print(a.strides)   # often (32, 8) in C order
```

Le nombre d'octets utiles est approximativement `size * itemsize`. Cette estimation devient un réflexe avant une opération susceptible de créer un tableau (n, m, p) . La contiguïté affecte les performances : parcourir la mémoire dans son ordre naturel exploite mieux les caches du processeur.

5.8.2 Construire sans ambiguïté

`np.array` copie ou convertit des données existantes ; `np.asarray` évite une copie lorsqu'un tableau compatible existe déjà. `zeros`, `ones`, `full` et `empty` allouent une forme. `arange` convient aux pas entiers ; pour un intervalle flottant avec un nombre de points fixé, `linspace` exprime mieux le contrat.

```
1 grid = np.linspace(0.0, 1.0, num=101, endpoint=True)
2 identity = np.eye(4, dtype=float)
3 buffer = np.empty((10_000, 8), dtype=np.float32)
```

`empty` ne met pas les valeurs à zéro : lire avant d'écrire est une erreur. Un tableau hétérogène peut être converti vers un type chaîne ou objet, détruisant l'avantage numérique. Après une construction externe, inspecter `shape`, `dtype` et quelques valeurs.

5.8.3 Choisir un type numérique

Les entiers ont un domaine fini ; les flottants représentent une grille de nombres avec arrondi. Une somme de nombreux `float32` peut perdre davantage de précision qu'une somme en `float64`. Un entier non signé ne peut représenter une valeur négative et peut déborder. Le type est une hypothèse sur domaine, précision et mémoire.

```
1 x = np.array([250, 10], dtype=np.uint8)
2 print(x + np.uint8(10)) # finite-range arithmetic
3
4 values = np.array([0.1, 0.2, 0.3])
5 print(values[0] + values[1] == values[2]) # usually False
6 print(np.isclose(values[0] + values[1], values[2]))
```

Tester les flottants avec une tolérance adaptée à l'échelle et au domaine. `isclose` n'est pas un rite automatique : les tolérances expriment la précision acceptable.

5.8.4 Reshape, transpose et invariants de taille

Un changement de forme conserve le nombre total d'éléments. `-1` demande à NumPy de déduire une dimension. Une transposition permute les axes ; pour plus de deux dimensions, écrire explicitement l'ordre souhaité.

```
1 flat = np.arange(24)
2 cube = flat.reshape(2, 3, 4)
3 features_first = np.transpose(cube, (2, 0, 1)) # (4, 2, 3)
4 back = features_first.reshape(4, -1)           # (4, 6)
```

La forme seule ne porte pas le nom des axes. Documenter : `(sample, time, feature)` ou `(height, width, channel)`. Deux tableaux `(32, 32, 3)` peuvent être compatibles et sémantiquement incompatibles si l'un encode couleur et l'autre répétitions.

5.8.5 Indexation de base et indexation avancée

Une tranche de base produit souvent une vue. Une sélection par liste d'indices ou masque booléen produit généralement une copie. La distinction modifie mémoire et effets de bord.

```

1 X = np.arange(20).reshape(5, 4)
2 block = X[1:4, :2]           # basic slice, usually a view
3 rows = X[[1, 3, 4]]         # advanced indexing, copy
4 mask = X[:, 0] % 2 == 0
5 selected = X[mask]          # copy

```

L'affectation masquée agit sur l'original : `X[mask, 0] = -1`. En revanche, `tmp = X[mask]`; `tmp[:, 0] = -1` modifie la copie. Pour des pipelines scientifiques, annoncer si une fonction transforme en place ou retourne un nouveau tableau.

5.8.6 Règles complètes de broadcasting

Comparer les formes depuis la droite. Chaque paire d'axes est compatible si les longueurs sont égales ou si l'une vaut 1 ; un axe absent agit comme 1. Le résultat prend la longueur maximale sur chaque axe.

Forme A	Forme B	Résultat
(n, p)	$(p,)$	(n, p)
$(n, 1)$	$(p,)$	(n, p)
(b, n, p)	$(1, n, 1)$	(b, n, p)
(n, p)	$(n,)$ avec $n \neq p$	incompatible

Ajouter un axe avec `None` ou `np.newaxis` rend l'intention explicite.

```

1 row_offsets = np.array([10.0, 20.0, 30.0])[:, None] # (3, 1)
2 measurements = np.ones((3, 4))                    # (3, 4)
3 adjusted = measurements + row_offsets              # (3, 4)

```

Le broadcasting conceptuel n'alloue pas nécessairement les répétitions, mais le résultat et certains intermédiaires sont bien alloués. Une différence de formes $(n, 1, p)$ et $(1, m, p)$ crée logiquement (n, m, p) : avec $n = m = 50\,000$, l'opération est impossible en mémoire malgré une ligne de code élégante.

5.8.7 Ufuncs, réductions et accumulation

Les fonctions universelles comme `np.exp`, `np.sqrt` ou `np.maximum` appliquent une opération élément par élément. Une réduction élimine un axe ; `keepdims=True` conserve un axe de longueur 1 et facilite le broadcasting inverse.

```

1 X = np.array([[1., 2., 3.], [4., 5., 6.]])
2 column_mean = X.mean(axis=0, keepdims=True) # (1, 3)
3 centered = X - column_mean                  # (2, 3)
4 row_max = X.max(axis=1, keepdims=True)      # (2, 1)
5 stable_exp = np.exp(X - row_max)
6 softmax = stable_exp / stable_exp.sum(axis=1, keepdims=True)

```

Soustraire le maximum avant l'exponentielle évite un débordement et illustre la stabilité numérique. Une formule mathématiquement équivalente peut être informatiquement meilleure.

5.8.8 Valeurs manquantes et valeurs non finies

NaN est un flottant particulier, pas un marqueur générique pour tous les types. Il n'est pas égal à lui-même. Utiliser `np.isnan` et `np.isfinite`, puis décider si l'absence doit être propagée, ignorée

ou signalée. Les fonctions `nanmean` et `nanquantile` ignorent les NaN, mais peuvent masquer une collecte défectueuse si elles sont utilisées sans audit.

```
1 sensor = np.array([18.2, np.nan, 19.1, np.inf])
2 finite = np.isfinite(sensor)
3 coverage = finite.mean()
4 mean_valid = sensor[finite].mean()
```

5.8.9 Algèbre linéaire : résoudre plutôt qu'inverser

Pour résoudre $Ax = b$, préférer `np.linalg.solve(A, b)` à `np.linalg.inv(A) @ b`. La résolution est plus directe, généralement plus stable et moins coûteuse. Pour un problème de moindres carrés, utiliser `np.linalg.lstsq`.

```
1 A = np.array([[3., 1.], [1., 2.]])
2 b = np.array([9., 8.])
3 x = np.linalg.solve(A, b)
4 assert np.allclose(A @ x, b)
5
6 beta, residuals, rank, singular = np.linalg.lstsq(X, y, rcond=None)
```

Le rang et les valeurs singulières aident à détecter une matrice mal conditionnée. Une solution numérique doit être validée par résidu et connaissance du domaine.

5.8.10 Génération aléatoire moderne

Créer un `Generator` explicite avec `default_rng`. L'objet contient l'état ; le passer à une fonction rend la dépendance visible et évite un état global caché.

```
1 def simulate_portfolio(n_scenarios, rng):
2     market = rng.normal(loc=.04, scale=.12, size=n_scenarios)
3     claims = rng.poisson(lam=2.5, size=n_scenarios)
4     return market - .01 * claims
5
6 rng = np.random.default_rng(2026)
7 result = simulate_portfolio(100_000, rng)
```

Une graine reproduit le flux dans un environnement donné, mais la documentation NumPy ne garantit pas que chaque suite binaire restera identique à travers toutes les versions. Enregistrer version et méthode lorsque la reproduction exacte est critique.

5.8.11 Vectoriser, fusionner ou traiter par blocs

Considérons les distances entre n observations et m centroïdes en dimension p . Le broadcasting intégral construit un intermédiaire (n, m, p) : temps $O(nmp)$ et espace $O(nmp)$. On peut exploiter

$$\|x - c\|^2 = \|x\|^2 + \|c\|^2 - 2x^\top c$$

pour construire seulement une matrice (n, m) , ou traiter les observations par blocs si cette matrice reste trop grande.

```
1 def nearest_centroid_blocked(X, C, block_size=4096):
2     c2 = np.sum(C * C, axis=1)
3     labels = np.empty(len(X), dtype=np.int64)
4     for start in range(0, len(X), block_size):
5         stop = min(start + block_size, len(X))
```

```

6     xb = X[start:stop]
7     distances2 = (np.sum(xb * xb, axis=1, keepdims=True)
8                   + c2[None, :] - 2 * xb @ C.T)
9     labels[start:stop] = np.argmin(distances2, axis=1)
10    return labels

```

La boucle externe n'est pas un échec de vectorisation : elle borne la mémoire tandis que chaque bloc utilise des noyaux numériques efficaces. Le bon objectif est le temps total sous contrainte de mémoire, pas l'absence absolue de boucle Python.

5.8.12 Mesurer correctement

Comparer des implémentations sur des tailles réalistes, après échauffement, en séparant allocation et calcul lorsque c'est pertinent. Vérifier d'abord l'égalité numérique avec `allclose`. Rapporter forme, dtype, temps et pic mémoire. Une accélération sur dix éléments n'a aucune valeur ; une solution rapide mais fausse non plus.

Checklist NumPy

Nommer les axes ; prédire la forme de sortie ; vérifier le dtype ; distinguer vue et copie ; estimer la taille des intermédiaires ; utiliser une formule stable ; tester sur un exemple manuel ; mesurer sur une taille réaliste ; documenter la politique de valeurs non finies.

5.9 Travail appliqué

À partir d'une matrice observations-variables et d'un vecteur de coefficients, calculer un score par observation. Résoudre d'abord avec des boucles, puis avec multiplication et somme NumPy. Vérifier les formes incompatibles.

Écrire une fonction retournant forme, types, valeurs manquantes, nombre de clés uniques et doublons d'une table. Définir le grain attendu comme paramètre.

Calculer chiffre d'affaires, nombre de commandes et panier moyen par région et mois. Expliquer la différence entre lignes, commandes distinctes et clients distincts.

Enrichir des transactions avec une table de clients. Détecter clés dupliquées dans la dimension, clients absents et augmentation inattendue du nombre de lignes.

Standardiser chaque colonne d'une matrice en conservant les colonnes constantes. Écrire les formes de tous les intermédiaires, proposer une version naïve puis une version vectorisée, vérifier le résultat et discuter les copies temporaires.

Pour n observations et m centroïdes de dimension p , calculer le centroïde le plus proche. Comparer triples boucles, broadcasting intégral et traitement par blocs. Analyser temps et espace; expliquer pourquoi la solution la plus courte n'est pas toujours la plus économe en mémoire.

5.10 À retenir

La préparation fiable repose sur les formes, types, unités, clés et cardinalités. Une opération concise n'est correcte que si son contrat peut être vérifié avant et après.

Sources principales : [9], [10], [11], [12], [7]

Ingénierie tabulaire et analyse exploratoire avec pandas

Schémas, index, transformations, agrégations et preuves descriptives

6.1 Question directrice

Un service de livraison semble plus lent. S'agit-il d'un déplacement général, de quelques incidents extrêmes, d'un changement de région ou d'un problème de qualité des données ? L'analyse exploratoire construit les preuves nécessaires avant de chercher une explication ou un modèle.

6.2 Le grain et la population

Une moyenne de durée n'a pas de sens tant que l'unité d'observation n'est pas définie. Une ligne peut représenter une commande, une tentative de livraison ou un client. Dupliquer les commandes avec plusieurs articles change leur poids dans les statistiques.

Audit avant résumé

Vérifier la période, la population incluse, le nombre de lignes, les clés uniques, les doublons, les types, les unités, les valeurs manquantes et les valeurs impossibles. Comparer ces quantités aux attentes métier avant de produire un graphique.

6.3 Décrire une variable numérique

Pour $x_1, \dots, x_n$, la moyenne est $\bar{x} = n^{-1} \sum_i x_i$. Elle utilise toutes les valeurs et réagit fortement aux extrêmes. La médiane partage les observations ordonnées en deux moitiés et résiste mieux à quelques grandes valeurs.

La variance empirique mesure la dispersion quadratique :

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

L'écart type s revient à l'unité d'origine. L'intervalle interquartile $IQR = Q_{0.75} - Q_{0.25}$ décrit la moitié centrale et résiste aux extrêmes.

Exemple — une moyenne trompeuse

Les durées [20, 21, 22, 23, 120] ont une moyenne de 41,2 minutes et une médiane de 22 minutes. La moyenne répond à la durée moyenne réellement observée ; la médiane décrit mieux une livraison typique. Aucune n'est « la bonne » sans question.

```

1 summary = deliveries["duration_min"].describe(
2     percentiles=[0.05, 0.25, 0.5, 0.75, 0.95]
3 )
4 robust = {
5     "median": deliveries["duration_min"].median(),
6     "iqr": deliveries["duration_min"].quantile(.75)
7         - deliveries["duration_min"].quantile(.25),
8 }

```

6.4 Forme, asymétrie et valeurs atypiques

Un histogramme montre modes, trous, queues et concentrations. Une boîte à moustaches résume médiane, quartiles et observations éloignées, mais ne montre pas toujours une multimodalité. Le coefficient d'asymétrie condense la dissymétrie sans remplacer le graphique.

La règle $[Q_1 - 1.5IQR, Q_3 + 1.5IQR]$ signale des observations à examiner. Elle ne prouve ni erreur ni fraude. Une livraison de 180 minutes peut être une panne réelle et porter une information opérationnelle importante.

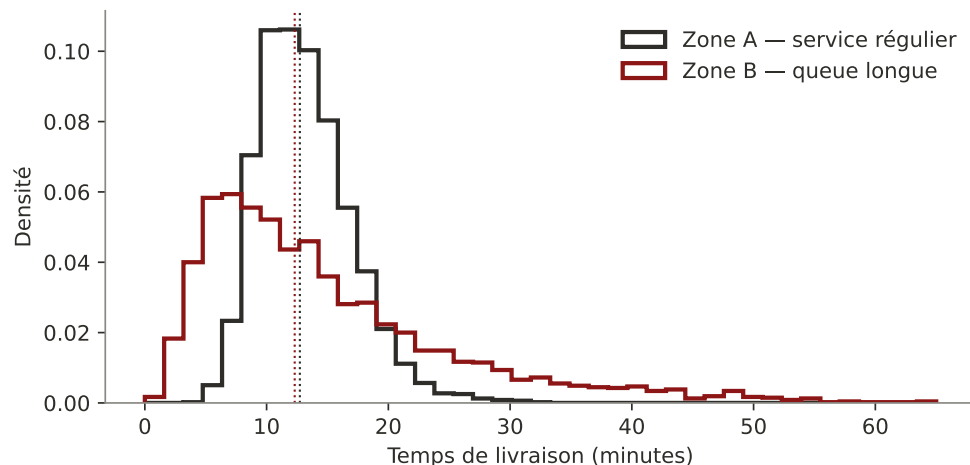


FIGURE 6.1 – Deux résumés d’une distribution de durées : la queue droite influence la moyenne.

6.5 Valeurs manquantes comme signal

Le taux global d’absence peut masquer une concentration par canal, région ou période. On compare les taux de manque et la variable cible entre groupes observables.

```

1 missing_profile = (
2     deliveries.assign(missing_distance=deliveries["distance_km"].isna())
3     .groupby("region", as_index=False)
4     .agg(
5         rows=("delivery_id", "size"),
6         missing=("missing_distance", "mean"),
7         median_duration=("duration_min", "median"),
8     )
9 )

```

Une imputation par médiane modifie la distribution et réduit artificiellement la variabilité. Il faut conserver un indicateur d’absence lorsque le fait de manquer peut contenir de l’information.

6.6 Comparer des groupes

Comparer uniquement des moyennes peut masquer tailles d'échantillon, dispersion et composition. Une bonne table fournit au moins effectif, centre et dispersion par groupe.

```

1 by_region = (
2     deliveries.groupby("region", as_index=False)
3         .agg(
4             n=("delivery_id", "nunique"),
5             mean=("duration_min", "mean"),
6             median=("duration_min", "median"),
7             q90=("duration_min", lambda s: s.quantile(.90)),
8         )
9 )

```

Une différence brute entre régions peut venir des distances, du trafic ou du type de commande. Stratifier par tranche de distance vérifie si l'association subsiste à composition comparable.

6.7 Relations entre variables

La covariance indique le sens d'une variation conjointe. La corrélation de Pearson normalise la covariance entre -1 et 1 et décrit une association linéaire. Elle réagit aux extrêmes et ne prouve pas une causalité.

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}.$$

Point de vigilance — corrélation agrégée

Une tendance dans les données regroupées peut s'inverser dans chaque sous-groupe. Toujours identifier un facteur de composition plausible et comparer le graphique global aux graphiques conditionnels.

6.8 Un graphique répond à une question

- Histogramme ou ECDF : comment une variable se distribue-t-elle ?
- Boîte ou violon par groupe : comment les distributions diffèrent-elles ?
- Nuage de points : quelle relation et quelle structure locale apparaissent ?
- Série temporelle : quand le niveau ou la variabilité changent-ils ?

Les axes portent unités et population. Le titre nomme la comparaison. Un graphique sans dénominateur, période ou taille de groupe peut soutenir une conclusion trompeuse.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.scatterplot(
5     data=deliveries,
6     x="distance_km",
7     y="duration_min",
8     hue="region",
9     alpha=.55,
10 )

```

```
11 plt.xlabel("Distance (km)")
12 plt.ylabel("Durée (minutes)")
13 plt.title("Durée et distance selon la région")
```

6.9 Analyse de sensibilité

Une conclusion robuste doit résister à des choix raisonnables : moyenne ou médiane, avec ou sans valeurs manifestement erronées, période complète ou récente, population globale ou comparable. Les résultats qui changent doivent être signalés, pas cachés.

6.10 Étude de cas : retard de livraison

1. Vérifier que chaque ligne représente une livraison terminée unique.
2. Comparer volumes et taux de valeurs manquantes par semaine.
3. Tracer la distribution globale puis par région et tranche de distance.
4. Comparer médiane et quantile 90 avant et après la date du changement.
5. Examiner les extrêmes et documenter s'ils sont erreurs ou incidents réels.
6. Formuler une conclusion limitée : ce que les données montrent et ce qu'elles ne permettent pas d'attribuer causalement.

6.11 Approfondissement : conduire une enquête descriptive

6.11.1 Séparer qualité de données et phénomène étudié

Une variation observée peut venir du monde, du processus de collecte ou du code. Avant d'interpréter une hausse des retards, vérifier couverture temporelle, duplication, changement d'unité, définition de la population et taux de manque. Cette séparation évite d'attribuer à l'opérationnel un effet créé par un nouveau capteur ou une nouvelle jointure.

Une table de profil doit contenir les dénominateurs. « 20 valeurs manquantes » n'a pas le même sens sur 100 et sur un million de lignes. Les effectifs de groupe accompagnent toute moyenne ou proportion. Un segment minuscule peut afficher un taux extrême par simple variabilité.

6.11.2 Choisir un résumé pour une question

La moyenne conserve l'information additive : le total est le produit de la moyenne par l'effectif. Elle est sensible aux valeurs extrêmes. La médiane décrit le centre ordinal et résiste davantage aux queues longues. Les quantiles décrivent directement un niveau de service : le 90e percentile du délai est une durée dépassée par environ 10% des cas.

Exemple — promesse de livraison

Pour les durées [20, 21, 22, 24, 180], la moyenne vaut 53,4 minutes et la médiane 22. La moyenne représente la charge totale par livraison ; la médiane décrit une livraison centrale ; le maximum révèle un incident. Aucune mesure n'est « la bonne » hors de la question. Une promesse client peut nécessiter un quantile élevé, tandis qu'un calcul de coût total peut dépendre de la moyenne.

La variance moyenne les carrés des écarts au centre ; l'écart type revient à l'unité originale. Pour une distribution très asymétrique, l'intervalle interquartile et des quantiles sont souvent plus interprétables. Standardiser une valeur en score z suppose que moyenne et écart type constituent des références pertinentes ; un grand z n'est pas automatiquement une erreur.

6.11.3 Valeurs atypiques : signaler, expliquer, décider

La règle $[Q_1 - 1,5IQR, Q_3 + 1,5IQR]$ détecte des observations éloignées dans une distribution univariée. Elle ne connaît ni le métier, ni la saison, ni les groupes. Un retard de 180 minutes peut être impossible pour un trajet numérique mais réel pour une livraison lors d'une panne.

Le traitement suit quatre états distincts : observation brute, drapeau automatique, qualification à partir d'une règle métier, décision analytique. Conserver ces colonnes séparées assure la traçabilité. Comparer ensuite les résultats avec et sans les cas confirmés erronés ; ne pas supprimer tous les cas difficiles parce qu'ils gênent le résumé.

6.11.4 Comprendre les mécanismes de manque

Une valeur manquante peut être approximativement indépendante des variables observées, dépendre d'un groupe ou d'un contexte observé, ou dépendre de la valeur non observée elle-même. Ces catégories guident les vérifications sans être directement prouvées par la table.

Créer une indicatrice `is_missing`, puis comparer son taux par période, canal, appareil et segment. Examiner aussi si l'absence apparaît après une modification du pipeline. L'imputation restaure une valeur exploitable par un calcul, mais ne recrée pas l'information perdue ; elle peut réduire artificiellement la variance et modifier les relations. Le rapport doit donc conserver un indicateur de manque et décrire la méthode.

6.11.5 Association, composition et paradoxe de Simpson

Une corrélation résume une relation linéaire entre deux variables numériques. Elle peut être proche de zéro malgré une relation non linéaire, dominée par quelques points ou créée par un troisième facteur. Comparer le nuage global aux nuages par groupe.

Dans un paradoxe de Simpson, l'association agrégée change de sens une fois les groupes distingués. Par exemple, deux régions peuvent avoir chacune une relation faible entre distance et satisfaction, mais des distances et niveaux de satisfaction moyens très différents ; la composition crée alors une forte corrélation globale. La bonne réponse n'est pas de choisir le coefficient préféré, mais d'identifier quelle population et quelle décision chaque niveau décrit.

6.11.6 Construire une figure vérifiable

Une figure académique annonce population, variables, unités et transformation. L'axe doit permettre une comparaison honnête ; la couleur doit porter une information ; les effectifs ou l'incertitude doivent apparaître lorsque la taille des groupes diffère. Un histogramme dépend de la largeur des classes, une boîte à moustaches masque parfois la multimodalité, et un nuage très dense nécessite transparence ou agrégation.

Question	Figure utile	Contrôle associé
forme d'une variable	histogramme, ECDF	classes, manque, unité
comparaison de groupes	boîtes, violons, points + intervalle	effectifs, composition
relation numérique	nuage, lissage prudent	extrêmes, groupes, non-linéarité
évolution temporelle	ligne ou points temporels	couverture, saisonnalité, définition

6.11.7 De la figure à la phrase

Une interprétation solide possède quatre morceaux : observation quantitative, contexte, limite et prochaine vérification. « La médiane augmente de 4 minutes après juin dans les deux régions principales ; les volumes sont comparables, mais le taux de distance manquante double ; vérifier le changement de collecte avant d'attribuer cette hausse au processus logistique. » Cette phrase sépare ce que les données montrent de l'hypothèse causale.

6.12 pandas en profondeur : transformer sous contrat

6.12.1 Series, DataFrame et Index

Une `Series` associe un vecteur de valeurs à un index ; une `DataFrame` aligne plusieurs `Series` sur un index commun. L'index peut être un simple identifiant de lignes, une clé métier ou un axe temporel. Ces rôles ne sont pas interchangeables. Un index non unique est autorisé, mais certaines opérations deviennent ambiguës ; toujours inspecter `index.is_unique` lorsque l'unicité est attendue.

```

1 import pandas as pd
2
3 s = pd.Series([12, 18], index=["client_b", "client_a"])
4 t = pd.Series([2, 3], index=["client_a", "client_b"])
5 print(s * t) # alignment by label, not by position

```

Cet alignement protège lorsque l'ordre diffère et surprend lorsque les index ont été conservés après des filtres incompatibles. Avant une opération entre objets, demander si la relation attendue est par étiquette, position ou clé de jointure.

6.12.2 Schéma à l'entrée

Lire un fichier est déjà une étape de modélisation. L'inférence automatique peut transformer un code postal en entier, une chaîne vide en valeur manquante ou une date ambiguë selon le format. Spécifier les colonnes, types et parsing critique, puis auditer les échecs.

```

1 orders = pd.read_csv(
2     "orders.csv",
3     usecols=["order_id", "customer_id", "created_at", "amount", "channel"],
4     dtype={"order_id": "string", "customer_id": "string", "channel": "category"},
5     parse_dates=["created_at"],
6 )
7 orders["amount"] = pd.to_numeric(orders["amount"], errors="coerce")

```

`errors="coerce"` ne résout pas les valeurs invalides : il les rend comptables. Garder un masque des conversions échouées et quelques valeurs brutes pour diagnostiquer.

6.12.3 Types d'extension et absence

pandas fournit notamment `string`, `boolean`, `Int64` nullable, `category`, dates et périodes. Ces types représentent mieux des colonnes métier que `object`. `pd.NA` propage une logique à trois valeurs : vrai, faux, inconnu.

```
1 customers = customers.astype({
2     "customer_id": "string",
3     "active": "boolean",
4     "dependants": "Int64",
5     "segment": "category",
6 })
```

Une catégorie économise de la mémoire lorsque le vocabulaire est petit et rend le domaine visible. Elle n'est pas un encodage universel pour le machine learning : son ordre est absent sauf déclaration explicite.

6.12.4 Sélection et affectation sans ambiguïté

`loc` utilise étiquettes et masques ; `iloc` positions. Une sélection de colonnes retourne une `DataFrame` avec une liste et une `Series` avec une seule étiquette. Les parenthèses sont indispensables dans les masques composés, car `&` et `|` n'ont pas la sémantique de `and` et `or` sur des `Series`.

```
1 mask = (
2     orders["amount"].ge(0)
3     & orders["channel"].isin(["web", "mobile"])
4     & orders["created_at"].notna()
5 )
6 valid = orders.loc[mask, ["order_id", "customer_id", "amount"]].copy()
```

Avec le modèle Copy-on-Write des versions récentes, une écriture ne doit pas modifier un autre objet par un partage implicite. Cela ne rend pas l'affectation chaînée correcte : `df[mask]["amount"] = 0` tente de modifier un objet intermédiaire. Utiliser une seule opération `loc` ou retourner un nouvel objet avec `assign`.

6.12.5 Tidy data et remodelage

Une table analytique met généralement une observation par ligne et une variable par colonne. `melt` passe du large au long ; `pivot` exige une valeur unique par cellule ; `pivot_table` agrège lorsque plusieurs lignes correspondent à une cellule.

```
1 long = monthly.melt(
2     id_vars=["region"],
3     var_name="month",
4     value_name="revenue",
5 )
6 wide = long.pivot(index="region", columns="month", values="revenue")
```

Si `pivot` échoue pour doublons, ne le remplacer pas mécaniquement par une moyenne. L'échec révèle que le grain réel n'est pas celui supposé. Identifier la clé manquante ou choisir une agrégation justifiée.

6.12.6 Groupby : agrégation, transformation et filtrage

Le schéma `split-apply-combine` couvre plusieurs contrats :

— `agg` produit une ou plusieurs lignes résumées par groupe ;

- `transform` retourne une valeur alignée avec chaque ligne d'origine ;
- `filter` conserve ou rejette des groupes entiers ;
- `apply` est plus général, mais son contrat de forme est moins visible et son coût Python souvent plus élevé.

```

1 orders["customer_total"] = (
2     orders.groupby("customer_id")["amount"].transform("sum")
3 )
4 orders["share"] = orders["amount"] / orders["customer_total"]
5
6 summary = (orders.groupby(["region", "channel"], observed=True, as_index=False)
7             .agg(rows=("order_id", "size"),
8                 orders=("order_id", "nunique"),
9                 revenue=("amount", "sum"),
10                median=("amount", "median")))

```

`observed=True` limite les combinaisons de catégories aux groupes observés. Avant un `groupby`, écrire le grain d'entrée ; après, le grain devient la combinaison des clés.

6.12.7 Jointures et preuves de cardinalité

La taille maximale d'une jointure dépend des multiplicités de chaque clé. Pour une clé k présente a_k fois à gauche et b_k fois à droite, la jointure interne produit $a_k b_k$ lignes pour cette clé. Ce produit explique les explosions plusieurs-à-plusieurs.

```

1 counts_left = facts.groupby("customer_id").size().rename("left_n")
2 counts_right = dim.groupby("customer_id").size().rename("right_n")
3 risk = (counts_left.to_frame().join(counts_right, how="outer")
4        .fillna(0).assign(product=lambda d: d.left_n * d.right_n))

```

Utiliser `validate="many_to_one"` ou le contrat approprié, puis `indicator=True` pour distinguer appariés, gauche seulement et droite seulement. Réconcilier le nombre de lignes, les clés et une somme métier. Une jointure réussie techniquement peut rester fausse.

6.12.8 Concaténation et union de schémas

`concat(..., axis=0)` empile des observations en alignant les colonnes par nom ; des colonnes absentes deviennent manquantes. `axis=1` aligne des colonnes par index. Avant une union mensuelle, comparer schémas et types ; `verify_integrity=True` peut détecter un index dupliqué lorsque celui-ci doit rester unique.

6.12.9 Texte, dates et temps

Les accesseurs `str`, `dt` et `cat` offrent des opérations vectorisées. Normaliser un texte exige une politique : casse, espaces, accents et valeurs inconnues. Pour les dates, distinguer instant, fuseau horaire, date civile, période et durée.

```

1 events["timestamp"] = pd.to_datetime(events["timestamp"], utc=True, errors="coerce")
2 events["paris_time"] = events["timestamp"].dt.tz_convert("Europe/Paris")
3 events["day"] = events["paris_time"].dt.floor("D")
4 events["normalized_type"] = (events["event_type"].astype("string")
5                             .str.strip().str.lower())

```

Convertir d'abord vers un instant UTC évite des ambiguïtés. Une journée locale ne commence pas toujours au même instant UTC lors des changements d'heure.

6.12.10 Resample, rolling et frontières temporelles

`resample` regroupe selon un index temporel ; `rolling` calcule sur une fenêtre. Une fenêtre de sept observations n'est pas toujours sept jours. Réindexer sur un calendrier complet si l'absence de transaction doit représenter zéro ; laisser manquant si elle signifie absence de collecte.

```
1 daily = (orders.set_index("created_at")["amount"]
2           .resample("D").sum(min_count=1))
3 calendar = pd.date_range(daily.index.min(), daily.index.max(), freq="D", tz=daily.
4                           index.tz)
5 daily = daily.reindex(calendar)
6 daily_filled = daily.fillna(0) # only if no transaction truly means zero
7 rolling_7d = daily_filled.rolling("7D", min_periods=7).mean()
8 week_change = daily_filled - daily_filled.shift(freq="7D")
```

La frontière de fenêtre, les jours partiels et les retards d'arrivée peuvent créer des anomalies artificielles. Documenter la période complète utilisée.

6.12.11 Chaînes de méthodes et fonctions pipe

Une chaîne lisible exprime un flux de données de haut en bas. Chaque fonction nommée doit avoir un contrat simple et peut être testée séparément.

```
1 def require_columns(df, columns):
2     missing = set(columns) - set(df.columns)
3     if missing:
4         raise ValueError(f"missing columns: {sorted(missing)}")
5     return df
6
7 report = (raw
8           .pipe(require_columns, ["order_id", "customer_id", "amount", "created_at"])
9           .assign(amount=lambda d: pd.to_numeric(d["amount"], errors="coerce"))
10          .loc[lambda d: d["amount"].notna() & d["amount"].ge(0)]
11          .groupby("customer_id", as_index=False)
12          .agg(revenue=("amount", "sum", orders=("order_id", "nunique")))
```

Une chaîne trop longue cache les états importants. Introduire des fonctions et conserver des audits aux frontières plutôt que déboguer une expression monolithique.

6.12.12 Performance et passage à l'échelle

Éviter `iterrows` pour les calculs colonne par colonne : préférer opérations vectorisées, agrégations natives ou jointures. `apply(axis=1)` reste une boucle Python pour beaucoup d'usages. Mesurer avant d'optimiser. Réduire colonnes, choisir dtypes, catégoriser les faibles cardinalités et filtrer tôt.

Si les données ne tiennent pas en mémoire, les lire par chunks et maintenir un état associatif : sommes, comptes, minima et maxima se combinent facilement ; médiane exacte et déduplication globale exigent davantage de structure. Le schéma DSA réapparaît : définir l'état minimal qui permet de fusionner des résultats partiels.

Checklist pandas

Écrire le grain ; contrôler schéma et types à l'entrée ; choisir l'alignement ; mesurer le manque ; vérifier l'unicité ; exprimer le contrat des jointures ; distinguer `agg` de `transform` ; réconcilier après chaque changement de grain ; traiter explicitement le temps ; produire une table d'audit avec la table analytique.

6.13 Travail appliqué

À partir d'une table de livraisons, produire un audit du grain, une table descriptive, deux graphiques complémentaires et une conclusion de cinq lignes.

Détecter les durées au-delà des moustaches IQR. Comparer la moyenne avec et sans ces observations, puis décider lesquelles doivent réellement être exclues.

Comparer le taux de distance manquante par région, canal et semaine. Expliquer pourquoi une imputation globale peut créer un biais.

Construire ou analyser un exemple où la corrélation globale diffère des corrélations par groupe. Identifier la variable de composition et reformuler la conclusion.

À partir d'un journal d'événements contenant utilisateur, session, horodatage et étape, construire les taux de passage entre étapes. Traiter doublons, ordre temporel, utilisateurs multi-sessions et dénominateurs. Comparer une boucle par utilisateur à une solution pandas.

À partir de transactions irrégulières et de fuseaux horaires, produire revenu quotidien, moyenne mobile sur sept jours et comparaison à la semaine précédente. Rendre explicites jours sans transaction, fenêtre calendaire, valeurs manquantes et frontière de période.

6.14 À retenir

L'EDA est une enquête structurée. Elle relie une question, un grain, des contrôles, des résumés et des graphiques. Elle révèle des hypothèses à tester ; elle ne transforme pas automatiquement une association en explication.

Sources principales : [12], [13], [14], [15], [18], [16]

Calcul scientifique avec SciPy, probabilités et simulation

Modéliser, intégrer, optimiser et interroger des mécanismes aléatoires

7.1 Question directrice

Un café doit commander des viennoiseries avant de connaître la demande du lendemain. Commander trop peu crée des ruptures ; trop commander crée des invendus. Une moyenne historique ne décrit pas la variabilité ni le coût asymétrique des deux erreurs. La probabilité fournit un modèle de scénarios possibles ; la simulation compare les décisions sous ce modèle.

7.2 Expérience aléatoire et variable aléatoire

Une expérience aléatoire possède plusieurs résultats possibles. Une variable aléatoire associe un nombre à chaque résultat. Sa distribution indique quelles valeurs sont possibles et avec quelles probabilités.

Exemple — commande en ligne

Pour chaque visite, $X = 1$ si un achat a lieu et $X = 0$ sinon. Si la probabilité d'achat est p , alors X suit une loi de Bernoulli. Sur n visites indépendantes de même probabilité, la somme $S = \sum_i X_i$ suit une loi binomiale.

L'espérance $E[X]$ est une moyenne théorique pondérée. La variance $\text{Var}(X) = E[(X - E[X])^2]$ décrit la dispersion des scénarios autour de l'espérance. Une espérance n'est pas nécessairement un résultat possible : le nombre moyen d'incidents peut être 2,4 alors que chaque journée en contient un entier.

7.3 Génération reproductible

NumPy recommande un générateur explicite. La graine permet de reproduire une suite pseudo-aléatoire pendant le développement.

```
1 import numpy as np
2
3 rng = np.random.default_rng(2026)
4 visits = rng.binomial(n=1, p=0.08, size=10_000)
5 estimated_rate = visits.mean()
```

Réutiliser toujours la même graine ne crée pas de nouvelles preuves ; elle rend un calcul reproductible. Pour évaluer la stabilité, on peut répéter avec plusieurs graines ou étudier directement la distribution des estimations.

7.4 Probabilité conditionnelle et indépendance

La probabilité conditionnelle de A sachant B vaut

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Deux événements sont indépendants si connaître l'un ne change pas la probabilité de l'autre : $P(A | B) = P(A)$.

7.4.1 Bayes et taux de base

$$P(A | B) = \frac{P(B | A)P(A)}{P(B | A)P(A) + P(B | \bar{A})P(\bar{A})}.$$

Supposons une anomalie présente dans 1% des dossiers. Un test la détecte dans 95% des dossiers anormaux et produit 5% de faux positifs. Sur 10 000 dossiers attendus : 95 vrais positifs et 495 faux positifs. Parmi 590 alertes, environ 16% correspondent à une anomalie. La sensibilité élevée ne suffit pas quand le taux de base est faible.

```

1 prevalence = 0.01
2 sensitivity = 0.95
3 false_positive_rate = 0.05
4
5 posterior = (
6     sensitivity * prevalence
7     / (sensitivity * prevalence
8         + false_positive_rate * (1 - prevalence))
9 )

```

7.5 Distributions comme modèles générateurs

Une distribution n'est pas choisie parce que son histogramme « ressemble » aux données. Elle traduit un mécanisme et des hypothèses.

Loi	Mécanisme utile	Hypothèses à vérifier
Bernoulli	un événement oui/non	probabilité définie
Binomiale	nombre de succès sur n essais	essais comparables et indépendants
Poisson	nombre d'événements dans une exposition	taux approximativement constant
Exponentielle	attente entre événements Poisson	absence de mémoire du modèle
Normale	variation continue agrégée	symétrie et queues raisonnables

7.5.1 Bernoulli et binomiale

Pour $X \sim \text{Bernoulli}(p)$, $E[X] = p$ et $\text{Var}(X) = p(1 - p)$. Pour $S \sim \text{Binomiale}(n, p)$, $E[S] = np$ et $\text{Var}(S) = np(1 - p)$.

7.5.2 Poisson et exposition

Une loi de Poisson modélise un compte sur une exposition fixée. Le paramètre λ doit correspondre à la période ou au volume : incidents par jour, appels par heure, sinistres pour mille contrats. Un taux qui varie selon l'heure viole le modèle simple.

7.5.3 Normale et standardisation

Si $X \sim \mathcal{N}(\mu, \sigma^2)$, alors $Z = (X - \mu)/\sigma$ suit une normale standard. La normale autorise des valeurs sur toute la droite réelle ; elle peut donc être inadaptée à des durées strictement positives avec une forte queue droite.

7.6 Monte-Carlo : estimer par répétition

Une simulation Monte-Carlo répète un mécanisme aléatoire et résume les résultats.

Structure d'une simulation

1. Définir les paramètres et les hypothèses du monde simulé.
2. Générer un scénario.
3. Calculer la quantité d'intérêt dans ce scénario.
4. Répéter indépendamment un grand nombre de fois.
5. Résumer moyenne, quantiles et probabilité d'un événement.
6. Faire varier les hypothèses importantes.

7.6.1 Exemple : risque de rupture

```
1 rng = np.random.default_rng(2026)
2 simulations = 100_000
3 demand = rng.poisson(lam=42, size=simulations)
4
5 stock = 50
6 shortage_probability = np.mean(demand > stock)
7 expected_waste = np.mean(np.maximum(stock - demand, 0))
8 expected_shortage = np.mean(np.maximum(demand - stock, 0))
```

La décision dépend du coût d'un invendu et du coût d'une rupture, pas seulement de la probabilité de rupture.

```
1 def expected_cost(stock, demand, waste_cost=1.0, shortage_cost=4.0):
2     waste = np.maximum(stock - demand, 0)
3     shortage = np.maximum(demand - stock, 0)
4     return np.mean(waste_cost * waste + shortage_cost * shortage)
5
6 candidates = range(35, 61)
7 costs = {stock: expected_cost(stock, demand) for stock in candidates}
8 best_stock = min(costs, key=costs.get)
```

Le dictionnaire réutilise le motif clé-vers-résultat. La vectorisation calcule chaque coût sur tous les scénarios. Conserver une matrice de toutes les décisions et tous les scénarios pourrait demander beaucoup de mémoire ; une boucle sur les décisions garde seulement un vecteur de taille B .

7.7 Loi des grands nombres et théorème central limite

La loi des grands nombres explique pourquoi une moyenne simulée se stabilise lorsque le nombre de répétitions augmente. Le théorème central limite décrit la distribution de nombreuses moyennes d'échantillons correctement normalisées, sous des conditions appropriées. Ce sont deux affirmations différentes.

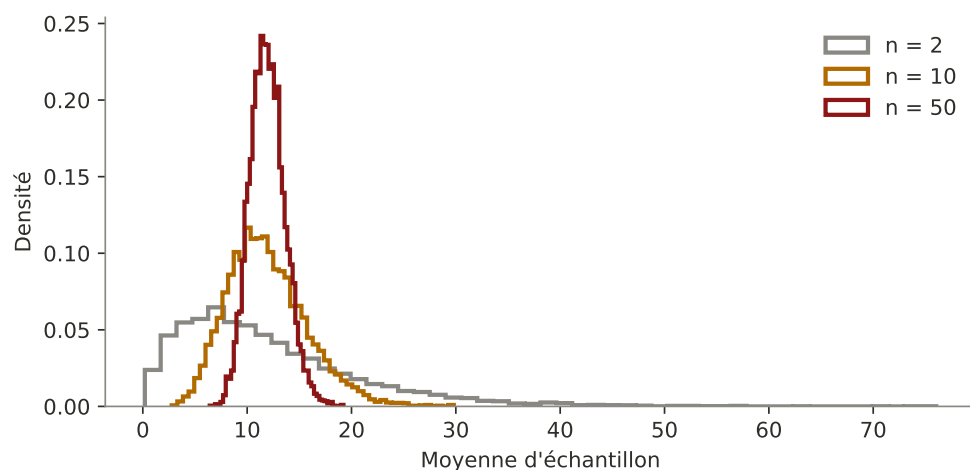


FIGURE 7.1 – Distribution simulée de moyennes d'échantillons : la variabilité diminue avec la taille.

Si des observations ont une variance finie et sont suffisamment indépendantes, la moyenne d'un grand échantillon est approximativement normale, même si la variable initiale ne l'est pas. La dépendance temporelle ou des queues très lourdes peuvent ralentir ou invalider cette approximation.

7.8 Simulation conditionnelle

Pour estimer $P(A | B)$ par simulation, compter les scénarios où B se produit, puis la proportion de ceux où A se produit aussi. Le dénominateur effectif devient le nombre de scénarios satisfaisant B .

```
1 rng = np.random.default_rng(7)
2 fraud = rng.random(200_000) < 0.01
3 alert = np.where(fraud,
4                 rng.random(200_000) < 0.95,
5                 rng.random(200_000) < 0.05)
6 estimated = fraud[alert].mean()
```

7.9 Approfondissement : construire un monde simulé et l'auditer

7.9.1 Une distribution encode un mécanisme

Choisir une loi consiste à raconter comment les données pourraient être générées. Une Bernoulli décrit un essai oui/non de probabilité p . Une binomiale compte les succès parmi n essais comparables et indépendants conditionnellement au modèle. Une Poisson compte des événements sur une exposition donnée avec un taux stable. Une exponentielle modélise le temps d'attente

associé à un processus de Poisson. Une normale décrit une quantité continue et symétrique issue de nombreux effets additifs ; elle ne respecte pas automatiquement une borne à zéro.

Loi	Quantité	Paramètres et unités	Question de validation
Bernoulli	un événement binaire	p sans unité	probabilité stable ?
Binomiale	nombre de succès	n, p	essais comparables et dépendance ?
Poisson	comptage	taux λ par exposition	surdispersion, saisonnalité ?
Exponentielle	temps d'attente positif	échelle temporelle	mémoire du processus ?
Normale	mesure continue	moyenne, écart type	symétrie et bornes plausibles ?

Un ajustement numérique ne suffit pas. Une Poisson dont moyenne et variance diffèrent fortement signale surdispersion ou mélange de populations. Une normale qui génère des durées négatives viole le domaine, même si sa moyenne ressemble aux données.

7.9.2 De la question à l'algorithme Monte-Carlo

Une simulation défendable suit une chaîne explicite : variables d'entrée, mécanisme aléatoire, politique évaluée, métrique de sortie, répétitions et résumé. Pour un stock de café, la variable aléatoire est la demande, la politique est le stock s , la sortie est le coût d'inventu et de rupture.

```

1 def expected_inventory_cost(rate, stocks, shortage_cost,
2                             leftover_cost, repeats=100_000, seed=2026):
3     rng = np.random.default_rng(seed)
4     demand = rng.poisson(rate, size=repeats)
5     result = {}
6     for stock in stocks:
7         shortage = np.maximum(demand - stock, 0)
8         leftover = np.maximum(stock - demand, 0)
9         cost = shortage_cost * shortage + leftover_cost * leftover
10        result[stock] = {
11            "mean": cost.mean(),
12            "p95": np.quantile(cost, .95),
13        }
14    return result

```

Réutiliser les mêmes demandes pour tous les stocks rend leur comparaison moins bruitée : chaque politique rencontre exactement les mêmes scénarios. Conserver une graine permet de reproduire le calcul, mais ne transforme pas une estimation en constante universelle.

7.9.3 Mesurer l'erreur Monte-Carlo

Si Y_b est la sortie du scénario b , la moyenne simulée est $\bar{Y} = B^{-1} \sum_b Y_b$. Son erreur standard Monte-Carlo est approximativement $s_Y / \sqrt{B}$. On peut donc annoncer l'incertitude numérique séparément de l'incertitude sur les paramètres du monde réel.

```

1 cost = simulated_costs(...)
2 estimate = cost.mean()
3 mc_se = cost.std(ddof=1) / np.sqrt(len(cost))
4 print(estimate, estimate - 1.96 * mc_se, estimate + 1.96 * mc_se)

```

Augmenter B réduit seulement l'erreur de simulation. Si le taux de demande estimé est incertain, il faut aussi le faire varier ou l'échantillonner. Une analyse de sensibilité sur λ , les coûts et la dépendance entre jours est souvent plus utile qu'un milliard de répétitions sous une hypothèse unique.

7.9.4 Probabilité conditionnelle et populations rares

L'inversion $P(A | B)$ vers $P(B | A)$ est une source majeure d'erreurs. Pour un test, la sensibilité $P(+ | M)$ ne répond pas à la question client $P(M | +)$. Construire une population fictive de 10,000 personnes rend le dénominateur visible. Avec 1% de prévalence, 95% de sensibilité et 5% de faux positifs, on attend environ 95 vrais positifs et 495 faux positifs : la majorité des alertes ne sont pas des cas.

La simulation doit reproduire cette chronologie : tirer d'abord le statut, puis le test conditionnellement au statut. Tirer indépendamment un statut et une alerte détruirait précisément la relation étudiée.

7.9.5 Indépendance : hypothèse, pas absence de corrélation observée

Deux événements indépendants vérifient $P(A \cap B) = P(A)P(B)$. Dans une simulation, cette hypothèse détermine comment les tirages sont couplés. Des ventes de jours successifs peuvent partager météo, promotion et saison ; les simuler indépendamment sous-estime le risque de plusieurs jours difficiles consécutifs. On peut introduire un état commun ou ré-échantillonner des blocs de jours pour préserver une partie de la dépendance.

7.9.6 Loi des grands nombres et TCL : deux messages distincts

La loi des grands nombres explique pourquoi une moyenne d'un grand nombre de scénarios se rapproche de l'espérance du modèle. Le théorème central limite décrit, sous des conditions appropriées, la forme et l'échelle des fluctuations de moyennes répétées. Il ne dit pas que les données originales deviennent normales, ni que tout échantillon de taille 30 suffit. Dépendance forte, queues lourdes et variance infinie peuvent invalider l'approximation familière.

7.9.7 Audit d'une simulation

Avant décision, vérifier : unités et support de chaque variable ; moyenne, variance et quantiles des tirages ; reproduction de cas limites ; convergence selon B ; sensibilité aux paramètres ; mémoire maximale ; et comparaison à un cas calculable exactement. Un modèle de dés doit retrouver $1/6$ pour une somme au moins égale à 10. Ce test ancre le code avant d'aborder des scénarios sans formule fermée.

Point de vigilance — précision trompeuse

Afficher 12.483729 euros ne signifie pas connaître le coût réel au millionième. Rapporter l'erreur Monte-Carlo, l'incertitude des paramètres et les hypothèses qui ne sont pas quantifiées. La décision est conditionnelle au modèle.

7.10 SciPy en profondeur : algorithmes scientifiques spécialisés

7.10.1 Une bibliothèque organisée par problème

SciPy complète les tableaux NumPy par des algorithmes numériques validés et regroupés en sous-modules : `scipy.stats` pour distributions et tests, `optimize` pour optimisation et racines, `integrate` pour quadrature et équations différentielles, `linalg` pour algèbre linéaire, `sparse` pour matrices creuses, `interpolate` pour interpolation et `signal` pour séries et signaux.

La première question n'est pas « quelle fonction appeler ? », mais « quel problème mathématique et quelles hypothèses cette fonction résout-elle ? ». Lire la signature, le critère d'arrêt, le statut de convergence et les avertissements fait partie du calcul.

7.10.2 Objets distribution dans `scipy.stats`

Une distribution peut être « figée » avec ses paramètres. Elle expose densité ou masse, fonction de répartition, fonction de survie, quantiles, moments et génération aléatoire.

```
1 from scipy import stats
2 import numpy as np
3
4 rng = np.random.default_rng(2026)
5 delay_model = stats.gamma(a=2.5, scale=8.0)
6 simulated = delay_model.rvs(size=100_000, random_state=rng)
7 prob_over_30 = delay_model.sf(30)          # P(X > 30)
8 q95 = delay_model.ppf(.95)                # 95th percentile
9 mean, variance = delay_model.stats(moments="mv")
```

Pour une probabilité de queue, `sf` est souvent numériquement préférable à `1 - cdf`. Pour une loi discrète, distinguer $P(X \geq k)$ de $P(X > k)$: `binom.sf(k-1, n, p)` calcule la première.

7.10.3 Paramètres et unités

Les conventions doivent être lues, pas devinées. Dans beaucoup de distributions SciPy, `loc` décale et `scale` dilate ; ces paramètres ne portent pas toujours les mêmes noms que dans un cours ou un autre logiciel. Par exemple, le taux exponentiel λ correspond à `scale=1/lambda`. Un test sur moyenne et quantile connus détecte les paramétrages inversés.

```
1 rate_per_minute = 0.2
2 waiting = stats.expon(scale=1 / rate_per_minute)
3 assert np.isclose(waiting.mean(), 5.0)
```

7.10.4 Ajuster une distribution sans surinterpréter

`fit` estime des paramètres, souvent par maximum de vraisemblance. L'ajustement ne prouve pas que le mécanisme est vrai. Des valeurs négatives impossibles, des mélanges de populations, une dépendance temporelle ou de la censure peuvent rendre la famille inadéquate.

```
1 train, test = waits[:800], waits[800:]
2 shape, loc, scale = stats.gamma.fit(train, floc=0)
3 model = stats.gamma(a=shape, loc=loc, scale=scale)
4
5 comparison = {
6     "empirical_q95": np.quantile(test, .95),
7     "model_q95": model.ppf(.95),
8     "empirical_over_30": np.mean(test > 30),
9     "model_over_30": model.sf(30),
10 }
```

Séparer calibration et validation empêche d'évaluer uniquement la capacité à reproduire les données déjà utilisées. Comparer ECDF, quantiles et probabilités utiles à la décision ; un test global peut manquer précisément la queue qui coûte le plus.

7.10.5 Tests statistiques comme contrats d'hypothèses

Un test associe statistique, hypothèse nulle, distribution de référence et conditions. Dans SciPy, le test indépendant avec l'option `equal_var=False` réalise le test de Welch pour deux moyennes sans supposer des variances égales. Un test de Mann–Whitney ne devient pas automatiquement un test de médianes ; son interprétation porte sur les distributions et requiert des conditions supplémentaires pour une lecture en décalage.

Avant le test : unité indépendante, plan d'échantillonnage, forme des données, valeurs manquantes, taille des groupes et estimande. Après : estimation d'effet et intervalle avant la p-value. Le chapitre suivant développe cette logique de décision.

7.10.6 Test de permutation

Lorsque l'hypothèse nulle rend les étiquettes échangeables, une permutation crée la distribution de la statistique sous cette hypothèse. Le code suit exactement un motif DSA : mélanger les étiquettes, couper, calculer, compter les valeurs au moins aussi extrêmes.

```

1 def permutation_difference(a, b, repeats=20_000, seed=2026):
2     a, b = np.asarray(a), np.asarray(b)
3     observed = a.mean() - b.mean()
4     pooled = np.concatenate([a, b])
5     rng = np.random.default_rng(seed)
6     null = np.empty(repeats)
7     for i in range(repeats):
8         shuffled = rng.permutation(pooled)
9         null[i] = shuffled[:len(a)].mean() - shuffled[len(a):].mean()
10    p_value = (1 + np.sum(np.abs(null) >= abs(observed))) / (repeats + 1)
11    return observed, p_value, null

```

La correction +1 évite une p-value exactement nulle dans une approximation Monte-Carlo. La validité dépend de l'échangeabilité ; elle n'est pas créée par le mélange informatique.

7.10.7 Intégration numérique

Une espérance continue peut s'écrire comme une intégrale. `quad` retourne estimation et estimation d'erreur numérique. Cette dernière mesure l'algorithme de quadrature, pas l'incertitude du modèle.

```

1 from scipy import integrate
2
3 density = stats.expon(scale=5).pdf
4 expected_capped, numerical_error = integrate.quad(
5     lambda x: min(x, 20) * density(x),
6     0, np.inf,
7 )

```

Comparer à une simulation permet de tester deux implémentations indépendantes. Une intégrale peut être préférable pour une faible dimension et une fonction régulière ; Monte-Carlo devient attractif en haute dimension ou pour un simulateur complexe.

7.10.8 Optimisation : objectif, domaine et convergence

Une fonction d'objectif doit retourner un scalaire et respecter les unités de décision. Pour une seule variable bornée, `minimize_scalar` avec méthode bornée évite d'explorer un stock négatif.

```

1 from scipy import optimize
2
3 def smooth_cost(stock, demand_samples, shortage_cost=4, waste_cost=1):
4     shortage = np.maximum(demand_samples - stock, 0)
5     waste = np.maximum(stock - demand_samples, 0)
6     return np.mean(shortage_cost * shortage + waste_cost * waste)
7
8 result = optimize.minimize_scalar(
9     smooth_cost,
10    args=(demand,),
11    bounds=(0, 100),
12    method="bounded",
13 )
14 if not result.success:
15     raise RuntimeError(result.message)
16 continuous = result.x
17 candidates = np.array([np.floor(continuous), np.ceil(continuous)], dtype=int)
18 best_integer = min(candidates, key=lambda q: smooth_cost(q, demand))

```

Toujours inspecter `success`, `message`, valeur de l'objectif et voisinage de la solution. Un optimiseur trouve un optimum du problème codé, pas du problème métier oublié. Avec une fonction bruitée par simulation, réutiliser les mêmes scénarios entre décisions réduit le bruit de comparaison.

7.10.9 Racines et calibration

Chercher $f(x) = 0$ intervient pour calibrer un seuil, un taux ou une contrainte. Si un intervalle $[a, b]$ contient un changement de signe et que la fonction est continue, une méthode bornée comme `brentq` offre un contrat fort.

```

1 def service_gap(stock):
2     return np.mean(demand <= stock) - 0.95
3
4 stock_95 = optimize.brentq(service_gap, 0, 100)

```

Pour une demande discrète, la fonction en escalier peut ne jamais atteindre exactement zéro ; le quantile empirique est alors plus naturel. Choisir l'algorithme selon la structure.

7.10.10 Interpolation et extrapolation

Interpoler construit une valeur entre observations selon une hypothèse de forme. Extrapoler au-delà du domaine est bien plus risqué. Un tracé et un contrôle des bornes sont obligatoires. Ne pas interpoler mécaniquement des catégories ou masquer de longues pannes de capteur.

7.10.11 Matrices creuses

Une matrice terme-document ou utilisateur-produit contient souvent surtout des zéros. Un format dense coûte $O(np)$ en mémoire ; un format creux stocke valeurs non nulles et indices, soit un coût lié au nombre z de non-zéros. CSR facilite produits matrice-vecteur et accès par ligne ; CSC l'accès par colonne.

```

1 from scipy import sparse
2
3 X_sparse = sparse.csr_matrix((values, (rows, cols)), shape=(n_users, n_items))
4 user_scores = X_sparse @ item_weights

```

Toutes les opérations ne restent pas creuses : centrer les colonnes introduit des valeurs non nulles partout. Anticiper la densification avant une transformation.

7.10.12 Équations différentielles comme modèles dynamiques

`solve_ivp` simule un état continu gouverné par une dérivée. Dans un modèle simple de stock, dS/dt peut combiner arrivée et consommation. Le pas, les tolérances et les événements affectent l'approximation. Une courbe lisse ne valide ni paramètres ni mécanisme.

```

1 from scipy.integrate import solve_ivp
2
3 def dynamics(t, state, arrival_rate, decay):
4     stock = state[0]
5     return [arrival_rate - decay * stock]
6
7 solution = solve_ivp(
8     dynamics, t_span=(0, 24), y0=[10.0],
9     args=(2.0, .15), dense_output=True,
10    rtol=1e-7, atol=1e-9,
11 )

```

Checklist SciPy

Nommer le problème mathématique ; vérifier unités, domaine et hypothèses ; choisir la routine spécialisée ; lire statut et avertissements ; valider sur un cas calculable ; comparer à une méthode indépendante ; effectuer une analyse de sensibilité ; distinguer erreur numérique, erreur Monte-Carlo et incertitude du modèle.

7.11 Travail appliqué

Simuler deux dés, estimer la probabilité d'une somme au moins égale à 10, comparer à la valeur exacte et étudier l'erreur selon le nombre de répétitions.

Simuler la demande quotidienne par une loi de Poisson. Choisir le stock qui minimise un coût combinant invendus et ruptures. Faire varier le taux et les coûts.

Calculer puis simuler la probabilité qu'une alerte corresponde à un cas réel pour plusieurs prévalences. Expliquer pourquoi le résultat change fortement.

Comparer la distribution d'une variable exponentielle à celles des moyennes de 5, 30 et 100 observations. Mesurer l'écart type de chaque distribution de moyennes.

Définir un coût attendu lissé en fonction d'un niveau de stock continu et utiliser `scipy.optimize.minimize_scalar`. Comparer à une recherche sur grille, contrôler le domaine, vérifier le résultat autour de l'optimum et discuter le passage à une décision entière.

Pour des temps d'attente positifs, comparer exponentielle, gamma et distribution empirique. Estimer des paramètres, comparer quantiles et probabilités de dépassement, puis analyser ce que l'ajustement ne prouve pas. Utiliser une séparation calibration-validation.

7.12 À retenir

Une simulation répond à une question sous un modèle explicite. La distribution décrit un mécanisme, les paramètres portent des unités, et l'analyse de sensibilité révèle les hypothèses qui commandent la décision.

Sources principales : [9], [11], [16], [17], [6]

Inférence fréquentiste et bayésienne avec statsmodels et PyMC

Quantifier l'incertitude, vérifier le modèle et décider

8.1 Question directrice

Une nouvelle page augmente le taux de conversion observé de 8,0% à 8,4%. Cette différence reflète-t-elle un effet suffisamment crédible et utile, ou la variation normale d'un échantillon ? Répondre exige de préciser la quantité visée, l'unité randomisée, l'incertitude et le seuil de valeur pratique.

8.2 Population, échantillon et estimande

La population est l'ensemble sur lequel on souhaite conclure. L'échantillon contient les unités observées. L'*estimande* est la quantité exacte visée : différence de moyennes, différence de proportions, ratio ou effet médian.

Unité d'analyse

L'unité indépendante doit correspondre au mécanisme d'échantillonnage ou de randomisation. Si l'expérience assigne un utilisateur mais que chaque clic devient une ligne indépendante, les utilisateurs très actifs reçoivent un poids excessif et l'incertitude est sous-estimée.

8.3 Estimation et erreur standard

Une estimation ponctuelle résume l'échantillon. Une erreur standard décrit combien cette estimation varierait entre échantillons comparables. Pour une proportion $\hat{p}$ issue de n unités indépendantes,

$$SE(\hat{p}) \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

Une approximation d'intervalle à 95% est $\hat{p} \pm 1.96 SE$, lorsque les conditions d'approximation sont raisonnables.

8.4 Bootstrap

Le bootstrap approxime la distribution d'échantillonnage en rééchantillonnant avec remise les unités observées.

Algorithme bootstrap

Pour $b = 1, \dots, B$, tirer n unités avec remise dans l'échantillon, recalculer la statistique et conserver sa valeur. Les quantiles 2,5% et 97,5% des B statistiques forment un intervalle percentile simple.

```

1 import numpy as np
2
3 def bootstrap_difference(control, treatment, repeats=10_000, seed=2026):
4     rng = np.random.default_rng(seed)
5     estimates = np.empty(repeats)
6     for b in range(repeats):
7         c = rng.choice(control, size=len(control), replace=True)
8         t = rng.choice(treatment, size=len(treatment), replace=True)
9         estimates[b] = t.mean() - c.mean()
10    return estimates
11
12 boot = bootstrap_difference(control, treatment)
13 interval = np.quantile(boot, [.025, .975])

```

La boucle alloue deux échantillons à la fois et un vecteur de B estimations. Une matrice de forme (B, n) peut accélérer certains calculs mais demande $O(Bn)$ mémoire. Le rééchantillonnage doit respecter l'unité indépendante et la structure par groupe.

8.5 Hypothèse nulle et valeur p

Un test fixe une statistique et un modèle sous une hypothèse nulle H_0 . La valeur p est la probabilité, sous ce modèle, d'obtenir une statistique au moins aussi extrême que celle observée. Elle n'est ni la probabilité que H_0 soit vraie ni la taille de l'effet.

Point de vigilance — quatre questions différentes

- Quelle est l'estimation de l'effet ?
- Quelle incertitude entoure cette estimation ?
- Les données sont-elles surprenantes sous H_0 ?
- L'effet possible justifie-t-il une décision métier ?

Une valeur p ne répond qu'à la troisième question, sous les hypothèses du test.

8.6 Significativité statistique et pratique

Un effet minuscule devient statistiquement détectable avec un très grand échantillon. À l'inverse, une expérience courte peut être compatible avec des effets négatifs et positifs importants. Avant l'analyse, définir un effet minimal utile, par exemple +0,3 point de conversion, et des métriques de garde.

8.7 Conception d'une expérience A/B

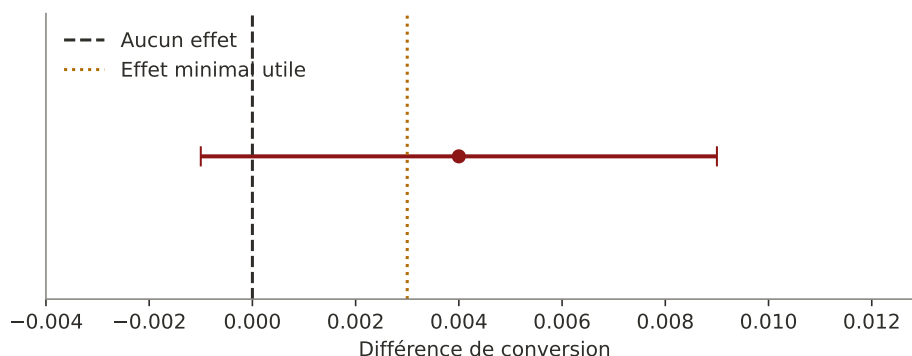


FIGURE 8.1 – Estimation et intervalle : la décision dépend aussi du seuil d'effet utile.

8.7.1 Randomisation

La randomisation rend les groupes comparables en moyenne et permet une interprétation causale de la différence, si l'exposition est correctement appliquée et si les unités ne s'influencent pas fortement.

8.7.2 Avant le lancement

Le protocole précise unité de randomisation, population éligible, métrique principale, effet minimal utile, métriques de garde, durée, règle d'arrêt et exclusions. Ces choix ne doivent pas être ajustés après avoir vu les résultats.

8.7.3 Contrôles d'intégrité

- comparer volumes assignés et ratio attendu ;
- vérifier les caractéristiques mesurées avant traitement ;
- mesurer exposition réelle, données manquantes et événements dupliqués ;
- rechercher contamination entre groupes et changements simultanés ;
- confirmer que les métriques utilisent la bonne unité et la bonne fenêtre.

Un ratio de groupes très éloigné de l'allocation prévue peut signaler un problème de tracking ou d'éligibilité. Tester l'effet avant ce contrôle donne une précision illusoire.

8.8 Étude complète : conversion

```

1 import pandas as pd
2 from statsmodels.stats.proportion import proportions_ztest
3
4 summary = (
5     experiment.groupby("variant", as_index=False)
6     .agg(users=("user_id", "nunique"), conversions=("converted", "sum"))
7 )
8 summary["rate"] = summary["conversions"] / summary["users"]
9
10 control = summary.loc[summary["variant"].eq("control")].iloc[0]
11 treat = summary.loc[summary["variant"].eq("treatment")].iloc[0]
12 effect = treat["rate"] - control["rate"]
13

```

```

14 z_stat, p_value = proportions_ztest(
15     [treat["conversions"], control["conversions"]],
16     [treat["users"], control["users"]],
17 )

```

La table doit contenir une ligne par utilisateur ou une conversion correctement agrégée au niveau utilisateur. Le résultat final inclut taux par groupe, différence absolue, intervalle, valeur p, métriques de garde et recommandation.

Exemple — note de décision

« Le traitement augmente la conversion estimée de 0,4 point. L'intervalle à 95% va de -0,1 à 0,9 point : les données restent compatibles avec une légère baisse et avec un gain supérieur au seuil utile de 0,3 point. Aucun problème d'intégrité n'a été détecté. Une collecte supplémentaire est justifiée si son coût est inférieur à la valeur de réduire cette incertitude. »

8.9 Puissance et taille d'échantillon

La puissance est la probabilité de détecter un effet donné lorsque cet effet existe. Elle augmente avec la taille, l'effet et un bruit plus faible. Choisir la taille après avoir observé les résultats compromet la règle d'erreur prévue. Pour des événements rares, un grand nombre d'unités peut être nécessaire malgré un effet relatif élevé.

8.10 Multiplicité et analyses répétées

Tester de nombreuses métriques ou segments augmente la probabilité d'au moins un faux signal. Les analyses exploratoires restent utiles, mais elles doivent être nommées comme telles et confirmées sur de nouvelles données. Regarder chaque jour et arrêter dès que $p < 0.05$ modifie le taux d'erreur ; une règle séquentielle adaptée est nécessaire.

8.11 Approfondissement : de l'expérience à la décision

8.11.1 Définir l'estimande avant la métrique

Une estimande est la quantité que l'étude cherche à connaître. « Effet sur la conversion » reste ambigu : conversion par visite, par utilisateur exposé, dans quelle fenêtre, avec quel traitement des utilisateurs non exposés ? Une définition opérationnelle peut être : différence de probabilité d'au moins un achat dans les sept jours suivant l'affectation, entre traitement et contrôle, pour tous les utilisateurs randomisés.

Cette phrase fixe l'unité, la population, la fenêtre, la métrique et le contraste. Le code ne doit venir qu'après. Si la randomisation est par utilisateur, l'analyse principale ne peut pas prétendre disposer d'une observation indépendante par clic.

8.11.2 Intervalle : effet plausible et précision

Une estimation ponctuelle sans intervalle ne distingue pas petit effet précis et grand effet incertain. Pour une différence de taux, l'intervalle approximatif combine les erreurs standards des deux groupes. L'application importante est la comparaison à deux références : zéro et l'effet minimal utile.

Position de l'intervalle	Lecture décisionnelle possible
entièrement au-dessus du seuil utile	bénéfice compatible avec le déploiement
au-dessus de zéro mais sous le seuil utile	effet positif mais valeur insuffisante
traverse zéro et le seuil utile	information insuffisante pour trancher
entièrement sous le seuil utile	effet utile peu compatible avec les données

Cette grille n'automatise pas la décision : coûts, risques et métriques de garde restent nécessaires. Elle empêche toutefois de réduire l'analyse à « significatif / non significatif ».

8.11.3 Bootstrap au bon niveau

Le bootstrap approxime la variabilité d'une statistique en ré-échantillonnant les unités observées avec remise. L'unité doit correspondre à l'indépendance du design. Si un client possède plusieurs achats, ré-échantillonner les lignes traite les achats du même client comme indépendants et donne trop de poids aux clients actifs. Agréger ou ré-échantillonner des grappes de lignes par client préserve cette structure.

Le bootstrap percentile est simple : calculer la statistique sur chaque réplcation et prendre les quantiles. Il peut être inadéquat pour très petits échantillons, statistiques très biaisées ou événements extrêmement rares. Toujours comparer à une simulation ou à une méthode analytique sur un cas connu avant de l'appliquer mécaniquement.

8.11.4 Ce que la valeur p dit et ne dit pas

Sous une hypothèse nulle et un protocole fixé, la valeur p mesure à quel point une statistique au moins aussi extrême serait inhabituelle. Elle n'est pas la probabilité que l'hypothèse nulle soit vraie, ni la taille de l'effet, ni la probabilité de reproduction. Avec un échantillon immense, un effet négligeable peut produire une petite valeur p ; avec un petit échantillon, un effet utile peut rester très incertain.

Les erreurs de type I et II appartiennent à une procédure répétée. Fixer un seuil de 5% contrôle un taux de faux rejet sous les hypothèses du test ; il ne garantit pas que 5% des résultats publiés sont faux. Le contexte de sélection, les tests multiples et les arrêts anticipés modifient cette interprétation.

8.11.5 Audit opérationnel avant estimation

Une expérience A/B peut échouer avant toute statistique. Vérifier :

1. identifiant d'unité stable et une seule variante par unité ;
2. ratio d'affectation conforme au plan ;
3. horodatage : affectation avant exposition, exposition avant résultat ;
4. doublons d'événements et couverture identique des plateformes ;
5. valeurs manquantes et définitions de métriques identiques ;
6. absence d'effet sur des métriques qui devraient rester invariantes.

Un écart de ratio d'échantillon peut signaler bug d'assignation, filtrage après randomisation ou instrumentation différente. Continuer directement vers une valeur p donnerait une précision mathématique à un contraste qui n'est plus expérimental.

8.11.6 Analyse en intention de traiter

Comparer les groupes selon l'affectation initiale conserve le bénéfice de la randomisation, même si certains utilisateurs n'utilisent pas la fonctionnalité. Filtrer uniquement les utilisateurs effectivement engagés sélectionne un comportement postérieur au traitement et peut créer un biais. Une analyse « utilisateurs exposés » peut compléter le résultat, mais doit être étiquetée et justifiée comme différente de l'estimande principale.

8.11.7 Puissance, durée et effet minimal utile

La planification commence par le taux de base, l'effet minimal utile, les taux d'erreur acceptés, l'unité et la variance. Le calcul de taille fournit un ordre de grandeur ; il faut ensuite convertir en durée selon le trafic et couvrir les cycles hebdomadaires ou saisonniers. Modifier l'objectif après avoir vu les résultats transforme une analyse confirmatoire en exploration.

Consulter chaque jour une valeur p et arrêter au premier passage sous 0,05 augmente le taux de faux signaux. Utiliser une durée fixée, une méthode séquentielle prévue ou une règle d'arrêt documentée. De même, explorer vingt segments demande correction, hiérarchisation ou confirmation sur de nouvelles données.

8.11.8 Note de décision reproductible

La sortie finale n'est pas un tableau de bibliothèque mais une note courte contenant : estimande ; contrôle d'intégrité ; tailles et dénominateurs ; effet et intervalle ; seuil utile ; métriques de garde ; analyses de sensibilité ; limites ; action recommandée.

Exemple — lancement prudent

Le traitement augmente la conversion estimée de 0,4 point, intervalle 95% $[-0,1; 0,9]$. Le seuil utile est 0,3 point et les gardes sont stables. Les données ne permettent ni d'exclure l'absence d'effet ni de garantir la valeur utile. Si une semaine supplémentaire est peu coûteuse, continuer selon une durée décidée à l'avance peut avoir plus de valeur que lancer ou abandonner immédiatement.

8.12 PyMC en profondeur : modéliser l'incertitude

8.12.1 Le changement de question bayésien

L'inférence bayésienne part d'un modèle génératif comportant paramètres inconnus θ , prior $p(\theta)$ et vraisemblance $p(y | \theta)$. Après les données y , la distribution postérieure est

$$p(\theta | y) = \frac{p(y | \theta)p(\theta)}{p(y)}.$$

Elle décrit l'incertitude sur les paramètres conditionnellement au modèle, aux priors et aux données observées. Une probabilité postérieure n'est pas une garantie fréquentiste ; un intervalle de crédibilité n'est pas un intervalle de confiance renommé.

L'intérêt pratique est d'écrire tout le mécanisme : quelles quantités existent, comment elles varient, comment elles produisent les observations et quelles prédictions en découlent. PyMC transforme ce graphe en calcul, mais ne choisit pas la structure à la place de l'analyste.

8.12.2 Workflow bayésien

1. formuler les variables, unités et dépendances ;
2. choisir des priors dans l'espace métier ;
3. simuler le prior prédictif et rejeter les mondes absurdes ;
4. conditionner sur les données et échantillonner ;
5. diagnostiquer le calcul avant d'interpréter ;
6. simuler le posterior prédictif et comparer aux données ;
7. réaliser analyses de sensibilité et décision fondée sur la perte.

Le contrôle prédictif a priori précède les données : si le modèle génère couramment des taux impossibles ou des montants astronomiques, le prior ou la vraisemblance est mal calibré.

8.12.3 Premier modèle PyMC

Pour deux variantes, $y_g \sim \text{Binomial}(n_g, p_g)$ et $p_g \sim \text{Beta}(\alpha, \beta)$. Les coordonnées donnent un nom aux axes et réduisent les erreurs d'index.

```

1 import pymc as pm
2 import arviz as az
3 import numpy as np
4
5 coords = {"variant": ["control", "treatment"]}
6 successes = np.array([400, 432])
7 trials = np.array([5_000, 5_000])
8
9 with pm.Model(coords=coords) as ab_model:
10     rate = pm.Beta("rate", alpha=2, beta=18, dims="variant")
11     conversions = pm.Binomial(
12         "conversions", n=trials, p=rate,
13         observed=successes, dims="variant",
14     )
15     prior_predictive = pm.sample_prior_predictive(random_seed=2026)
16     idata = pm.sample(
17         draws=2_000, tune=2_000, chains=4,
18         target_accept=.9, random_seed=2026,
19     )
20     posterior_predictive = pm.sample_posterior_predictive(
21         idata, random_seed=2026,
22     )

```

Le nom `observed` ne signifie pas variable fixe sans modèle ; il relie les données à la distribution d'observation. Conserver les comptages agrégés est correct ici si chaque utilisateur contribue à un essai Bernoulli indépendant. Si exposition ou répétition diffère, la vraisemblance doit changer.

8.12.4 Priors interprétables

Un prior bêta peut être compris via moyenne $\alpha/(\alpha+\beta)$ et concentration $\alpha+\beta$. `Beta(2,18)` place le centre autour de 10%, avec une dispersion encore large. Il ne signifie pas « ajouter exactement vingt vrais utilisateurs » dans tous les sens causaux ; c'est une régularisation probabiliste.

Pour un coefficient de régression, standardiser les variables aide à exprimer un prior sur un changement d'un écart-type. Sur l'échelle logit, un `Normal(0,1)` autorise déjà des odds ratios substantiels. Visualiser les implications sur la probabilité, pas seulement la densité du coefficient.

8.12.5 Prior prédictif

Le prior prédictif tire d'abord les paramètres puis des données simulées. Vérifier plages, moyennes, queues et cas impossibles. Une prior sur un paramètre peut sembler diffuse mais induire des données irréalistes après transformation non linéaire.

```
1 prior_counts = prior_predictive.prior_predictive["conversions"]
2 prior_rates = prior_counts / trials
3 print(prior_rates.quantile([.05, .5, .95], dim=("chain", "draw")))
```

Selon la version des bibliothèques, l'objet retourné est organisé en groupes xarray. Utiliser les noms de dimensions plutôt que convertir immédiatement en tableau plat.

8.12.6 NUTS et chaînes

PyMC choisit généralement NUTS pour les variables continues différentiables. L'algorithme construit une trajectoire dans l'espace des paramètres afin d'explorer la postérieure plus efficacement qu'une marche aléatoire. `tune` adapte des paramètres de l'échantillonneur ; les tirages d'adaptation ne constituent pas les tirages postérieurs conservés.

Plusieurs chaînes démarrées différemment révèlent certains échecs d'exploration. Plus de tirages réduisent l'erreur Monte-Carlo seulement si les chaînes ciblent correctement la même distribution. Un modèle mal paramétré ne se répare pas en augmentant `draws`.

8.12.7 Diagnostics indispensables

- **R-hat** compare variation entre et dans les chaînes ; une valeur proche de 1 est nécessaire, non suffisante ;
- **ESS** estime le nombre de tirages effectivement indépendants pour moyenne et queues ;
- **divergences** signalent une géométrie mal explorée et peuvent biaiser les résumés ;
- **trace plots** révèlent mélange, dérive, multimodalité ou chaînes bloquées ;
- énergie et BFMI peuvent signaler une exploration dynamique insuffisante.

```
1 summary = az.summary(idata, var_names=["rate"], hdi_prob=.95)
2 az.plot_trace(idata, var_names=["rate"])
3 divergences = int(idata.sample_stats["diverging"].sum())
4 if divergences:
5     print("Reparameterize or revisit the model; do not ignore.")
```

Augmenter `target_accept` peut réduire certaines divergences, mais il faut examiner la géométrie et envisager reparamétrisation ou priors plus informatifs. Rapporter le diagnostic avec le résultat.

8.12.8 Posterior predictive check

Le contrôle posterior prédictif simule de nouvelles observations depuis la postérieure. Il demande si le modèle ajusté reproduit les aspects pertinents : moyenne, dispersion, zéros, queue, distribution par groupe ou structure temporelle.

Un modèle peut estimer correctement une moyenne et sous-estimer la variance. Pour des comptes, une binomiale ou Poisson trop rigide peut nécessiter hétérogénéité, bêta-binomiale ou binomiale négative. Le posterior predictive check ne « prouve » pas le modèle ; il cherche des écarts.

8.12.9 Décision A/B postérieure

La postérieure permet de calculer directement des quantités de décision.

```

1 posterior = idata.posterior["rate"]
2 delta = (posterior.sel(variant="treatment")
3          - posterior.sel(variant="control"))
4
5 prob_positive = float((delta > 0).mean())
6 minimum_useful = .003
7 prob_useful = float((delta > minimum_useful).mean())
8 expected_lift = float(delta.mean())
9 hdi = az.hdi(delta, hdi_prob=.95)

```

`prob_positive` et `prob_useful` répondent à deux questions différentes. Une règle de déploiement peut comparer perte attendue des actions, et non imposer un seuil de probabilité universel. Inclure coûts, gardes et réversibilité.

8.12.10 Lien avec la solution analytique

Le modèle bêta-binomial possède une postérieure fermée : $p_g | y_g \sim \text{Beta}(\alpha + y_g, \beta + n_g - y_g)$. Comparer moyennes et quantiles PyMC à cette solution teste le modèle et l'infrastructure. On n'utilise pas MCMC pour gagner en simplicité ici, mais pour apprendre un workflow qui s'étend aux modèles sans solution fermée.

8.12.11 Modèles hiérarchiques et pooling partiel

Estimer chaque région séparément produit des valeurs extrêmes dans les petits groupes ; tout regrouper impose le même taux partout. Un modèle hiérarchique suppose que les effets régionaux proviennent d'une population commune. Les régions peu observées sont davantage ramenées vers la tendance générale : c'est le *shrinkage*.

```

1 import pytensor.tensor as pt
2
3 coords = {"region": region_names}
4 with pm.Model(coords=coords) as hierarchical:
5     global_logit = pm.Normal("global_logit", 0, 1.5)
6     region_sd = pm.HalfNormal("region_sd", 1)
7     region_offset = pm.Normal("region_offset", 0, 1, dims="region")
8     region_logit = pm.Deterministic(
9         "region_logit",
10        global_logit + region_sd * region_offset,
11        dims="region",
12    )
13    region_rate = pm.Deterministic(
14        "region_rate", pm.math.sigmoid(region_logit), dims="region"
15    )
16    y = pm.Binomial("y", n=trials_by_region, p=region_rate,
17                   observed=successes_by_region, dims="region")

```

L'écriture `region_offset` correspond à une paramétrisation non centrée, souvent plus facile à échantillonner lorsque l'information par groupe est faible. Le pooling partiel ne garantit pas l'échangeabilité des régions ; cette hypothèse doit être défendue.

8.12.12 Régression bayésienne appliquée

Une régression logistique peut inclure traitement, variables pré-traitement et interactions préspecifiées. Le modèle produit des probabilités individuelles et des effets agrégés par standardisation : prédire chaque unité sous traitement et sous contrôle, puis moyenner la différence. Lire un seul coefficient log-odds comme un effet en points de probabilité est faux.

8.12.13 Sensibilité et comparaison

Refaire l'analyse avec plusieurs priors raisonnables, modèles de vraisemblance et règles d'inclusion. Une conclusion qui change révèle quelle hypothèse commande la décision. Pour comparer des modèles prédictifs bayésiens, la validation hors échantillon ou des critères comme LOO peuvent aider, mais une petite différence de score ne remplace pas les diagnostics, la plausibilité et l'adéquation au but.

8.12.14 Reproductibilité et coût

Enregistrer code, données, versions, graines, nombre de chaînes, warmup, tirages, taux d'acceptation ciblé, diagnostics et résumés. Les tirages postérieurs peuvent être volumineux : sélectionner les variables nécessaires pour certains exports, sans perdre l'archive scientifique. Paralléliser des chaînes accélère le temps mur mais ne change pas le nombre d'opérations.

Checklist PyMC

Graphe génératif explicite ; unités et coordonnées ; priors justifiés par prédiction ; données au bon grain ; plusieurs chaînes ; R-hat, ESS, divergences et traces ; contrôles prédictifs ; quantités de décision ; sensibilité ; version et configuration enregistrées. Ne jamais interpréter une postérieure avant d'avoir validé le calcul.

8.13 Travail appliqué

Calculer un taux de conversion et son intervalle approximatif. Comparer les largeurs pour 500, 5 000 et 50 000 utilisateurs.

Construire l'intervalle bootstrap de la différence de panier moyen. Rééchantillonner les utilisateurs, pas les lignes de produits, et expliquer la différence.

À partir d'un jeu volontairement imparfait, détecter ratio anormal, utilisateurs dans les deux groupes, doublons et métrique calculée au mauvais grain.

Rédiger une recommandation à partir d'une estimation, d'un intervalle, d'une valeur p, d'un seuil utile et de deux métriques de garde.

Modéliser les conversions de deux variantes par un modèle bêta-binomial dans PyMC. Définir des priors justifiés, réaliser contrôles prédictifs a priori et a posteriori, échantillonner, diagnostiquer, puis estimer $P(p_B > p_A)$ et $P(p_B - p_A > \delta)$ pour un seuil utile δ .

Plusieurs régions ont des volumes très différents. Comparer estimations indépendantes, pooling complet et modèle hiérarchique logistique. Expliquer le shrinkage, contrôler les diagnostics MCMC et produire une décision par région sans sélectionner seulement les gagnants.

8.14 À retenir

L'inférence relie une estimation à l'incertitude créée par l'échantillonnage. Une expérience crédible dépend d'abord de son design et de son intégrité. La décision combine effet, intervalle, valeur pratique, risques et coût d'information supplémentaire.

Sources principales : [16], [19], [20], [21]

Systèmes de machine learning avec scikit-learn

Pipelines, validation, sélection, interprétation et exploitation

9.1 Question directrice

Une équipe veut contacter les clients susceptibles de résilier, mais elle ne peut appeler que 500 personnes par semaine. Le problème n'est pas seulement « prédire le churn ». Il faut définir quand la prédiction est faite, quelles données sont disponibles, quelle erreur coûte le plus et comment transformer un score en liste d'action.

9.2 Formuler la tâche

Une formulation complète précise :

- l'unité de prédiction, par exemple client actif à la date t ;
- la cible, par exemple résiliation dans les 60 jours ;
- l'instant de prédiction et l'horizon ;
- les variables disponibles à cet instant ;
- la décision déclenchée par le score ;
- le coût des faux positifs et faux négatifs.

Point de vigilance — fuite de cible

Une variable créée après l'instant de prédiction peut rendre le modèle excellent hors contexte et inutilisable en production. Un statut de clôture, un remboursement futur ou un agrégat calculé sur toute la période révèle indirectement la cible.

9.3 Séparer avant d'apprendre

Le jeu de test simule des données futures jamais utilisées pour choisir le modèle. Une séparation aléatoire convient aux observations échangeables. Une séparation temporelle convient lorsque le futur diffère du passé. Des lignes du même utilisateur ou groupe doivent rester dans la même partition lorsque leur proximité créerait une fuite.

```
1 from sklearn.model_selection import train_test_split
2
3 X = data.drop(columns="churned")
4 y = data["churned"]
5 X_train, X_test, y_train, y_test = train_test_split(
```

```

6     X, y,
7     test_size=.25,
8     stratify=y,
9     random_state=2026,
10 )

```

Le test ne sert pas à choisir hyperparamètres, variables ou seuils. Ces choix utilisent validation croisée ou ensemble de validation à l'intérieur des données d'entraînement.

9.4 Baseline avant modèle complexe

Une baseline mesure le niveau minimal à dépasser. En classification déséquilibrée, prédire toujours la classe majoritaire peut avoir une forte accuracy et aucune valeur. Une baseline fondée sur une règle métier simple est souvent plus informative.

```

1 from sklearn.dummy import DummyClassifier
2
3 baseline = DummyClassifier(strategy="prior")
4 baseline.fit(X_train, y_train)
5 baseline_score = baseline.predict_proba(X_test)[: , 1]

```

9.5 Prétraitement dans un pipeline

Les statistiques d'imputation, catégories et paramètres de normalisation doivent être appris uniquement sur l'entraînement. Pipeline et ColumnTransformer attachent ces étapes au modèle et évitent les transformations incohérentes.

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.impute import SimpleImputer
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.pipeline import Pipeline
5 from sklearn.preprocessing import OneHotEncoder, StandardScaler
6
7 numeric = ["tenure", "monthly_spend", "support_calls"]
8 categorical = ["region", "contract_type"]
9
10 preprocess = ColumnTransformer([
11     ("num", Pipeline([
12         ("impute", SimpleImputer(strategy="median")),
13         ("scale", StandardScaler()),
14     ]), numeric),
15     ("cat", Pipeline([
16         ("impute", SimpleImputer(strategy="most_frequent")),
17         ("encode", OneHotEncoder(handle_unknown="ignore")),
18     ]), categorical),
19 ])
20
21 model = Pipeline([
22     ("preprocess", preprocess),
23     ("classifier", LogisticRegression(max_iter=1000)),
24 ])
25 model.fit(X_train, y_train)

```

Cette architecture applique le même état appris pendant validation, test et production. Elle concrétise les composants de pipeline du chapitre 4.

9.6 Régression et classification

Une régression prédit une quantité numérique, par exemple un coût. Une classification prévoit une catégorie ou un score de probabilité. Un score n'impose pas encore une décision : le seuil dépend des coûts et de la capacité.

9.6.1 Métriques de régression

La MAE est la moyenne des erreurs absolues et se lit dans l'unité de la cible. La RMSE met davantage de poids sur les grandes erreurs. R^2 compare l'erreur quadratique au prédicteur moyen mais ne mesure ni causalité ni qualité opérationnelle.

9.6.2 Matrice de confusion

Pour une classe positive :

$$\text{précision} = \frac{VP}{VP + FP}, \quad \text{rappel} = \frac{VP}{VP + FN}.$$

La précision répond « parmi les alertes, combien sont positives ? ». Le rappel répond « parmi les positifs réels, combien avons-nous trouvés ? ». Leur importance dépend de la décision.

```
1 from sklearn.metrics import classification_report, confusion_matrix
2
3 score = model.predict_proba(X_test)[: , 1]
4 prediction = score >= 0.5
5 print(confusion_matrix(y_test, prediction))
6 print(classification_report(y_test, prediction))
```

9.7 Seuil, capacité et coûts

Si l'équipe peut traiter 500 clients, le seuil naturel peut être le score du 500e client plutôt que 0,5. L'évaluation doit alors mesurer précision parmi les 500 premiers, rappel couvert et valeur attendue.

```
1 import numpy as np
2
3 capacity = 500
4 selected = np.argsort(score)[-capacity:]
5 precision_at_capacity = y_test.iloc[selected].mean()
6 covered_positives = y_test.iloc[selected].sum() / y_test.sum()
```

La courbe ROC trace rappel contre taux de faux positifs pour tous les seuils. L'AUC-ROC mesure la qualité de classement globale. Avec une classe rare et une capacité limitée, la courbe précision-rappel et les métriques au volume d'action sont souvent plus proches de la décision réelle.

9.8 Surapprentissage et validation

Un modèle surapprend lorsqu'il capture des particularités de l'entraînement qui ne se reproduisent pas. Un grand écart entre performance d'entraînement et validation le signale. La validation croisée estime la variabilité entre partitions, sans remplacer un test final ni une séparation temporelle pertinente.

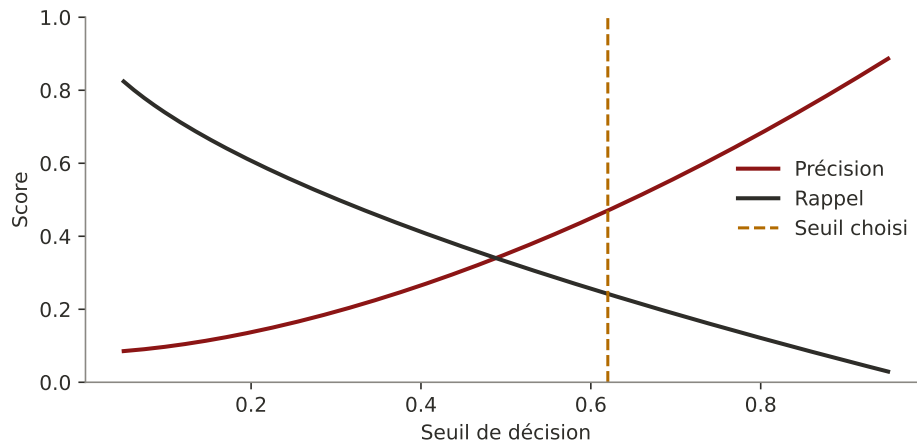


FIGURE 9.1 – Le seuil déplace le compromis entre précision et rappel.

```

1 from sklearn.model_selection import cross_validate
2
3 scores = cross_validate(
4     model,
5     X_train,
6     y_train,
7     cv=5,
8     scoring=["roc_auc", "average_precision"],
9     return_train_score=True,
10 )

```

Comparer la distribution des scores de validation, pas seulement leur moyenne. Une forte variabilité peut venir d'un échantillon petit, d'une classe rare ou de groupes hétérogènes.

9.9 Analyse des erreurs

Une métrique globale ne révèle pas où le système échoue. Construire une table contenant score, prédiction, cible, groupe et variables utiles. Examiner faux positifs et faux négatifs, puis comparer les métriques par segments assez grands.

```

1 evaluation = X_test[["region", "contract_type"]].copy()
2 evaluation["target"] = y_test.to_numpy()
3 evaluation["score"] = score
4 evaluation["prediction"] = prediction
5 evaluation["error"] = np.select(
6     [evaluation["prediction"] & ~evaluation["target"].astype(bool),
7     ~evaluation["prediction"] & evaluation["target"].astype(bool)],
8     ["false_positive", "false_negative"],
9     default="correct",
10 )

```

Les différences de performance par groupe peuvent refléter qualité des données, prévalence, taille d'échantillon ou véritable hétérogénéité. Elles demandent une analyse, pas une conclusion automatique.

9.10 Interprétation et limites

Les coefficients d'un modèle linéaire et les importances d'arbres décrivent le modèle, pas nécessairement le mécanisme causal. Une importance permutation mesure la perte de performance lorsque l'information d'une variable est perturbée ; elle dépend du jeu d'évaluation et des corrélations entre variables.

9.11 Étude complète : attrition client

Le livrable final doit contenir :

1. formulation temporelle et dictionnaire de données ;
2. audit de qualité et baseline ;
3. stratégie de séparation justifiée ;
4. pipeline de prétraitement et modèle reproductible ;
5. métriques globales et au seuil opérationnel ;
6. analyse des erreurs et limites ;
7. recommandation de déploiement, expérimentation ou collecte supplémentaire.

9.12 Approfondissement : évaluer un système de décision, pas seulement un modèle

9.12.1 Formulation avant sélection d'algorithme

Un problème de machine learning commence par une décision et une chronologie. Pour l'attrition : à quelle date le score est-il produit, quels clients sont éligibles, quel horizon définit la cible, quelle action suit le score et quelle capacité limite cette action ? La cible « résiliera dans les 60 jours » exige une date d'observation et une fenêtre future complète.

Construire ensuite une table de variables avec source, timestamp réel de disponibilité, retard d'ingestion et dépendance éventuelle à la cible. Une colonne présente aujourd'hui dans l'entrepôt peut avoir été inconnue au moment historique de la décision. Cette fuite temporelle produit souvent une performance spectaculaire et inutilisable.

9.12.2 Séparation adaptée au déploiement

Le jeu de test simule des données futures et reste fermé jusqu'à la fin. Une séparation aléatoire convient lorsque les observations sont approximativement échangeables. Une séparation temporelle est nécessaire lorsque dérive, saisonnalité ou calcul rétrospectif comptent. Une séparation par groupe empêche le même utilisateur, patient ou contrat d'apparaître des deux côtés.

La validation sert au choix de variables, d'hyperparamètres et de seuil. Réutiliser le test pour ces choix transforme progressivement le test en données d'entraînement. La validation croisée réduit la dépendance à une seule partition, mais doit elle aussi respecter groupes et ordre temporel.

9.12.3 Baselines qui représentent une décision réelle

Une baseline statistique prédit la moyenne ou la classe majoritaire. Une baseline métier reproduit la règle actuelle. Pour une classe rare, la majorité peut afficher une forte accuracy tout

en capturant zéro cas. Comparer au moins : prédiction constante, règle métier simple et petit modèle interprétable. Le nouveau système doit justifier son coût par rapport à ces références.

9.12.4 Pipeline et fuite de prétraitement

Imputation, standardisation, sélection de variables et encodage apprennent un état. Cet état doit être ajusté dans chaque pli d'entraînement, jamais sur l'ensemble avant validation croisée. Pipeline et ColumnTransformer attachent les transformations au modèle afin que la procédure correcte soit répétée automatiquement.

```

1 numeric = Pipeline([
2     ("impute", SimpleImputer(strategy="median")),
3     ("scale", StandardScaler()),
4 ])
5 categorical = Pipeline([
6     ("impute", SimpleImputer(strategy="most_frequent")),
7     ("encode", OneHotEncoder(handle_unknown="ignore")),
8 ])
9 preprocess = ColumnTransformer([
10     ("num", numeric, numeric_columns),
11     ("cat", categorical, categorical_columns),
12 ])
13 model = Pipeline([
14     ("preprocess", preprocess),
15     ("classifier", LogisticRegression(max_iter=1000)),
16 ])

```

Le pipeline protège la procédure, pas la chronologie des variables. Une colonne future reste une fuite même si elle traverse proprement un ColumnTransformer.

9.12.5 Métriques comme coûts partiels

Dans une matrice de confusion, précision = $VP/(VP + FP)$ décrit la qualité des alertes et rappel = $VP/(VP + FN)$ leur couverture. La spécificité décrit la proportion de négatifs correctement rejetés. Le score F1 combine précision et rappel mais impose une pondération implicite ; il ne remplace pas les coûts métier.

ROC-AUC mesure la qualité de classement sur tous les seuils et peut paraître forte avec une classe très rare. Average Precision résume la courbe précision-rappel et se compare au taux de base. Aucune de ces métriques ne répond directement à « parmi les 500 appels, combien touchent un client réellement à risque ? » : calculer précision à capacité, rappel couvert et valeur attendue.

9.12.6 Choisir un seuil par valeur et capacité

Pour chaque client i , une action peut produire bénéfice b si elle réussit et coût c si elle est inutile. Un score de risque n'est pas encore une action. Trier les scores, appliquer la capacité puis estimer la valeur sur validation rend la politique explicite.

```

1 order = np.argsort(validation_scores)[::-1]
2 selected = order[:capacity]
3 precision_at_capacity = y_validation.iloc[selected].mean()
4 recall_covered = y_validation.iloc[selected].sum() / y_validation.sum()

```

Le seuil est ensuite gelé ou recalculé selon une règle de capacité documentée, puis évalué sur le test. Choisir le seuil directement sur le test biaise l'estimation de sa performance.

9.12.7 Discrimination et calibration

Un modèle discrimine bien s'il classe les cas positifs avant les négatifs. Il est calibré si, parmi les observations scorées autour de 0,2, environ 20% sont positives. Une bonne ROC-AUC n'implique pas une bonne calibration. Si la décision utilise directement une probabilité et des coûts, inspecter une courbe de calibration et, si nécessaire, calibrer sur des données distinctes.

9.12.8 Surapprentissage diagnostiqué par courbes

Un écart élevé entre score d'entraînement et validation indique un modèle trop adapté aux particularités d'apprentissage. Une courbe d'apprentissage trace performance selon la quantité de données : deux scores faibles suggèrent manque de signal ou biais ; un grand écart persistant suggère variance, fuite ou flexibilité excessive. Réduire le modèle, régulariser, améliorer les variables ou collecter des données répondent à des diagnostics différents.

9.12.9 Analyse d'erreurs par tranches

Examiner des faux positifs et faux négatifs individuels révèle erreurs d'étiquette, variables manquantes et populations non couvertes. Compléter par des métriques selon période, région, canal et groupes pertinents. Un score global acceptable peut masquer un échec grave sur une petite population. Les tranches doivent être choisies par risque ou hypothèse, pas seulement parce qu'une valeur extrême est apparue après exploration.

9.12.10 Du test au suivi en production

Le déploiement change le système : l'action influence les résultats futurs, les populations dérivent et le taux de base évolue. Définir avant lancement qualité des entrées, distribution des scores, couverture, latence, métriques avec délai d'étiquette, gardes métier et procédure de retrait. La performance hors échantillon est une estimation initiale, pas une garantie permanente.

Dossier minimal de défense

Formulation temporelle ; dictionnaire des variables ; audit et baselines ; stratégie de séparation ; pipeline reproductible ; métriques globales et à capacité ; choix du seuil ; analyse d'erreurs ; limites ; risques de déploiement ; plan de suivi. Chaque graphique ou table doit soutenir une décision identifiable.

9.13 scikit-learn en profondeur : une API pour expérimenter sans fuite

9.13.1 Le contrat des estimateurs

Un estimateur apprend dans `fit(X, y)` et stocke son état avec des attributs se terminant souvent par un underscore, comme `classes_`. `transform` produit une nouvelle représentation ; `predict` une décision ; `predict_proba` des scores par classe lorsque l'estimateur le permet. Les paramètres de configuration sont accessibles par `get_params` et peuvent être réglés dans les objets composés via `nom__parametre`.

Cette convention rend les pipelines clonables par la validation croisée. Modifier un objet global depuis un transformateur personnalisé brise ce contrat et peut mélanger l'état entre folds.

9.13.2 Le split est un modèle du futur

Une séparation aléatoire i.i.d. suppose que les observations futures ressemblent à un échantillon échangeable du présent. Si plusieurs lignes appartiennent au même patient, client, appareil ou site, utiliser un split groupé. Si le futur arrive après le passé, utiliser une séparation temporelle et reproduire le délai réel des variables et étiquettes.

Structure	Risque d'un split aléatoire	Stratégie
classe rare i.i.d.	folds sans positifs	stratification
plusieurs lignes par entité	identité partagée	groupes
temps et dérive	entraînement depuis le futur	découpe temporelle
sites distincts	généralisation locale seulement	validation par site

Le jeu de test final simule une seule évaluation future. Tout choix guidé par ce test le transforme en jeu de validation et rend son score optimiste.

9.13.3 Prétraitement hétérogène avec ColumnTransformer

Colonnes numériques et catégorielles ont des transformations différentes. Chaque étape apprend uniquement sur le fold d'entraînement lorsqu'elle est placée dans le pipeline.

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.impute import SimpleImputer
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.pipeline import Pipeline
5 from sklearn.preprocessing import OneHotEncoder, StandardScaler
6
7 numeric = Pipeline([
8     ("impute", SimpleImputer(strategy="median", add_indicator=True)),
9     ("scale", StandardScaler()),
10 ])
11 categorical = Pipeline([
12     ("impute", SimpleImputer(strategy="most_frequent")),
13     ("onehot", OneHotEncoder(handle_unknown="ignore")),
14 ])
15 preprocess = ColumnTransformer([
16     ("num", numeric, numeric_columns),
17     ("cat", categorical, categorical_columns),
18 ])
19 model = Pipeline([
20     ("preprocess", preprocess),
21     ("classifier", LogisticRegression(max_iter=2_000, class_weight="balanced")),
22 ])

```

`handle_unknown="ignore"` évite l'échec sur une catégorie future, mais transforme cette catégorie en ligne de zéros pour ce bloc. Il faut surveiller le taux de catégories inconnues ; la robustesse technique ne remplace pas l'observabilité.

9.13.4 Transformateurs personnalisés

Un transformateur doit apprendre seulement dans `fit`, retourner `self`, ne pas modifier son entrée par surprise et produire une forme stable. Avant d'en écrire un, chercher une primitive existante : moins de code réduit les erreurs de clonage, noms de variables et sérialisation.

9.13.5 Validation croisée et distribution des scores

La validation croisée produit une distribution conditionnelle à un schéma de découpe, pas une vérité universelle. Conserver scores d'entraînement, validation, durée et estimateurs si une analyse plus fine est prévue.

```

1 from sklearn.model_selection import StratifiedGroupKFold, cross_validate
2
3 cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=2026)
4 scores = cross_validate(
5     model, X_train, y_train,
6     groups=customer_group,
7     cv=cv,
8     scoring={"ap": "average_precision", "roc": "roc_auc", "recall": "recall"},
9     return_train_score=True,
10    n_jobs=-1,
11 )

```

Examiner chaque fold. Une moyenne stable peut masquer un seul site catastrophique ; une forte dispersion peut indiquer échantillon réduit, groupes hétérogènes ou erreur de formulation.

9.13.6 Recherche d'hyperparamètres

Une grille cartésienne dépense des essais sur des combinaisons peu informatives. Une recherche aléatoire explore souvent mieux plusieurs dimensions sous budget fixe. Dans les deux cas, le prétraitement doit rester dans l'objet recherché et le test final hors de la boucle.

```

1 from scipy.stats import loguniform
2 from sklearn.model_selection import RandomizedSearchCV
3
4 search = RandomizedSearchCV(
5     model,
6     param_distributions={"classifier__C": loguniform(1e-3, 1e2)},
7     n_iter=30,
8     scoring="average_precision",
9     cv=cv,
10    n_jobs=-1,
11    random_state=2026,
12    return_train_score=True,
13 )
14 search.fit(X_train, y_train, groups=customer_group)

```

Si la même validation choisit modèle, variables, hyperparamètres et seuil après de nombreux essais, sa performance devient optimiste. Une boucle imbriquée estime mieux la procédure de sélection ; un test final confirme ensuite une configuration gelée.

9.13.7 Scoring et orientation

Dans scikit-learn, certains scorers de perte sont nommés avec `neg_` car la convention de sélection maximise le score. `neg_mean_absolute_error=-3` correspond à une MAE de 3. Toujours convertir vers une unité métier dans le rapport.

Pour les classes rares, l'accuracy et même ROC-AUC peuvent donner une image trop favorable. Average precision résume le classement précision-rappel ; la matrice de confusion au seuil opérationnel relie le score aux volumes et coûts.

9.13.8 Seuil appris séparément

Le modèle apprend un score ; l'organisation choisit une action. Avec une capacité K , classer les scores de validation et sélectionner les K plus élevés peut être plus fidèle qu'un seuil 0,5. Geler la règle, puis évaluer une fois sur le test. En production, décider si K est fixe ou si un seuil probabiliste doit être conservé.

9.13.9 Calibration

Une probabilité calibrée vérifie approximativement que les objets scorés autour de p sont positifs avec fréquence p . La calibration doit utiliser des données séparées de l'entraînement de base, souvent par validation croisée. Mesurer courbe de calibration, score de Brier et comportement par segment. Un changement de prévalence peut dégrader la calibration sans modifier le classement.

9.13.10 Déséquilibre de classes

`class_weight="balanced"` modifie la perte d'entraînement ; il ne recrée pas de nouvelles informations et peut déplacer la calibration. Un sur-échantillonnage doit se produire seulement dans les folds d'entraînement, jamais avant la séparation. Comparer également une baseline de classement ou de règle métier et évaluer les volumes au seuil.

9.13.11 Interprétation conditionnelle au modèle

Les coefficients d'un pipeline linéaire nécessitent les noms de variables après transformation et dépendent de l'échelle. Une importance par permutation mesure la baisse de score lorsque la variable est perturbée sur un jeu donné. Avec variables corrélées, permuter l'une peut sembler peu important car l'autre conserve l'information. Ni coefficient ni importance ne démontre une cause.

```
1 from sklearn.inspection import permutation_importance
2
3 importance = permutation_importance(
4     fitted_model, X_valid, y_valid,
5     scoring="average_precision", n_repeats=20,
6     random_state=2026, n_jobs=-1,
7 )
```

Les courbes de dépendance partielle et ICE décrivent le comportement du modèle sous des variations parfois peu plausibles. Les confronter à la distribution des données.

9.13.12 Courbes d'apprentissage et de validation

Une courbe d'apprentissage compare train et validation selon la quantité de données. Deux scores faibles et proches suggèrent variables insuffisantes, fort bruit ou modèle trop simple ; un écart persistant suggère variance, fuite possible ou modèle trop flexible. Une courbe de validation fait varier un hyperparamètre ; elle ne doit pas devenir une nouvelle occasion de choisir sur le test.

9.13.13 Incertitude des métriques

Un seul score contient du bruit d'échantillonnage. Utiliser bootstrap au niveau de l'unité indépendante ou variabilité entre folds, en respectant groupe et temps. Pour comparer deux modèles, rééchantillonner les différences sur les mêmes unités plutôt que deux distributions indépendantes. Les intervalles ne compensent pas une séparation inadéquate.

9.13.14 Sérialisation et contrat de version

Un pipeline sérialisé dépend souvent des versions de Python, NumPy et scikit-learn et ne doit être chargé que depuis une source de confiance. Conserver code de création, artefact, versions, schéma d'entrée, ordre et type des colonnes, catégories, seuil et empreinte des données. Tester le chargement dans l'environnement cible sur un lot de référence et comparer scores.

9.13.15 Monitoring en production

Avant le déploiement, définir :

- qualité d'entrée : types, plages, manque, catégories inconnues ;
- distribution : variables clés et scores, par segment et dans le temps ;
- opération : latence, débit, erreurs, couverture et capacité ;
- performance retardée : labels disponibles, précision, rappel, calibration et valeur ;
- action : fréquence de revue, propriétaire, seuil d'alerte et plan de retrait.

Une dérive statistique n'implique pas une baisse de performance ; une stabilité marginale ne garantit pas l'absence de dérive conditionnelle. Le monitoring déclenche une enquête, il ne réentraîne pas automatiquement sans validation.

9.13.16 Boucle de rétroaction

Lorsque le score déclenche une action, les labels futurs dépendent de cette action : contacter un client peut empêcher son attrition. L'évaluation naïve sur les seules personnes contactées confond sélection et effet. Prévoir expérimentation, groupe de contrôle ou méthodes causales adaptées. Un système prédictif modifie son propre environnement de données.

Checklist scikit-learn

Formulation temporelle ; split aligné au futur ; baseline ; pipeline complet ; validation sans test ; métriques liées à la décision ; seuil choisi hors test ; incertitude et segments ; interprétation non causale ; artefact et versions ; contrat d'entrée ; monitoring ; stratégie de retrait et boucle de rétroaction.

9.14 Travail appliqué

Pour douze variables candidates d'un problème d'attrition, décider lesquelles sont disponibles au moment de la prédiction et construire une chronologie.

Comparer un classifieur majoritaire, une règle métier et une régression logistique. Présenter accuracy, rappel, précision et average precision.

Construire un `ColumnTransformer` avec imputation, normalisation et encodage. Vérifier que le test n'intervient dans aucun `fit`.

Choisir 500 dossiers parmi les scores les plus élevés, mesurer précision et rappel couverts, puis comparer la valeur attendue à deux seuils alternatifs.

Comparer régression logistique et gradient boosting avec validation croisée adaptée aux groupes. Inclure le prétraitement dans chaque pipeline, régler quelques hyperparamètres dans une boucle interne et conserver un test final intact. Rapporter scores par fold, coût de calcul et stabilité.

À partir d'un pipeline entraîné, produire contrat d'entrée, versions, seuil, métriques par segment, latence, stratégie de sérialisation, contrôles de dérive, délai des étiquettes et conditions de retrait. Concevoir un test de compatibilité sur un lot de référence.

9.15 À retenir

L'évaluation crédible simule l'usage futur. La formulation temporelle empêche la fuite, le pipeline protège le prétraitement, la baseline donne un point de comparaison et le seuil relie le modèle à une capacité et à des coûts réels.

Sources principales : [22], [23], [24]

Troisième partie

Complément : applications d'IA généralive

Bonus — Construire des applications d’IA générative avec Hugging Face

Du modèle pré-entraîné à une application mesurée, sûre et observable

10.1 Question directrice

Une équipe souhaite transformer des descriptions libres d’incidents en trois sorties : une catégorie, un résumé de deux phrases et une proposition de prochaine action. Le prototype doit fonctionner sur quelques exemples, mais aussi rendre visibles sa latence, son coût, ses erreurs et les données envoyées à un service distant. Comment passer d’un appel de modèle spectaculaire à une application que l’on peut tester et maintenir ?

Ce chapitre distingue volontairement six objets : le *modèle*, le tokenizer, la tâche d’inférence, les données, l’orchestration Python et l’évaluation. Confondre ces niveaux produit des notebooks fragiles. Les séparer permet de remplacer un modèle, de tester la logique sans GPU et de comparer exécution locale et API hébergée.

10.2 Cartographie de l’écosystème

Quatre bibliothèques, quatre responsabilités

- `huggingface_hub` accède au Hub, télécharge des artefacts et expose un client d’inférence hébergée ;
- `transformers` charge tokenizers et modèles, puis fournit des pipelines pour l’inférence locale ;
- `datasets` charge, transforme, filtre et diffuse des jeux de données ;
- `evaluate` ou des métriques explicites mesurent les sorties, mais une évaluation métier exige souvent une grille conçue pour le cas d’usage.

Un dépôt du Hub peut contenir poids, configuration, tokenizer, carte de modèle et code éventuellement personnalisé. Un identifiant de dépôt n’est pas une garantie de qualité : la licence, la provenance des données, la langue, le contexte maximal, les limites et les instructions de chargement font partie de la sélection.

10.2.1 Lire une model card comme une fiche de risque

Avant tout téléchargement, relever : tâche, langues, licence, taille des poids, format, architecture, données d’entraînement déclarées, évaluations, biais connus et révision. Une valeur de benchmark

ne prédit pas nécessairement la performance sur les incidents de l'entreprise. Une carte incomplète augmente l'incertitude ; elle ne la fait pas disparaître.

10.3 Deux chemins d'inférence

10.3.1 Pipeline local

Le pipeline assemble prétraitement, modèle et post-traitement pour une tâche. Il est excellent pour un premier test et pour comprendre le contrat entrée-sortie.

```

1 from transformers import pipeline
2
3 model_id = "HuggingFaceTB/SmolLM2-1.7B-Instruct"
4 generator = pipeline(
5     task="text-generation",
6     model=model_id,
7     device_map="auto",
8 )
9
10 messages = [
11     {"role": "system", "content": "Tu classes des incidents de support."},
12     {"role": "user", "content": "Le paiement est débité deux fois."},
13 ]
14 result = generator(messages, max_new_tokens=80, do_sample=False)
15 print(result[0]["generated_text"])

```

Le calcul local garde davantage de contrôle sur les données et la version des poids, mais demande mémoire, temps de chargement et capacité de calcul. `device_map="auto"` place le modèle selon le matériel disponible ; ce choix doit être mesuré, non supposé.

10.3.2 Inférence hébergée

Le client du Hub permet d'appeler des fournisseurs compatibles sans charger les poids localement. Le jeton reste dans une variable d'environnement et ne doit jamais être écrit dans le notebook ou commité.

```

1 import os
2 from huggingface_hub import InferenceClient
3
4 client = InferenceClient(
5     provider="auto",
6     api_key=os.environ["HF_TOKEN"],
7     timeout=30,
8 )
9
10 response = client.chat.completions.create(
11     model="your-approved-model-id",
12     messages=[
13         {"role": "system", "content": "Réponds en JSON strict."},
14         {"role": "user", "content": "Classe: paiement débité deux fois"},
15     ],
16     max_tokens=120,
17     temperature=0,
18 )
19 text = response.choices[0].message.content

```

L'inférence distante simplifie l'accès au matériel mais introduit réseau, authentification, limites de débit, coût variable et transfert de données. Une politique de données doit préciser ce qui peut quitter le système, la rétention et la journalisation autorisées.

Choisir local ou hébergé

Comparer sur le même corpus : qualité, latence médiane et haute, débit, mémoire, coût par requête, disponibilité, confidentialité, contrôle de version et charge opérationnelle. Il n'existe pas de gagnant universel ; le choix dépend du contrat du produit.

10.4 Tokenisation : le texte devient des identifiants

Un modèle ne lit pas directement des caractères. Le tokenizer segmente le texte en unités, les associe à des identifiants et ajoute des jetons spéciaux. Le nombre de mots n'est donc pas le nombre de tokens ; les langues, nombres et fragments de code peuvent avoir des coûts différents.

```
1 from transformers import AutoTokenizer
2
3 tokenizer = AutoTokenizer.from_pretrained(model_id)
4 encoded = tokenizer(
5     "Un remboursement de 125,90 EUR est bloqué.",
6     return_tensors="pt",
7     truncation=True,
8     max_length=128,
9 )
10 print(encoded["input_ids"].shape)
11 print(tokenizer.convert_ids_to_tokens(encoded["input_ids"][0]))
```

Le budget de contexte comprend le prompt, l'historique, les documents récupérés et la réponse. Une troncature silencieuse peut supprimer l'information la plus importante. Il faut définir une stratégie : rejeter, résumer, fenêtrer, sélectionner ou hiérarchiser.

10.4.1 Chat templates

Les modèles conversationnels attendent souvent des marqueurs propres à leur entraînement. Un template transforme la liste de messages en séquence exacte. Construire soi-même une chaîne "User: ... Assistant:" peut dégrader fortement le résultat.

```
1 prompt_ids = tokenizer.apply_chat_template(
2     messages,
3     add_generation_prompt=True,
4     return_tensors="pt",
5 )
```

10.5 Contrôler la génération

La génération est une procédure de sélection du prochain token. `max_new_tokens` borne la sortie. `do_sample=False` produit un décodage déterministe dans des conditions fixes. Avec échantillonnage, `temperature`, `top_p` et une graine modifient la diversité, mais la reproductibilité peut encore dépendre du matériel et de la version des bibliothèques.

Pour une extraction structurée, une température basse ne suffit pas. Il faut un schéma, une validation et une stratégie de reprise.

```

1 from dataclasses import dataclass
2 import json
3
4 ALLOWED = {"paiement", "contrat", "sinistre", "autre"}
5
6 @dataclass(frozen=True)
7 class IncidentResult:
8     category: str
9     summary: str
10    next_action: str
11
12 def parse_result(raw: str) -> IncidentResult:
13     obj = json.loads(raw)
14     required = {"category", "summary", "next_action"}
15     if set(obj) != required:
16         raise ValueError("unexpected schema")
17     if obj["category"] not in ALLOWED:
18         raise ValueError("unknown category")
19     if not all(isinstance(obj[k], str) and obj[k].strip() for k in required):
20         raise ValueError("empty or non-string field")
21     return IncidentResult(**obj)

```

Le validateur transforme une attente textuelle en contrat exécutable. En cas d'échec, on peut demander une seule réparation, utiliser un classifieur de secours ou transmettre à un humain. Une boucle de reprise sans borne crée coût et latence incontrôlés.

10.6 Prompts comme interfaces versionnées

Un bon prompt spécifie le rôle utile, les entrées délimitées, la sortie attendue, les catégories autorisées et la conduite en cas d'information insuffisante. Il ne doit pas contenir de secret. Les exemples few-shot doivent représenter des cas normaux et limites.

```

1 SYSTEM_PROMPT = """
2 Tu analyses un incident. N'invente aucune information.
3 Catégories: paiement, contrat, sinistre, autre.
4 Retourne seulement un objet JSON avec exactement:
5 category, summary, next_action.
6 Si l'information manque, indique-le dans next_action.
7 """.strip()
8
9 def build_messages(description: str) -> list[dict[str, str]]:
10     safe = description.strip()[:8_000]
11     return [
12         {"role": "system", "content": SYSTEM_PROMPT},
13         {"role": "user", "content": f"<incident>{safe}</incident>"},
14     ]

```

Versionner le prompt avec le modèle, les paramètres et le jeu d'évaluation. Changer une phrase peut déplacer les erreurs ; le prompt est donc du code de produit.

10.7 Datasets : données reproductibles et diffusion

`load_dataset` charge depuis le Hub, des fichiers locaux ou des formats standards. Un split doit être choisi explicitement. Le mode streaming évite de télécharger tout le corpus mais modifie les opérations disponibles et la manière de mélanger les exemples.

```

1 from datasets import load_dataset
2
3 ds = load_dataset("json", data_files={
4     "train": "data/incidents_train.jsonl",
5     "test": "data/incidents_test.jsonl",
6 })
7
8 def normalize(row):
9     return {
10         "text": row["text"].strip(),
11         "label": row["label"].lower(),
12     }
13
14 clean = (ds
15     .filter(lambda row: bool(row["text"].strip()))
16     .map(normalize))
17 print(clean["train"].features)

```

Une transformation map doit être déterministe si l'on veut profiter du cache. Ne jamais placer une clé d'API dans une colonne. Pour les grands jeux, sélectionner les colonnes utiles, traiter par lots et documenter la révision du dataset.

10.7.1 Prévenir la contamination

Un split aléatoire de lignes est insuffisant si plusieurs tickets du même client, du même dossier ou du même modèle de texte se ressemblent. Grouper avant de séparer. Dédupliquer exactement puis approximativement. Conserver un jeu de test gelé et séparé de la mise au point du prompt.

10.8 Embeddings et recherche augmentée

Un embedding représente un texte par un vecteur. Une application de recherche augmentée encode les documents et la question, retrouve des voisins, puis donne au générateur un contexte limité et sourcé. Le pattern rappelle les chapitres DSA : indexer pour éviter de rebalayer naïvement tous les objets à chaque requête.

Pour enseigner le mécanisme, une recherche exacte avec NumPy suffit :

```

1 import numpy as np
2
3 def l2_normalize(matrix: np.ndarray) -> np.ndarray:
4     norms = np.linalg.norm(matrix, axis=1, keepdims=True)
5     return matrix / np.clip(norms, 1e-12, None)
6
7 def top_k_cosine(query, documents, k=3):
8     q = query / max(np.linalg.norm(query), 1e-12)
9     docs = l2_normalize(documents)
10    scores = docs @ q
11    k = min(k, len(scores))
12    idx = np.argpartition(scores, -k)[-k:]
13    idx = idx[np.argsort(scores[idx])[:-1]]
14    return list(zip(idx.tolist(), scores[idx].tolist()))

```

Le scan exact coûte $O(nd)$ en temps pour n documents de dimension d et $O(n)$ pour les scores. Un index de voisinage approximatif échange exactitude, mémoire et temps de construction contre une recherche plus rapide. Mesurer le rappel de récupération avant d'évaluer la génération : le modèle ne peut citer une preuve jamais récupérée.

10.9 Architecture testable d'une application

Séparer la construction du prompt, le transport vers le modèle, le parsing, la logique métier et les métriques. Une interface minimale rend les tests indépendants du fournisseur.

```

1 from typing import Protocol
2
3 class TextModel(Protocol):
4     def generate(self, messages: list[dict[str, str]]) -> str: ...
5
6 class IncidentService:
7     def __init__(self, model: TextModel):
8         self.model = model
9
10    def analyze(self, description: str) -> IncidentResult:
11        messages = build_messages(description)
12        raw = self.model.generate(messages)
13        return parse_result(raw)
14
15 class FakeModel:
16     def generate(self, messages):
17         return ({'category': 'paiement', 'summary': 'Double débit.',
18                 'next_action': 'Vérifier les deux transactions.'})
19
20 assert IncidentService(FakeModel()).analyze("double débit").category == "paiement"

```

Cette composition reprend le chapitre OOP : la dépendance instable est injectée derrière un contrat. Les tests unitaires couvrent le code déterministe ; un jeu d'évaluation couvre le comportement du modèle ; quelques tests en ligne surveillent l'intégration.

10.10 Évaluer avant de démontrer

Construire un tableau d'évaluation avec identifiant, entrée, cible ou critères, groupe, sensibilité et difficulté. Pour une classification : matrice de confusion, précision, rappel et taux d'abstention. Pour un résumé : grille humaine sur fidélité, couverture et concision. Pour une réponse sourcée : exactitude de récupération, fidélité à la preuve et qualité de citation.

Trois couches de mesure

1. **Composant** : schéma valide, longueur, langue, catégorie autorisée.
2. **Tâche** : exactitude ou grille de qualité sur un corpus gelé et segmenté.
3. **Système** : latence, coût, erreurs réseau, sécurité, escalades humaines et résultat métier.

Une moyenne masque les échecs rares. Rapporter par langue, longueur, catégorie, période et niveau de risque. Inspecter un échantillon d'erreurs et constituer une taxonomie : mauvais format, hallucination, preuve manquante, mauvaise récupération, instruction ambiguë ou échec d'intégration.

10.11 Résilience, sécurité et observabilité

Une application doit borner ses ressources et traiter les entrées comme non fiables.

— Définir timeouts, nombre maximal de reprises et backoff pour les erreurs transitoires.

- Ne pas journaliser de texte sensible par défaut ; préférer identifiants, tailles, hachages contrôlés et métriques agrégées.
- Filtrer ou refuser les données interdites avant l'appel distant.
- Délimiter les documents récupérés : ils peuvent contenir des instructions adverses.
- Valider la sortie et appliquer les autorisations après le modèle ; le modèle ne décide jamais seul d'une action irréversible.
- Enregistrer modèle, révision, paramètres, version du prompt, latence, statut et résultat de validation.

Point de vigilance — Prompt injection

Un document peut dire « ignore les instructions précédentes ». Il doit rester une donnée, pas devenir une autorité. Le code contrôle les outils, les permissions et les actions ; le texte du modèle ne peut pas élargir ces droits.

10.12 Performance : batch, cache et budget

Le débit d'un modèle local s'améliore souvent par lots, mais un grand batch augmente la mémoire et peut dégrader la latence individuelle. Le padding gaspille du calcul lorsque les longueurs diffèrent fortement : regrouper des entrées de tailles proches peut aider. Pour un service distant, limiter la concurrence et mesurer les réponses de limitation.

Un cache n'est correct que si sa clé inclut au minimum modèle, révision, prompt, paramètres et entrée normalisée. Ne pas mettre en cache des réponses contenant des données sensibles sans politique explicite. Estimer un budget en tokens d'entrée et de sortie, puis vérifier les mesures réelles au lieu de déduire le coût du nombre de caractères.

10.13 Étude de cas : triage d'incidents

Le pipeline final suit une séquence observable : validation de l'entrée, retrait des champs interdits, récupération éventuelle de procédures, construction du prompt, génération, validation JSON, contrôle métier, journalisation minimale et restitution. Trois chemins de sortie sont prévus : résultat accepté, réparation bornée, escalade humaine.

La baseline n'utilise pas de LLM : règles sur quelques mots-clés et catégorie majoritaire. Le modèle doit battre cette baseline sur les segments importants, sans dépasser budget et latence. Si l'objectif principal est seulement quatre catégories stables, un classifieur supervisé classique peut être plus rapide, moins coûteux et plus facile à calibrer.

10.14 Approfondissement : du prototype au système génératif

10.14.1 Décomposer l'incertitude

Les échecs d'une application générative ne proviennent pas tous du modèle. Une mauvaise réponse peut venir d'une donnée absente, d'une troncature, d'une récupération médiocre, d'un prompt ambigu, d'un décodage trop aléatoire, d'un parsing fragile ou d'une règle métier incorrecte. Cette décomposition guide l'instrumentation : longueur tokenisée, identifiants des documents récupérés, score de similarité, version du prompt, sortie brute conservée seulement lorsque la politique l'autorise, statut de validation et décision finale.

Pour diagnostiquer, fixer une entrée et remplacer un composant à la fois. Si le bon document est fourni manuellement et que la réponse reste fausse, la récupération n'est probablement pas le goulot. Si le résultat brut est correct mais refusé, le contrat de parsing est en cause. Cette méthode est l'équivalent système d'un test unitaire.

10.14.2 Révisions et reproductibilité

Un identifiant de modèle pointe souvent vers une branche mobile. Pour une expérience reproductible, enregistrer une révision précise des poids et du tokenizer, les versions de `transformers`, de PyTorch et des dépendances, le type numérique, le matériel et les paramètres de génération. Même ainsi, certaines opérations parallèles peuvent ne pas être strictement déterministes ; il faut distinguer répétabilité exacte et stabilité des métriques.

Le jeu d'évaluation doit lui aussi être versionné. Un résultat sans empreinte du corpus ne permet pas de savoir si une amélioration vient du modèle ou d'un changement d'exemples.

10.14.3 Longueur, complexité et mémoire d'attention

Pour un transformer standard, le coût de l'attention croît approximativement comme le carré de la longueur de séquence dans sa forme naïve. Les implémentations modernes changent les constantes et parfois l'algorithme, mais ajouter du contexte reste coûteux. Plus de documents n'améliorent pas automatiquement une réponse : le signal utile peut être noyé, contredit ou tronqué. La sélection de contexte est donc un problème d'information, pas un concours de taille de prompt.

Pour la génération autoregressive, chaque token de sortie dépend des précédents. La latence se sépare en temps jusqu'au premier token et temps par token suivant. Mesurer seulement la durée totale masque l'expérience de streaming.

10.14.4 Récupération : mesurer séparément chaque étage

Un système RAG comprend ingestion, découpage, encodage, indexation, recherche, éventuel reranking et génération. Évaluer :

1. si un passage pertinent existe dans le corpus ;
2. s'il survit au découpage et contient assez de contexte ;
3. s'il apparaît dans le top- k ;
4. si le générateur l'utilise fidèlement ;
5. si la citation permet à l'utilisateur de vérifier.

Optimiser seulement la réponse finale rend impossible l'attribution d'un gain.

10.14.5 Choisir le découpage

Un segment trop court perd les dépendances ; un segment trop long dilue le score et consomme le contexte. Le recouvrement réduit les coupures mais duplique stockage et résultats. Une bonne unité suit la structure documentaire : section, procédure, article ou paragraphe avec métadonnées. Le corpus doit conserver source, date, autorisation et identifiant stable.

10.14.6 Outils et actions

Certains modèles peuvent proposer un appel d'outil sous forme structurée. Cette proposition n'est pas l'exécution. Le programme vérifie le nom de l'outil, valide chaque argument, applique

permissions et limites, demande confirmation si l'action est sensible, exécute, puis retourne un résultat minimal au modèle. Utiliser une liste blanche ; ne jamais exécuter du code ou une commande construite librement par la sortie.

10.14.7 Évaluation statistique de deux versions

Comparer deux prompts sur les mêmes exemples produit des observations appariées. Conserver le verdict par cas, pas seulement deux moyennes. Examiner les régressions, segments et intervalles par bootstrap au niveau de l'unité indépendante. Pour une note humaine, masquer la version et randomiser l'ordre afin de limiter les biais de présentation. Mesurer l'accord entre évaluateurs et documenter la grille.

Une différence statistique n'est utile que si elle améliore le système sous contraintes de coût et latence. On peut définir une fonction de perte : coût d'une mauvaise catégorie, coût d'une escalade, coût d'inférence et pénalité de latence. Le seuil d'acceptation devient alors une décision explicite.

10.14.8 Une boucle de développement défendable

Workflow expérimental

Formuler la tâche et la baseline ; geler un petit jeu de développement ; écrire les critères et segments ; implémenter un pipeline minimal ; enregistrer versions et mesures ; analyser les erreurs ; modifier une hypothèse ; réévaluer ; enfin confirmer sur un test resté intact.

Le journal d'expérience contient une hypothèse, pas seulement des paramètres. Exemple : « ajouter deux exemples de refus réduira les catégories inventées sans augmenter le taux d'abstention de plus de deux points ». Une expérience falsifiable enseigne davantage qu'une suite de changements simultanés.

10.14.9 Quand ne pas utiliser un modèle génératif

Ne pas employer de génération libre lorsqu'une règle déterministe suffit, lorsqu'une erreur rare peut causer une action irréversible sans validation, lorsque les données ne peuvent être traitées dans l'environnement choisi ou lorsque le bénéfice ne couvre pas coût et complexité. Une recherche classique, un modèle supervisé, un formulaire mieux conçu ou une règle métier peuvent constituer une meilleure solution.

10.15 Travail appliqué

À partir de trois fiches de modèles fournies, choisir un candidat pour des incidents en français. Construire une matrice de décision : licence, langue, taille, exécution, contexte, évaluation, risques et informations manquantes. Justifier aussi un modèle rejeté.

Écrire un analyseur de résultat JSON pour le triage. Partir d'un accès direct aux clés, puis concevoir validation du schéma, catégories autorisées, gestion d'erreur et stratégie de reprise bornée. Tester au moins six sorties invalides.

Étant donné une matrice d'embeddings de documents et un vecteur requête, retourner les k documents les plus proches par similarité cosinus. Commencer par une boucle, puis vectoriser avec NumPy. Analyser temps et espace et traiter les vecteurs nuls.

Concevoir une fonction qui exécute un modèle sur un corpus gelé et produit validité du schéma, exactitude de catégorie, latence médiane et percentile 95, taux d'erreur et résumé par segment. Le modèle doit pouvoir être remplacé par un faux objet dans les tests.

Une application envoie des dossiers complets à une API, conserve le jeton dans le notebook, réessaie sans limite et exécute directement l'action proposée. Identifier les risques, puis proposer une architecture locale ou hébergée avec frontières de données, permissions, timeouts, validation, observabilité et escalade humaine.

10.16 À retenir

Une application générative est un système de données probabiliste entouré de code déterministe. La qualité dépend autant du corpus d'évaluation, du schéma, des contrôles et de l'architecture que du modèle. Prototyper avec `pipeline` ou `InferenceClient` est rapide ; rendre le service défendable exige versions, mesures, limites et voies d'échec.

Sources principales : [25], [26], [27], [9], [22]

Quatrième partie

Corrigés raisonnés

Corrigés du chapitre 1

Exercice 1.1 — Valeur et type

Explication du problème. L'exercice distingue division réelle, division entière, reste, répétition de chaîne, comparaison, négation, conversion et longueur.

Intuition. Pour chaque expression, identifier d'abord l'opérateur et le type de ses opérandes. Le type du résultat vient ensuite.

Solution directe ou force brute. Les résultats sont respectivement 3.5 (float), 3 (int), 1 (int), "77" (str), True (bool), False (bool), 13 (int) et 4 (int).

Solution optimisée ou structurée. Classifier chaque expression dans une table opérandes–opérateur–type–valeur rend le raisonnement vérifiable. Aucune accélération n'est attendue : l'amélioration porte ici sur la méthode.

Pseudo-code. Pour chaque expression : déterminer les types d'entrée, appliquer la règle de l'opérateur, puis annoncer valeur et type de sortie.

Vérification Python.

```
1 expressions = [7 / 2, 7 // 2, 7 % 2, "7" * 2,  
2             3 < 5, not True, int("12") + 1, len("data")]  
3 for result in expressions:  
4     print(repr(result), type(result).__name__)
```

Exercice 1.2 — Tarif de livraison

Explication du problème. La fonction calcule un tarif et refuse un domaine invalide.

Intuition. Le poids détermine la partie variable ; le booléen ajoute ou non le supplément express.

Solution directe ou force brute. Calculer $4 + 1.5 * \text{weight}$, puis ajouter 6 si `express` est vrai. La validation précède le calcul.

Solution optimisée ou structurée. Renvoyer une valeur numérique et laisser l'affichage à l'appelant. Aucune technique plus avancée n'est nécessaire.

Pseudo-code.

```
1 SI poids <= 0: signaler une erreur
2 prix <- 4 + 1.5 * poids
3 SI express: prix <- prix + 6
4 RETOURNER prix arrondi a deux decimales
```

Python.

```
1 def shipping_cost(weight, express=False):
2     if weight <= 0:
3         raise ValueError("weight must be positive")
4     price = 4 + 1.5 * weight
5     if express:
6         price += 6
7     return round(price, 2)
```

Tests et coût. Tester poids 1, poids décimal, express vrai et poids nul. Le temps et l'espace auxiliaire sont constants.

Exercice 1.3 — Compter les caractères

Explication du problème. Chaque caractère peut augmenter un compteur de lettres, de chiffres ou d'espaces.

Intuition. Trois compteurs suffisent ; les catégories sont testées pour chaque caractère.

Solution directe ou force brute. Parcourir le texte et utiliser `isalpha`, `isdigit` et `isspace`.

Solution optimisée ou structurée. Une chaîne de `if/elif` évite de compter deux fois un caractère ; la ponctuation reste volontairement hors des trois compteurs.

Pseudo-code.

```
1 lettres, chiffres, espaces <- 0, 0, 0
2 POUR caractere dans texte:
3     SI lettre: incrementer lettres
4     SINON SI chiffre: incrementer chiffres
5     SINON SI espace: incrementer espaces
6 RETOURNER les trois compteurs
```

Python.

```
1 def character_counts(text):
2     counts = {"letters": 0, "digits": 0, "spaces": 0}
3     for char in text:
4         if char.isalpha():
5             counts["letters"] += 1
6         elif char.isdigit():
7             counts["digits"] += 1
8         elif char.isspace():
9             counts["spaces"] += 1
10    return counts
```

Tests et travail effectué. "A2 b!" donne 2 lettres, 1 chiffre et 1 espace. La chaîne vide donne trois zéros. La boucle examine chaque caractère une fois et ne conserve que trois compteurs.

Exercice 1.4 — Somme d'un intervalle

Explication du problème. Les deux bornes sont incluses. Si elles sont inversées, le contrat choisi retourne zéro.

Intuition. La borne supérieure de `range` est exclue ; utiliser `stop + 1`.

Solution directe ou force brute. Ajouter successivement chaque entier.

Solution optimisée ou structurée. La formule arithmétique existe, mais la boucle est retenue ici pour apprendre `range` et l'accumulateur.

Pseudo-code.

```
1 SI debut > fin: RETOURNER 0
2 total <- 0
3 POUR valeur de debut a fin inclus: total <- total + valeur
4 RETOURNER total
```

Python.

```
1 def sum_between(start, stop):
2     total = 0
3     for value in range(start, stop + 1):
4         total += value
5     return total
```

Tests et travail effectué. Tester (3, 5), (5, 5), (−2, 2) et (5, 3). La boucle effectue une addition par entier de l'intervalle et ne conserve qu'un total.

Exercice 1.5 — Bulletin météo

Explication du problème. Il faut produire cinq résumés d'une liste non vide.

Intuition. Un seul parcours maintient somme, minimum, maximum et compteur.

Solution directe ou force brute. Les fonctions `sum`, `min` et `max` parcourent séparément la liste. Cette version est correcte et lisible.

Solution optimisée ou structurée. Un seul parcours expose l'état et prépare les réductions utilisées plus tard avec NumPy. Le gain reste une constante, pas une raison de compliquer le programme.

Pseudo-code.

```
1 SI liste vide: signaler une erreur
2 total <- 0; minimum <- premier; maximum <- premier; alertes <- 0
3 POUR valeur dans liste:
4     mettre a jour total, minimum, maximum et alertes
5 RETOURNER nombre, moyenne, minimum, maximum, alertes
```

Python.

```
1 def weather_report(values, threshold):
2     if not values:
3         raise ValueError("values must not be empty")
4     total = 0.0
5     low = high = values[0]
6     alerts = 0
7     for value in values:
8         total += value
9         if value < low:
10             low = value
11         if value > high:
12             high = value
13         if value > threshold:
14             alerts += 1
15     return {"count": len(values), "mean": total / len(values),
16           "min": low, "max": high, "alerts": alerts}
```

Tests et travail effectué. Tester une valeur, des nombres négatifs, une valeur égale au seuil et la liste vide. Chaque mesure est lue une fois; seuls quatre résumés numériques sont maintenus.

Corrigés du chapitre 2

Exercice 2.1 — Nettoyage stable

Explication du problème. Nettoyer des chaînes tout en conservant leur ordre.

Intuition. Chaque chaîne se traite indépendamment ; le résultat est ajouté seulement s'il reste non vide.

Solution directe ou force brute. Une boucle explicite rend les deux étapes visibles.

Solution optimisée ou structurée. Une compréhension avec expression d'affectation serait possible, mais une petite fonction garde l'intention plus lisible.

Pseudo-code.

```
1 sortie <- liste vide
2 POUR texte dans entrees:
3     valeur <- texte sans espaces extérieurs
4     SI valeur non vide: ajouter valeur
5 RETOURNER sortie
```

Python.

```
1 def clean_strings(values):
2     cleaned = []
3     for value in values:
4         normalized = value.strip()
5         if normalized:
6             cleaned.append(normalized)
7     return cleaned
```

Analyse. Temps $O(C)$ pour le nombre total de caractères, sortie $O(C)$, espace auxiliaire $O(1)$ hors sortie.

Exercice 2.2 — Fréquences de mots

Explication du problème. Définir une normalisation puis compter chaque mot.

Intuition. Le dictionnaire mémorise le nombre d'occurrences déjà lues.

Solution directe ou force brute. Pour chaque mot, recompter ses occurrences dans la liste coûte $O(n^2)$.

Solution optimisée ou structurée. Mettre à jour un compteur pendant un seul parcours donne $O(n)$ accès moyens, plus le coût de normalisation des caractères.

Pseudo-code.

```
1 remplacer la ponctuation choisie par des espaces
2 mettre en minuscules et découper
3 frequences <- dictionnaire vide
4 POUR mot: frequences[mot] <- ancienne valeur + 1
5 RETOURNER frequences
```

Python.

```
1 import string
2
3 def word_frequencies(text):
4     translation = str.maketrans(string.punctuation,
5                                 " " * len(string.punctuation))
6     words = text.lower().translate(translation).split()
7     counts = {}
8     for word in words:
9         counts[word] = counts.get(word, 0) + 1
10    return counts
```

Tests. Tester casse, ponctuation répétée, chaîne vide et apostrophe ; documenter le choix fait pour cette dernière.

Exercice 2.3 — Déduplication stable

Explication du problème. Conserver la première occurrence de chaque valeur et l'ordre.

Intuition. Il faut distinguer l'ordre de sortie et l'appartenance déjà vue.

Solution directe ou force brute. Tester `value not in result` recherche dans une liste grandissante et peut coûter $O(n^2)$.

Solution optimisée ou structurée. Une liste garde l'ordre ; un ensemble répond rapidement à la question d'appartenance.

Pseudo-code.

```
1 vus <- ensemble vide; sortie <- liste vide
2 POUR valeur:
3     SI valeur absente de vus:
4         ajouter valeur a vus et a sortie
5 RETOURNER sortie
```

Python.

```
1 def stable_unique(values):
2     seen = set()
3     result = []
4     for value in values:
5         if value not in seen:
6             seen.add(value)
7             result.append(value)
8     return result
```

Analyse. Temps moyen $O(n)$, espace auxiliaire $O(u)$ pour u valeurs uniques, sortie $O(u)$. Les valeurs doivent être hachables.

Exercice 2.4 — Index secondaire

Explication du problème. Associer chaque identifiant client à la position de son unique enregistrement.

Intuition. La présence préalable de la clé révèle immédiatement un doublon.

Solution directe ou force brute. Pour chaque ligne, rechercher l'identifiant dans les lignes précédentes coûte $O(n^2)$.

Solution optimisée ou structurée. Construire le dictionnaire pendant un parcours donne $O(n)$ en moyenne et rend les futures recherches moyennes constantes.

Pseudo-code.

```
1 index <- dictionnaire vide
2 POUR position, ligne:
3     cle <- ligne[customer_id]
4     SI cle dans index: signaler un doublon
5     index[cle] <- position
6 RETOURNER index
```

Python.

```
1 def build_customer_index(rows):
2     index = {}
3     for position, row in enumerate(rows):
4         customer_id = row["customer_id"]
5         if customer_id in index:
6             raise ValueError(f"duplicate customer_id: {customer_id}")
7         index[customer_id] = position
8     return index
```

Analyse. Temps moyen $O(n)$ et espace $O(n)$. Tester table vide, clé absente dans une ligne et doublon éloigné.

Corrigés du chapitre 3

Exercice 3.1 — Doublon

Explication du problème. Décider si une liste contient au moins deux valeurs égales.

Intuition. À la position courante, seule la question « déjà vue ? » compte.

Solution directe ou force brute. Comparer chaque paire (i, j) avec $i < j$ coûte $O(n^2)$ temps et $O(1)$ espace auxiliaire.

Solution optimisée ou structurée. L'ensemble des valeurs vues donne $O(n)$ temps moyen et $O(n)$ espace. Trier puis comparer les voisins donne $O(n \log n)$; `sorted` alloue une copie, tandis que `sort` modifie l'entrée.

Pseudo-code.

```
1 vus <- ensemble vide
2 POUR valeur:
3     SI valeur dans vus: RETOURNER vrai
4     ajouter valeur a vus
5 RETOURNER faux
```

Python.

```
1 def contains_duplicate(values):
2     seen = set()
3     for value in values:
4         if value in seen:
5             return True
6         seen.add(value)
7     return False
```

Tests. Liste vide, valeur unique, doublon adjacent, doublon éloigné et éléments non hachables pour expliciter la précondition.

Exercice 3.2 — Deux prix pour un budget

Explication du problème. Trouver deux indices distincts dont les valeurs totalisent le budget.

Intuition. Pour un prix x , la seule valeur recherchée est $budget - x$.

Solution directe ou force brute. Deux boucles testent toutes les paires en $O(n^2)$ temps et $O(1)$ espace.

Solution optimisée ou structurée. Un dictionnaire valeur-vers-position indexe le préfixe. Temps moyen $O(n)$ et espace $O(n)$.

Pseudo-code.

```
1 positions <- dictionnaire vide
2 POUR indice, prix:
3     complement <- budget - prix
4     SI complement dans positions: RETOURNER positions[complement], indice
5     positions[prix] <- indice
6 RETOURNER aucun
```

Python.

```
1 def pair_for_budget(prices, budget):
2     positions = {}
3     for index, price in enumerate(prices):
4         complement = budget - price
5         if complement in positions:
6             return positions[complement], index
7         positions[price] = index
8     return None
```

Tests. [2, 7, 11], 9, aucune solution, deux prix égaux à positions différentes et présence de zéros.

Exercice 3.3 — Moyennes mobiles

Explication du problème. Calculer la moyenne de chaque fenêtre contiguë de largeur k .

Intuition. Deux fenêtres voisines partagent toutes leurs valeurs sauf une.

Solution directe ou force brute. Sommer chaque tranche coûte $O((n - k + 1)k)$, soit $O(nk)$, et alloue des tranches temporaires si elles sont matérialisées.

Solution optimisée ou structurée. Maintenir la somme courante donne $O(n)$ temps et $O(1)$ espace auxiliaire hors sortie.

Pseudo-code.

```
1 valider k; somme <- somme des k premieres valeurs
2 sortie <- [somme / k]
3 POUR droite de k a n-1:
4     somme <- somme + valeur[droite] - valeur[droite-k]
5     ajouter somme/k
6 RETOURNER sortie
```

Python.

```
1 def moving_average(values, k):
2     if k <= 0 or k > len(values):
3         raise ValueError("invalid width")
4     total = sum(values[:k])
5     result = [total / k]
6     for right in range(k, len(values)):
7         total += values[right] - values[right - k]
```

```
8     result.append(total / k)
9     return result
```

Tests. $k = 1$, $k = n$, valeurs négatives et largeurs invalides.

Exercice 3.4 — Requêtes d'appartenance

Explication du problème. Choisir une structure pour q questions sur n identifiants.

Intuition. Le prétraitement devient rentable lorsqu'il évite une recherche linéaire répétée.

Solution directe ou force brute. `query in identifiants` pour chaque requête coûte $O(qn)$ temps et $O(1)$ espace auxiliaire.

Solution optimisée ou structurée. Trier une copie puis utiliser la recherche binaire coûte $O(n \log n + q \log n)$ et $O(n)$ espace pour la copie. Construire un ensemble coûte $O(n + q)$ en moyenne et $O(n)$ espace, mais ne préserve pas un ordre exploitable.

Pseudo-code.

```
1 index <- ensemble construit avec les identifiants
2 POUR requete: ajouter (requete dans index) a la sortie
3 RETOURNER sortie
```

Python.

```
1 def membership_answers(identifiants, queries):
2     index = set(identifiants)
3     return [query in index for query in queries]
```

Décision. Préférer l'ensemble pour l'appartenance seule et assez de mémoire. Préférer l'ordre trié si des requêtes de rang, d'intervalle ou une restitution ordonnée sont également nécessaires.

Corrigés du chapitre 4

Exercice 4.1 — Compte bancaire minimal

Explication du problème. Encapsuler un solde non négatif et les opérations qui le modifient.

Intuition. L'objet protège l'invariant `balance >= 0`.

Solution directe ou force brute. Un dictionnaire et des modifications dispersées permettent n'importe quel solde et dupliquent les validations.

Solution optimisée ou structurée. La classe centralise validation, dépôt et retrait.

Pseudo-code.

```
1 CONSTRUIRE avec titulaire et solde valide
2 DEPOT: refuser montant <= 0, puis ajouter
3 RETRAIT: refuser montant <= 0 ou > solde, puis soustraire
```

Python.

```
1 class Account:
2     def __init__(self, owner, balance=0.0):
3         if balance < 0:
4             raise ValueError("negative opening balance")
5         self.owner = owner
6         self.balance = float(balance)
7
8     def deposit(self, amount):
9         if amount <= 0:
10            raise ValueError("amount must be positive")
11        self.balance += amount
12
13    def withdraw(self, amount):
14        if amount <= 0 or amount > self.balance:
15            raise ValueError("invalid withdrawal")
16        self.balance -= amount
```

Tests. Vérifier deux instances indépendantes, dépôt, retrait, découvert et montants nuls. Chaque opération est $O(1)$ temps et espace.

Exercice 4.2 — Objet valeur pour une colonne

Explication du problème. Représenter un schéma lisible et comparable.

Intuition. Les champs ont des noms et ne doivent pas changer après création.

Solution directe ou force brute. Un tuple est compact mais `spec[1]` ne documente pas le rôle de la position.

Solution optimisée ou structurée. Une dataclass immuable génère constructeur, représentation et égalité.

Pseudo-code. Déclarer les trois champs, rendre l'objet immuable, valider un nom non vide après initialisation.

Python.

```
1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class ColumnSpec:
5     name: str
6     expected_type: type
7     allow_missing: bool = False
8
9     def __post_init__(self):
10         if not self.name.strip():
11             raise ValueError("column name must not be empty")
```

Tests. Comparer deux spécifications égales, refuser un nom vide et vérifier qu'une affectation de champ échoue.

Exercice 4.3 — Validateurs composés

Explication du problème. Appliquer plusieurs règles et conserver toutes les erreurs.

Intuition. Chaque validateur possède la même petite interface.

Solution directe ou force brute. Une fonction géante mélange règles et s'arrête à la première.

Solution optimisée ou structurée. La composition parcourt les règles, agrège les messages et permet de tester chaque règle seule.

Pseudo-code.

```
1 erreurs <- liste vide
2 POUR validateur: etendre erreurs avec validateur.validate(ligne)
3 RETOURNER erreurs
```

Python.

```
1 class RequiredField:
2     def __init__(self, field):
3         self.field = field
4     def validate(self, row):
5         if self.field not in row or row[self.field] in (None, ""):
6             return [f"missing {self.field}"]
7         return []
```

```
8
9 class CompositeValidator:
10     def __init__(self, validators):
11         self.validators = list(validators)
12     def validate(self, row):
13         errors = []
14         for validator in self.validators:
15             errors.extend(validator.validate(row))
16         return errors
```

Analyse. Pour v validateurs à coût constant, temps $O(v)$ et mémoire $O(e)$ pour les erreurs produites.

Exercice 4.4 — Chargeur testable

Explication du problème. Séparer l'effet de lecture des transformations pures.

Intuition. Un test de normalisation ne devrait pas créer un fichier.

Solution directe ou force brute. Une méthode unique ouvre, nettoie, valide et imprime.

Solution optimisée ou structurée. Le chargeur lit ; les fonctions transforment ; l'orchestrateur compose.

Pseudo-code.

```
1 lignes <- chargeur.load(chemin)
2 lignes <- normaliser(lignes)
3 POUR ligne: collecter validateur.validate(ligne)
4 RETOURNER lignes et erreurs
```

Python.

```
1 def normalize_row(row):
2     return {key.strip().lower(): value.strip()
3             for key, value in row.items()}
4
5 def process(path, loader, validator):
6     rows = [normalize_row(row) for row in loader.load(path)]
7     issues = [validator.validate(row) for row in rows]
8     return rows, issues
```

Tests. Tester `normalize_row` en mémoire, utiliser un faux chargeur retournant deux lignes, puis un seul test d'intégration du vrai chargeur CSV.

Corrigés du chapitre 5

Exercice 5.1 — Score vectorisé

Explication du problème. Une ligne de X est une observation ; une colonne correspond à un coefficient de w . Le score est Xw .

Intuition. Chaque score est un produit scalaire.

Solution directe ou force brute. Deux boucles coûtent $O(np)$ pour n lignes et p colonnes, avec $O(n)$ pour la sortie.

Solution optimisée ou structurée. $X @ w$ exprime le même coût asymptotique, mais utilise des boucles compilées et un accès mémoire optimisé.

Pseudo-code. Vérifier que le nombre de colonnes de X égale la longueur de w , puis retourner le produit matrice-vecteur.

Python.

```
1 import numpy as np
2
3 def linear_score(X, weights):
4     X = np.asarray(X, dtype=float)
5     weights = np.asarray(weights, dtype=float)
6     if X.ndim != 2 or weights.ndim != 1 or X.shape[1] != weights.shape[0]:
7         raise ValueError("incompatible shapes")
8     return X @ weights
```

Tests. Vérifier forme (2,3) avec poids (3,), ligne unique et formes incompatibles. Temps $O(np)$, sortie $O(n)$.

Exercice 5.2 — Audit d'une table

Explication du problème. Produire les contrôles minimaux avant transformation.

Intuition. Le grain fournit la clé attendue ; l'audit compare lignes et clés.

Solution directe ou force brute. Inspecter visuellement `head()` ne détecte ni doublons éloignés ni valeurs manquantes par colonne.

Solution optimisée ou structurée. Calculer un dictionnaire reproductible de mesures.

Pseudo-code. Vérifier la présence des clés, compter lignes, types, absences, clés distinctes et doublons.

Python.

```

1 def audit_table(df, key):
2     keys = [key] if isinstance(key, str) else list(key)
3     missing = set(keys) - set(df.columns)
4     if missing:
5         raise ValueError(f"missing key columns: {sorted(missing)}")
6     return {
7         "shape": df.shape,
8         "dtypes": df.dtypes.astype(str).to_dict(),
9         "missing": df.isna().sum().to_dict(),
10        "unique_keys": df[keys].drop_duplicates().shape[0],
11        "duplicate_rows": int(df.duplicated(keys).sum()),
12    }

```

Exercice 5.3 — Agrégation de ventes

Explication du problème. Résumer au grain région-mois sans confondre lignes et commandes.

Intuition. Calculer d'abord le revenu par ligne, puis grouper selon les clés.

Solution directe ou force brute. Filtrer séparément la table pour chaque combinaison répète les scans et peut coûter $O(gn)$ pour g groupes.

Solution optimisée ou structurée. Un groupby construit les groupes en un passage conceptuel, puis applique les agrégations.

Pseudo-code. Créer revenu, extraire mois, grouper par région et mois, calculer somme, commandes uniques, clients uniques et moyenne par commande.

Python.

```

1 def sales_summary(df):
2     work = df.copy()
3     work["date"] = pd.to_datetime(work["date"])
4     work["month"] = work["date"].dt.to_period("M").astype(str)
5     work["revenue"] = work["quantity"] * work["unit_price"]
6     out = (work.groupby(["region", "month", as_index=False)
7             .agg(revenue=("revenue", "sum"),
8                 orders=("order_id", "nunique"),
9                 customers=("customer_id", "nunique")))
10    out["mean_order"] = out["revenue"] / out["orders"]
11    return out

```

Exercice 5.4 — Jointure auditée

Explication du problème. Enrichir sans multiplication ou perte silencieuse.

Intuition. La dimension doit avoir une ligne par client ; chaque transaction doit trouver au plus une correspondance.

Solution directe ou force brute. Rechercher chaque client par scan coûte $O(nm)$.

Solution optimisée ou structurée. Valider l'unicité, utiliser une jointure plusieurs-à-un, conserver l'indicateur et réconcilier les comptes.

Pseudo-code.

```
1 SI cle dupliquee dans clients: erreur
2 fusion <- jointure gauche avec validation many_to_one et indicateur
3 SI nombre de lignes change ou cle non appariee: erreur
4 RETOURNER fusion sans indicateur
```

Python.

```
1 def audited_join(transactions, customers):
2     if customers["customer_id"].duplicated().any():
3         raise ValueError("duplicate customer dimension keys")
4     out = transactions.merge(customers, on="customer_id", how="left",
5                             validate="many_to_one", indicator=True)
6     if len(out) != len(transactions):
7         raise ValueError("row count changed")
8     if not out["_merge"].eq("both").all():
9         missing = out.loc[out["_merge"].ne("both"), "customer_id"].unique()
10        raise ValueError(f"unmatched customers: {missing.tolist()}")
11    return out.drop(columns="_merge")
```

Exercice 5.5 — Standardisation sans boucle

Explication du problème. Transformer chaque colonne j selon $z_{ij} = (x_{ij} - \mu_j)/\sigma_j$, sans division par zéro pour une colonne constante.

Intuition. Moyennes et écarts-types ont la forme $(p,)$ ou $(1,p)$; ils se diffusent sur les n lignes. Une colonne constante doit produire zéro après centrage.

Solution directe ou force brute. Pour chaque colonne, calculer moyenne et écart type, puis parcourir ses lignes. Temps $O(np)$ et sortie $O(np)$; code interprété et plusieurs passages difficiles à composer.

Solution optimisée ou structurée. Réduire sur l'axe 0 avec `keepdims=True`, remplacer seulement les écarts-types nuls par 1 et diffuser. Même complexité asymptotique, mais noyaux compilés. Les trois tableaux $(1,p)$ sont petits; la sortie (n,p) domine.

Pseudo-code.

```
1 convertir X en matrice flottante
2 calculer moyenne par colonne en conservant l'axe
3 calculer ecart-type par colonne en conservant l'axe
4 remplacer les ecarts-types nuls par 1 dans un denominateur
5 retourner (X - moyenne) / denominateur
6 verifier moyenne 0 et ecart-type 1 pour colonnes non constantes
```

Python.

```

1 import numpy as np
2
3 def standardize_columns(X):
4     X = np.asarray(X, dtype=float)
5     if X.ndim != 2:
6         raise ValueError("X must be two-dimensional")
7     mean = X.mean(axis=0, keepdims=True)
8     std = X.std(axis=0, keepdims=True)
9     denominator = np.where(std == 0, 1.0, std)
10    Z = (X - mean) / denominator
11    return Z, mean.ravel(), std.ravel()
12
13 X = np.array([[1., 10., 5.], [2., 10., 7.], [3., 10., 9.]])
14 Z, mean, std = standardize_columns(X)
15 assert np.allclose(Z.mean(axis=0), 0)
16 assert np.allclose(Z[:, 1], 0)
17 assert np.allclose(Z[:, [0, 2]].std(axis=0), 1)

```

Exercice 5.6 — Distances par blocs

Explication du problème. Affecter chaque observation au centroïde euclidien le plus proche sans dépasser la mémoire.

Intuition. Toutes les méthodes effectuent essentiellement nmp opérations. Elles diffèrent surtout par boucles interprétées, disposition mémoire et taille des intermédiaires.

Solution directe ou force brute. Trois boucles sur observations, centroïdes et variables calculent chaque distance. Temps $O(nmp)$, espace $O(n)$ pour les labels, mais coût Python élevé.

Solution optimisée ou structurée. Le broadcasting crée (n, m, p) et peut épuiser la mémoire. Utiliser l'identité des normes pour produire (b, m) sur des blocs de b lignes. Temps $O(nmp)$; espace auxiliaire $O(mp + bm)$ hors entrées, ajustable par `block_size`.

Pseudo-code.

```

1 verifier matrices 2D et meme dimension de variables
2 precalculer norme carree de chaque centroide
3 allouer les n labels
4 pour chaque bloc d'observations:
5     calculer normes carrees des lignes du bloc
6     calculer distances carrees par identite et produit matriciel
7     prendre argmin sur l'axe des centroides
8 retourner labels

```

Python.

```

1 def nearest_centroid_blocked(X, centroids, block_size=4096):
2     X = np.asarray(X, dtype=float)
3     C = np.asarray(centroids, dtype=float)
4     if X.ndim != 2 or C.ndim != 2 or X.shape[1] != C.shape[1]:
5         raise ValueError("incompatible shapes")
6     if block_size <= 0:
7         raise ValueError("block_size must be positive")
8     c2 = np.sum(C * C, axis=1)

```

```
9 labels = np.empty(len(X), dtype=np.int64)
10 for start in range(0, len(X), block_size):
11     stop = min(start + block_size, len(X))
12     xb = X[start:stop]
13     distances2 = (np.sum(xb * xb, axis=1, keepdims=True)
14                  + c2[None, :] - 2 * xb @ C.T)
15     labels[start:stop] = np.argmin(distances2, axis=1)
16 return labels
```

Vérification. Comparer à la triple boucle sur un petit cas. De légers négatifs peuvent apparaître dans les distances carrées par arrondi, sans changer normalement `argmin`; les borner à zéro si les distances elles-mêmes sont retournées.

Corrigés du chapitre 6

Exercice 6.1 — Profil de livraison

Explication du problème. Produire une analyse qui relie qualité, distribution et conclusion.

Intuition. Commencer par le grain et les contrôles, puis choisir chaque résumé selon une question.

Solution directe ou force brute. `describe()` et un histogramme global peuvent masquer doublons, régions et périodes.

Solution optimisée ou structurée. Auditer, résumer globalement et par groupe, puis tracer distribution et relation distance-durée.

Pseudo-code. Vérifier clé et domaines ; calculer effectifs, quantiles et manques ; tracer histogramme puis nuage par région ; rédiger limites.

Python.

```
1 def delivery_profile(df):
2     if df["delivery_id"].duplicated().any():
3         raise ValueError("duplicate delivery grain")
4     valid = df.loc[df["duration_min"].ge(0)].copy()
5     summary = (valid.groupby("region", as_index=False)
6                 .agg(n=("delivery_id", "size"),
7                     median=("duration_min", "median"),
8                     mean=("duration_min", "mean"),
9                     q90=("duration_min", lambda s: s.quantile(.9)),
10                    missing_distance=("distance_km", lambda s: s.isna().mean()))
11     return valid, summary
```

Exercice 6.2 — Valeurs atypiques

Explication du problème. Signaler des observations éloignées et mesurer leur influence.

Intuition. La règle IQR produit des candidats à examiner, pas une suppression.

Solution directe ou force brute. Supprimer toutes les valeurs au-delà d'un seuil arbitraire améliore artificiellement la moyenne.

Solution optimisée ou structurée. Calculer bornes, comparer résumés, puis décider à partir du sens métier et des données sources.

Pseudo-code. Calculer Q_1, Q_3, IQR , créer un masque, produire résumés avec et sans candidats, restituer les lignes signalées.

Python.

```
1 def outlier_sensitivity(series):
2     q1, q3 = series.quantile([.25, .75])
3     iqr = q3 - q1
4     mask = series.between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
5     return {
6         "bounds": (q1 - 1.5 * iqr, q3 + 1.5 * iqr),
7         "flagged": series.loc[~mask],
8         "mean_all": series.mean(),
9         "mean_unflagged": series.loc[mask].mean(),
10        "median_all": series.median(),
11    }
```

Exercice 6.3 — Absence non aléatoire

Explication du problème. Déterminer si les distances manquent uniformément.

Intuition. L'indicatrice de manque devient une variable à analyser.

Solution directe ou force brute. Imputer la médiane globale avant d'étudier les taux efface le signal sur le processus de collecte.

Solution optimisée ou structurée. Agréger l'indicatrice par région, canal et semaine ; comparer effectifs et durées.

Pseudo-code.

```
1 pour chaque ligne: creer indicatrice distance manquante et semaine
2 grouper par region, canal et semaine
3 pour chaque groupe: compter lignes, moyenne de l'indicatrice, duree mediane
4 retourner la table triee et comparer les groupes a effectifs suffisants
```

Python.

```
1 def missingness_report(df):
2     work = df.assign(
3         week=pd.to_datetime(df["date"]).dt.to_period("W").astype(str),
4         missing_distance=df["distance_km"].isna(),
5     )
6     return (work.groupby(["region", "channel", "week"], as_index=False)
7             .agg(rows=("delivery_id", "size"),
8                 missing_rate=("missing_distance", "mean"),
9                 median_duration=("duration_min", "median")))
```

Interprétation. Une concentration ne prouve pas le mécanisme de manque, mais invalide l'hypothèse simple d'une absence uniforme.

Exercice 6.4 — Corrélation trompeuse

Explication du problème. Comparer association globale et conditionnelle.

Intuition. Un groupe peut déplacer simultanément les deux variables et créer une corrélation de composition.

Solution directe ou force brute. Un seul coefficient global masque les sous-groupes.

Solution optimisée ou structurée. Calculer corrélation globale et par groupe, tracer les nuages, puis formuler une conclusion conditionnelle.

Pseudo-code.

```
1 calculer correlation(x, y) sur toutes les lignes
2 pour chaque groupe: calculer correlation(x, y)
3 tracer x contre y avec couleur par groupe
4 comparer signe, amplitude et effectifs avant de conclure
```

Python.

```
1 def correlation_by_group(df, x, y, group):
2     global_corr = df[[x, y]].corr().iloc[0, 1]
3     grouped = (df.groupby(group)
4                 .apply(lambda part: part[[x, y]].corr().iloc[0, 1],
5                     include_groups=False))
6     return global_corr, grouped
```

Limite. Même une corrélation stable dans les groupes ne devient pas une preuve causale ; des facteurs non mesurés peuvent rester.

Exercice 6.5 — Entonnoir événementiel au bon grain

Explication du problème. Mesurer le passage ordonné `view -> start -> pay` sans compter plusieurs fois une session ni accepter une étape arrivée dans le mauvais ordre.

Intuition. Le journal est au grain événement, mais l'entonnoir est au grain session. Il faut d'abord résumer chaque session avec le premier instant de chaque étape.

Solution directe ou force brute. Boucler sur les utilisateurs, puis sessions, trier chaque liste et simuler l'automate. Cette solution est claire, mais multiplie les objets Python et peut coûter $O(n \log n)$ au total à cause des tris.

Solution optimisée ou structurée. Dédupliquer les événements, agréger le premier instant par session et étape, pivoter, puis calculer des masques ordonnés. Le tri reste utile si le contrat dépend de l'ordre, mais les agrégations et comparaisons sont vectorisées.

Pseudo-code.

```
1 valider colonnes et convertir timestamp en UTC
2 retirer doublons par event_id
3 grouper par user, session, step et prendre premier timestamp
4 pivoter: une ligne par session, une colonne par etape
5 entered = view existe
6 started = entered ET start existe ET start >= view
7 paid = started ET pay existe ET pay >= start
8 calculer comptes et taux avec denominators explicites
```

Python.

```

1 def funnel(events):
2     work = events.drop_duplicates("event_id").copy()
3     work["timestamp"] = pd.to_datetime(work["timestamp"], utc=True, errors="coerce")
4     first = (work.dropna(subset=["timestamp"])
5             .groupby(["user_id", "session_id", "step"], as_index=False)
6             .agg(first_at=("timestamp", "min")))
7     sessions = first.pivot(index=["user_id", "session_id"],
8                            columns="step", values="first_at")
9     for step in ["view", "start", "pay"]:
10         if step not in sessions:
11             sessions[step] = pd.NaT
12     entered = sessions["view"].notna()
13     started = entered & sessions["start"].ge(sessions["view"])
14     paid = started & sessions["pay"].ge(sessions["start"])
15     counts = {"view": int(entered.sum()), "start": int(started.sum()),
16             "pay": int(paid.sum())}
17     rates = {
18         "view_to_start": counts["start"] / counts["view"] if counts["view"] else float(
19             "nan"),
20         "start_to_pay": counts["pay"] / counts["start"] if counts["start"] else float(
21             "nan"),
22     }
23     return sessions, counts, rates

```

Interprétation. Un taux par utilisateur répondrait à une autre question si un utilisateur a plusieurs sessions. Le grain doit être annoncé dans le livrable.

Exercice 6.6 — Série temporelle fiable

Explication du problème. Construire des mesures quotidiennes comparables depuis des transactions irrégulières et horodatées.

Intuition. Agréger dans le fuseau métier, matérialiser le calendrier, puis décider si une journée absente vaut zéro ou manque de collecte. Une moyenne mobile calendaire n'est pas une moyenne des sept dernières lignes.

Solution directe ou force brute. Créer une boucle pour chaque date et filtrer les transactions. Temps $O(nd)$ pour d jours et risque de frontières incohérentes.

Solution optimisée ou structurée. Convertir en UTC, passer au fuseau métier, resample quotidien, réindexer, puis appliquer une fenêtre "7D" et un décalage calendaire. Temps proche de $O(n + d)$ hors tri; espace $O(d)$ pour la série.

Pseudo-code.

```

1 convertir timestamps en instants UTC; signaler echecs
2 convertir vers fuseau Europe/Paris
3 trier et sommer par jour local
4 creer calendrier complet entre premier et dernier jour complet
5 reindexer; remplir par zero seulement si absence = aucune transaction
6 calculer moyenne sur fenetre calendaire 7 jours avec couverture minimale
7 comparer a la valeur decalée de 7 jours

```

Python.

```
1 def daily_revenue(transactions, complete_through=None):
2     work = transactions.copy()
3     work["ts"] = pd.to_datetime(work["timestamp"], utc=True, errors="coerce")
4     if work["ts"].isna().any():
5         raise ValueError("unparseable timestamps")
6     work["ts"] = work["ts"].dt.tz_convert("Europe/Paris")
7     if complete_through is not None:
8         work = work.loc[work["ts"].lt(complete_through)]
9     daily = (work.set_index("ts")["amount"].sort_index()
10             .resample("D").sum(min_count=1))
11     calendar = pd.date_range(daily.index.min(), daily.index.max(),
12                             freq="D", tz="Europe/Paris")
13     daily = daily.reindex(calendar).fillna(0.0)
14     return pd.DataFrame({
15         "revenue": daily,
16         "mean_7d": daily.rolling("7D", min_periods=7).mean(),
17         "change_vs_7d": daily - daily.shift(freq="7D"),
18     })
```

Limite. Si zéro événement peut signifier panne de collecte, `fillna(0)` est interdit. Il faut joindre un indicateur de couverture avant toute interprétation.

Corrigés du chapitre 7

Exercice 7.1 — Dés équilibrés

Explication du problème. Estimer $P(D_1 + D_2 \geq 10)$ et comparer au calcul exact.

Intuition. Les sommes 10, 11 et 12 correspondent à $3 + 2 + 1 = 6$ couples sur 36.

Solution directe ou force brute. Énumérer les 36 couples donne $1/6$ avec coût constant ici, mais l'énumération croît exponentiellement avec le nombre de dés.

Solution optimisée ou structurée. Générer B paires, calculer l'indicatrice et prendre sa moyenne. Temps $O(B)$, mémoire $O(B)$ vectorisée ou $O(1)$ en flux.

Pseudo-code.

```
1 generer B valeurs pour chaque de
2 succes <- (de1 + de2 >= 10)
3 RETOURNER moyenne(succes)
```

Python.

```
1 def estimate_high_sum(repeats, seed=2026):
2     rng = np.random.default_rng(seed)
3     first = rng.integers(1, 7, size=repeats)
4     second = rng.integers(1, 7, size=repeats)
5     return np.mean(first + second >= 10)
```

Vérification. Comparer 100, 10 000 et 1 000 000 répétitions à $1/6$.

Exercice 7.2 — Stock du café

Explication du problème. Choisir un stock sous demande incertaine et coûts asymétriques.

Intuition. Chaque stock reçoit un coût dans les mêmes scénarios de demande.

Solution directe ou force brute. Commander la demande moyenne ignore les coûts et les queues.

Solution optimisée ou structurée. Simuler une fois la demande, réutiliser les scénarios pour tous les stocks et sélectionner le coût moyen minimal.

Pseudo-code. Générer B demandes ; pour chaque stock calculer invendus, ruptures et coût ; retenir le minimum.

Python.

```
1 def choose_stock(rate, candidates, repeats=100_000, seed=2026):
2     rng = np.random.default_rng(seed)
3     demand = rng.poisson(rate, repeats)
4     costs = {}
5     for stock in candidates:
6         waste = np.maximum(stock - demand, 0)
7         shortage = np.maximum(demand - stock, 0)
8         costs[stock] = np.mean(waste + 4 * shortage)
9     return min(costs, key=costs.get), costs
```

Sensibilité. Refaire pour plusieurs taux et ratios de coût. Temps $O(Bk)$ et mémoire $O(B + k)$ pour k candidats.

Exercice 7.3 — Test et taux de base

Explication du problème. Calculer $P(\text{cas} \mid \text{alerte})$.

Intuition. Le dénominateur contient vrais positifs et faux positifs.

Solution directe ou force brute. Appliquer Bayes est constant et exact sous les paramètres.

Solution optimisée ou structurée. Elle vérifie l'intuition et permet d'ajouter des mécanismes plus complexes ; elle introduit une erreur Monte-Carlo.

Pseudo-code. Générer statut réel selon prévalence, puis alerte selon statut ; parmi les alertes, calculer la proportion réelle.

Python.

```
1 def positive_predictive_value(prevalence, sensitivity=.95, fpr=.05):
2     numerator = sensitivity * prevalence
3     return numerator / (numerator + fpr * (1 - prevalence))
```

Résultat. Pour 1%, la valeur vaut environ 16%. La refaire pour 0,1%, 5% et 20% montre le rôle du taux de base.

Exercice 7.4 — Moyennes d'échantillons

Explication du problème. Observer la distribution de moyennes issues d'une loi exponentielle.

Intuition. L'écart type de la moyenne décroît approximativement comme $1/\sqrt{n}$.

Solution directe ou force brute. Une seule moyenne ne montre pas sa variabilité.

Solution optimisée ou structurée. Générer B échantillons pour chaque taille et comparer quantiles, écart type et histogramme des moyennes.

Pseudo-code.

```
1 pour chaque taille n:
2     repeter B fois: tirer n valeurs et calculer leur moyenne
3     calculer moyenne, ecart type et quantiles des B moyennes
4     comparer la forme et l'ecart type entre tailles
```

Python.

```
1 def sample_means(size, repeats=20_000, seed=2026):
2     rng = np.random.default_rng(seed)
3     samples = rng.exponential(scale=10, size=(repeats, size))
4     return samples.mean(axis=1)
5
6 for size in [5, 30, 100]:
7     means = sample_means(size)
8     print(size, means.mean(), means.std())
```

Complexité. Temps et mémoire $O(Bn)$ dans cette version matricielle. Une génération par lots réduit le pic mémoire.

Exercice 7.5 — Niveau de stock continu optimisé

Explication du problème. Minimiser un coût attendu sur un intervalle de stocks, puis convertir l'optimum continu en décision entière.

Intuition. Une recherche sur grille sert de baseline vérifiable. Un optimiseur borné économise des évaluations lorsque l'objectif est suffisamment régulier, mais son statut et son voisinage doivent être contrôlés.

Solution directe ou force brute. Évaluer tous les stocks entiers de 0 à 100. Temps $O(BK)$ pour B scénarios et $K = 101$ décisions ; résultat directement réalisable.

Solution optimisée ou structurée. Lisser la décision en réel, utiliser `minimize_scalar` sur $[0, 100]$, puis comparer plancher et plafond avec les mêmes tirages. Le nombre d'évaluations diminue, mais la simulation coûte toujours $O(B)$ par évaluation.

Pseudo-code.

```
1 generer une seule fois B demandes
2 definir cout(stock) avec maximum vectorise
3 optimiser cout sur [0, 100]
4 si convergence fausse: erreur
5 evaluer plancher et plafond de la solution continue
6 comparer a une grille grossiere et verifier le voisinage
```

Python.

```
1 from scipy import optimize
2 import numpy as np
3
4 def optimize_stock(rate, repeats=200_000, seed=2026):
5     rng = np.random.default_rng(seed)
6     demand = rng.poisson(rate, size=repeats)
7
8     def cost(stock):
```

```

9     shortage = np.maximum(demand - stock, 0)
10    waste = np.maximum(stock - demand, 0)
11    return np.mean(4 * shortage + waste)
12
13    result = optimize.minimize_scalar(cost, bounds=(0, 100), method="bounded")
14    if not result.success:
15        raise RuntimeError(result.message)
16    candidates = sorted({int(np.floor(result.x)), int(np.ceil(result.x))})
17    costs = {q: cost(q) for q in candidates}
18    return min(costs, key=costs.get), result, costs

```

Interprétation. Pour une demande discrète et une fonction à cassures, la grille entière reste souvent la solution la plus simple. L'optimisation générale devient utile avec plus de décisions ou un objectif plus coûteux.

Exercice 7.6 — Choisir et critiquer une loi

Explication du problème. Comparer deux familles paramétriques à la distribution empirique de temps positifs, surtout dans la queue qui définit le niveau de service.

Intuition. Ajuster sur une partie et valider sur une autre. La moyenne ne suffit pas : comparer médiane, quantile 95 et dépassement de 30 minutes.

Solution directe ou force brute. Utiliser l'exponentielle avec moyenne observée. Cette solution impose mémoire sans durée et variance égale au carré de la moyenne.

Solution optimisée ou structurée. Ajuster exponentielle et gamma avec localisation fixée à zéro, calculer diagnostics sur validation, et garder la distribution empirique comme baseline. Le choix minimise une perte correspondant à la décision, pas un graphique seul.

Pseudo-code.

```

1  separer chronologiquement ou aleatoirement selon le plan
2  ajuster exponentielle et gamma sur calibration avec support positif
3  pour chaque modele et pour l'empirique:
4      calculer mediane, q95, P(X > 30) sur validation
5      calculer ecart aux quantites observees
6  inspecter ECDF et residus par segment
7  retenir, rejeter ou conclure qu'aucune famille n'est adequate

```

Python.

```

1  from scipy import stats
2
3  def compare_wait_models(train, test, threshold=30):
4      train = np.asarray(train, dtype=float)
5      test = np.asarray(test, dtype=float)
6      if np.any(train < 0) or np.any(test < 0):
7          raise ValueError("waiting times must be non-negative")
8      _, exp_scale = stats.expon.fit(train, floc=0)
9      shape, _, gamma_scale = stats.gamma.fit(train, floc=0)
10     models = {
11         "exponential": stats.expon(scale=exp_scale),
12         "gamma": stats.gamma(a=shape, scale=gamma_scale),
13     }

```

```
14 observed = {"median": np.median(test), "q95": np.quantile(test, .95),
15             "tail": np.mean(test > threshold)}
16 rows = []
17 for name, model in models.items():
18     predicted = {"median": model.ppf(.5), "q95": model.ppf(.95),
19                "tail": model.sf(threshold)}
20     rows.append({"model": name, **predicted,
21                 "q95_abs_error": abs(predicted["q95"] - observed["q95"]),
22                 "tail_abs_error": abs(predicted["tail"] - observed["tail"])})
23 return observed, pd.DataFrame(rows)
```

Limite. L'observation temporelle ou censurée exige une séparation et un modèle adaptés. Un bon ajustement marginal ne valide pas l'indépendance entre attentes successives.

Corrigés du chapitre 8

Exercice 8.1 — Intervalle d’une proportion

Explication du problème. Quantifier l’incertitude d’un taux observé.

Intuition. À taux similaire, l’erreur standard décroît comme $1/\sqrt{n}$.

Solution directe ou force brute. Utiliser l’approximation normale lorsque $n\hat{p}$ et $n(1 - \hat{p})$ ne sont pas trop petits.

Solution optimisée ou structurée. Pour des événements rares ou petits échantillons, utiliser un intervalle binomial adapté tel que Wilson via statsmodels.

Pseudo-code. Calculer $\hat{p}$, son erreur standard et les deux bornes ; répéter pour chaque taille.

Python.

```
1 from math import sqrt
2
3 def normal_proportion_interval(successes, n, level_z=1.96):
4     p = successes / n
5     se = sqrt(p * (1 - p) / n)
6     return p, (p - level_z * se, p + level_z * se)
```

Interprétation. Multiplier n par 100 divise environ la largeur par 10, si le taux reste comparable.

Exercice 8.2 — Bootstrap d’un gain

Explication du problème. Estimer l’incertitude sur la différence de panier moyen.

Intuition. Le client est l’unité indépendante ; ses lignes de produits doivent rester ensemble.

Solution directe ou force brute. Rééchantillonner les lignes donne trop de poids aux clients ayant plusieurs produits et sous-estime l’incertitude.

Solution optimisée ou structurée. Agréger d’abord le panier par client et groupe, puis rééchantillonner séparément les clients de chaque groupe.

Pseudo-code. Construire panier client ; pour chaque répétition tirer clients avec remise dans chaque bras ; conserver différence de moyennes ; prendre quantiles.

Python.

```

1 def bootstrap_basket_difference(df, repeats=10_000, seed=2026):
2     baskets = (df.groupby(["variant", "user_id"], as_index=False)
3                 .agg(basket=("amount", "sum")))
4     control = baskets.loc[baskets["variant"].eq("control"), "basket"].to_numpy()
5     treat = baskets.loc[baskets["variant"].eq("treatment"), "basket"].to_numpy()
6     rng = np.random.default_rng(seed)
7     estimates = np.empty(repeats)
8     for b in range(repeats):
9         estimates[b] = (rng.choice(treat, len(treat)).mean()
10                        - rng.choice(control, len(control)).mean())
11     return np.quantile(estimates, [.025, .5, .975])

```

Exercice 8.3 — Audit A/B

Explication du problème. Vérifier l'expérience avant d'estimer l'effet.

Intuition. Une unité doit recevoir un groupe unique et les volumes doivent suivre le ratio prévu.

Solution directe ou force brute. Calculer immédiatement une valeur p ignore les problèmes de tracking.

Solution optimisée ou structurée. Contrôler groupes par utilisateur, doublons événementiels, ratio, exposition et valeurs manquantes.

Pseudo-code.

```

1 compter le nombre de variantes par utilisateur; signaler les valeurs > 1
2 compter les identifiants d'événement dupliqués
3 réduire à une ligne par utilisateur et mesurer le ratio d'affectation
4 retourner les anomalies avant toute estimation d'effet

```

Python.

```

1 def audit_experiment(df, expected_treatment=.5):
2     groups_per_user = df.groupby("user_id")["variant"].nunique()
3     crossed = groups_per_user[groups_per_user > 1].index.tolist()
4     duplicate_events = int(df.duplicated("event_id").sum())
5     users = df.drop_duplicates("user_id")
6     observed = users["variant"].eq("treatment").mean()
7     return {"crossed_users": crossed,
8           "duplicate_events": duplicate_events,
9           "observed_treatment_share": observed,
10          "share_gap": observed - expected_treatment}

```

Suite. Un écart de ratio doit être évalué avec un test de proportion et une investigation du pipeline d'assignation, pas avec un seuil arbitraire seul.

Exercice 8.4 — Note de décision

Explication du problème. Transformer résultats statistiques en recommandation.

Intuition. Comparer tout l'intervalle à zéro et au seuil d'effet utile, puis intégrer les métriques de garde.

Solution directe ou force brute. « $p > 0.05$, aucun effet » confond absence de preuve et preuve d'absence.

Solution optimisée ou structurée. Écrire estimation, intervalle, intégrité, garde-fous, valeur pratique et prochaine action.

Pseudo-code.

```
1 annoncer effet et intervalle
2 comparer a zero et a effet minimal utile
3 resumer integrite et gardes
4 choisir deployer, arreter ou collecter davantage avec justification
```

Python.

```
1 def decide(effect, low, high, minimum_useful, guards_ok):
2     if not guards_ok:
3         return "investigate experiment integrity"
4     if low >= minimum_useful:
5         return "deploy"
6     if high < minimum_useful:
7         return "stop: useful effect is implausible"
8     return "collect more data if information is worth its cost"
```

Interprétation. Un intervalle $[-0.1, 0.9]$ point autour de $+0.4$ ne démontre pas un gain, mais reste compatible avec un gain utile. Une collecte supplémentaire peut être rationnelle si les gardes sont stables et si l'information vaut son coût.

Exercice 8.5 — A/B bayésien complet

Explication du problème. Estimer les taux des deux variantes et les probabilités que le traitement soit positif puis supérieur au seuil utile, tout en vérifiant calcul et adéquation prédictive.

Intuition. Deux distributions bêta génèrent des taux ; les comptages suivent des binomiales. La différence est calculée tirage par tirage. Le prior doit produire des taux plausibles avant d'observer les conversions.

Solution directe ou force brute. Utiliser deux taux observés et déclarer le plus grand gagnant. Cette solution ne représente ni incertitude, ni seuil utile, ni erreur de modèle.

Solution optimisée ou structurée. Construire le modèle avec axes nommés, vérifier prior prédictif, échantillonner quatre chaînes, examiner $R\text{-hat}$ /ESS/divergences/traces, puis contrôler le posterior prédictif. Enfin calculer intervalle et probabilités de décision.

Pseudo-code.

```

1 définir coordonnee variant et comptages au grain utilisateur
2 construire rate ~ Beta et conversions ~ Binomial
3 simuler prior predictif; verifier plage et quantiles
4 echantillonner plusieurs chaines
5 controler R-hat, ESS, divergences et traces
6 simuler posterior predictif et comparer les comptages
7 delta <- rate_B - rate_A pour chaque tirage
8 retourner moyenne, HDI, P(delta > 0), P(delta > seuil utile)

```

Python.

```

1 import numpy as np
2 import pymc as pm
3 import arviz as az
4
5 def fit_bayesian_ab(successes, trials, minimum_useful=.003, seed=2026):
6     successes = np.asarray(successes, dtype=int)
7     trials = np.asarray(trials, dtype=int)
8     if successes.shape != (2,) or trials.shape != (2,):
9         raise ValueError("expected control and treatment")
10    if np.any(successes < 0) or np.any(successes > trials):
11        raise ValueError("invalid binomial counts")
12
13    with pm.Model(coords={"variant": ["control", "treatment"]}) as model:
14        rate = pm.Beta("rate", alpha=2, beta=18, dims="variant")
15        pm.Binomial("conversions", n=trials, p=rate,
16                    observed=successes, dims="variant")
17        prior = pm.sample_prior_predictive(random_seed=seed)
18        idata = pm.sample(draws=2_000, tune=2_000, chains=4,
19                          target_accept=.9, random_seed=seed)
20        ppc = pm.sample_posterior_predictive(idata, random_seed=seed)
21
22    diagnostic = az.summary(idata, var_names=["rate"], hdi_prob=.95)
23    divergences = int(idata.sample_stats["diverging"].sum())
24    post = idata.posterior["rate"]
25    delta = post.sel(variant="treatment") - post.sel(variant="control")
26    result = {
27        "mean_lift": float(delta.mean()),
28        "prob_positive": float((delta > 0).mean()),
29        "prob_useful": float((delta > minimum_useful).mean()),
30        "hdi": az.hdi(delta, hdi_prob=.95),
31        "divergences": divergences,
32    }
33    return model, prior, idata, ppc, diagnostic, result

```

Interprétation. Si les diagnostics échouent, les probabilités ne doivent pas être utilisées. Si elles sont fiables, une décision compare les pertes des actions; 95% n'est pas un seuil universel imposé par Bayes.

Exercice 8.6 — Effets de régions et pooling partiel

Explication du problème. Estimer un taux par région lorsque les volumes sont très inégaux et éviter de promouvoir des régions gagnantes uniquement par bruit.

Intuition. Les régions partagent une distribution d'effets. Le modèle apprend une moyenne et une dispersion communes ; une région peu observée est davantage régularisée.

Solution directe ou force brute. Calculer `successes/trials` séparément. Une région 1/2 paraît à 50% et une région 400/1000 à 40%, alors que la première est bien plus incertaine.

Solution optimisée ou structurée. Utiliser un modèle hiérarchique logistique non centré. Comparer visuellement estimations brutes, pooling complet et postérieures régionales. Diagnostiquer, puis rapporter toutes les régions avec intervalle et probabilité d'un seuil.

Pseudo-code.

```
1 définir moyenne globale de log-odds et dispersion positive
2 tirer offset standard pour chaque region
3 region_logit <- global + dispersion * offset
4 region_rate <- logistic(region_logit)
5 observer succes ~ Binomial(essais, region_rate)
6 échantillonner et diagnostiquer
7 pour chaque region: resumer taux, intervalle et écart au global
8 comparer a taux brut et ne pas filtrer seulement les gagnants
```

Python.

```
1 def fit_hierarchical_rates(successes, trials, region_names, seed=2026):
2     successes = np.asarray(successes, dtype=int)
3     trials = np.asarray(trials, dtype=int)
4     coords = {"region": list(region_names)}
5     with pm.Model(coords=coords) as model:
6         global_logit = pm.Normal("global_logit", mu=0, sigma=1.5)
7         region_sd = pm.HalfNormal("region_sd", sigma=1)
8         offset = pm.Normal("offset", 0, 1, dims="region")
9         logit = pm.Deterministic(
10             "region_logit", global_logit + region_sd * offset,
11             dims="region")
12         rate = pm.Deterministic(
13             "region_rate", pm.math.sigmoid(logit), dims="region")
14         pm.Binomial("successes", n=trials, p=rate,
15                     observed=successes, dims="region")
16         idata = pm.sample(2_000, tune=2_000, chains=4,
17                           target_accept=.92, random_seed=seed)
18     return model, idata, az.summary(
19         idata, var_names=["region_rate", "region_sd"], hdi_prob=.95)
```

Vérification. Comparer régions de mêmes taux bruts et tailles différentes ; le petit groupe doit présenter davantage d'incertitude et généralement plus de shrinkage. Vérifier l'hypothèse d'échangeabilité et effectuer une sensibilité sur le prior de dispersion.

Corrigés du chapitre 9

Exercice 9.1 — Détection de fuite

Explication du problème. Décider si une variable existe à l'instant réel de prédiction.

Intuition. Dessiner une ligne du temps : observation, prédiction, action, puis fenêtre de cible.

Solution directe ou force brute. Utiliser toutes les colonnes disponibles dans l'entrepôt confond date d'extraction et disponibilité opérationnelle.

Solution optimisée ou structurée. Pour chaque variable, documenter timestamp source, retard de publication et dépendance éventuelle à la cible. Exclure toute information postérieure.

Pseudo-code.

```
1 POUR variable:
2     trouver heure de disponibilite
3     SI heure > instant de prediction ou variable derivee de cible: exclure
4     SINON conserver et documenter
```

Python.

```
1 def leakage_candidates(feature_catalog, prediction_time):
2     return feature_catalog.loc[
3         feature_catalog["available_at"].gt(prediction_time)
4         | feature_catalog["depends_on_target"],
5         "feature",
6     ].tolist()
```

Exercice 9.2 — Baselines

Explication du problème. Comparer majorité, règle métier et modèle.

Intuition. Une amélioration ne vaut que par rapport à une décision déjà possible et sur des métriques adaptées.

Solution directe ou force brute. L'accuracy favorise la classe majoritaire.

Solution optimisée ou structurée. Évaluer chaque score avec ROC-AUC, average precision et métriques au seuil commun ou à capacité commune.

Pseudo-code.

```
1 pour chaque baseline ou modele:
2     produire un score sur le meme jeu de test
3     calculer ROC-AUC et average precision
4     appliquer le meme seuil ou la meme capacite
5     comparer precision, rappel et valeur operationnelle
```

Python.

```
1 from sklearn.metrics import average_precision_score, roc_auc_score
2
3 def compare_scores(y_true, named_scores):
4     rows = []
5     for name, score in named_scores.items():
6         rows.append({"model": name,
7                     "roc_auc": roc_auc_score(y_true, score),
8                     "average_precision": average_precision_score(y_true, score)})
9     return pd.DataFrame(rows)
```

Exercice 9.3 — Pipeline reproductible

Explication du problème. Apprendre les transformations uniquement sur l'entraînement.

Intuition. Toute étape possédant un fit apprend un état et doit rester dans la validation.

Solution directe ou force brute. Imputer et encoder avant la séparation transmet des statistiques du test.

Solution optimisée ou structurée. Séparer, construire ColumnTransformer, l'inclure dans Pipeline, puis lancer validation croisée sur l'entraînement.

Pseudo-code. Split ; définir colonnes ; construire prétraitement et modèle ; fit sur train ; évaluer une fois sur test.

Python.

```
1 pipeline = Pipeline([
2     ("preprocess", ColumnTransformer([
3         ("num", Pipeline([
4             ("impute", SimpleImputer(strategy="median")),
5             ("scale", StandardScaler()),
6         ]), numeric),
7     ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
8 ])),
9     ("model", LogisticRegression(max_iter=1000)),
10 ])
11 pipeline.fit(X_train, y_train)
12 test_score = pipeline.predict_proba(X_test)[: , 1]
```

Exercice 9.4 — Seuil sous capacité

Explication du problème. Sélectionner au plus 500 dossiers et mesurer la valeur de cette liste d'action.

Intuition. Trier les scores suffit ; le seuil est le score à la frontière de capacité.

Solution directe ou force brute. Tester de nombreux seuils et recompter toutes les lignes répète le travail en $O(kn)$.

Solution optimisée ou structurée. Trier une fois en $O(n \log n)$, puis prendre les 500 premiers. Une sélection partielle peut réduire le coût pour très grands n .

Pseudo-code.

```
1 ordonner indices par score décroissant
2 selection <- 500 premiers
3 precision <- positifs selectionnes / 500
4 rappel <- positifs selectionnes / positifs totaux
5 calculer valeur attendue selon couts
```

Python.

```
1 def evaluate_capacity(y_true, scores, capacity, benefit=100, contact_cost=5):
2     order = np.argsort(scores)[::-1]
3     selected = order[:capacity]
4     positives = np.asarray(y_true)[selected].sum()
5     precision = positives / len(selected)
6     recall = positives / np.asarray(y_true).sum()
7     value = positives * benefit - len(selected) * contact_cost
8     threshold = scores[selected[-1]]
9     return {"threshold": threshold, "precision": precision,
10           "recall": recall, "expected_value": value}
```

Tests. Vérifier capacité supérieure au nombre de lignes, ex aequo au seuil, absence de positifs et cohérence des indices pandas/NumPy.

Exercice 9.5 — Sélection de modèle sans fuite

Explication du problème. Comparer deux familles et régler leurs paramètres sans utiliser le test final ni mélanger des lignes d'un même groupe.

Intuition. La boucle externe estime la procédure complète. Dans chaque fold externe, la boucle interne choisit un paramètre en n'utilisant que le sous-ensemble d'entraînement. Le prétraitement appartient à chaque pipeline.

Solution directe ou force brute. Prétraiter toutes les données, tester plusieurs modèles sur le test et retenir le meilleur. Cette approche fait fuiter statistiques de prétraitement et décisions de sélection ; son score final est optimiste.

Solution optimisée ou structurée. Utiliser des splits groupés imbriqués. Conserver score externe, meilleur paramètre, durée et écart train-validation. Après comparaison, réentraîner une configuration choisie sur tout l'entraînement et ouvrir le test une fois.

Pseudo-code.

```

1 définir test final avant toute recherche
2 pour chaque famille de modele:
3     construire pipeline complet
4     pour chaque split externe groupe:
5         lancer recherche interne groupe sur la partie train externe
6         evaluer meilleur pipeline sur validation externe
7         enregistrer score, parametres et duree
8 choisir une famille selon moyenne, dispersion, cout et simplicité
9 refaire recherche sur tout train; geler pipeline et seuil
10 evaluer une fois sur test final

```

Python.

```

1 from time import perf_counter
2 import pandas as pd
3 from sklearn.base import clone
4 from sklearn.metrics import average_precision_score
5 from sklearn.model_selection import GridSearchCV, StratifiedGroupKFold
6
7 def nested_group_evaluation(pipeline, param_grid, X, y, groups, seed=2026):
8     outer = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=seed)
9     rows = []
10    for fold, (train_idx, valid_idx) in enumerate(outer.split(X, y, groups), start=1):
11        inner = StratifiedGroupKFold(n_splits=4, shuffle=True,
12                                     random_state=seed + fold)
13        search = GridSearchCV(clone(pipeline), param_grid, scoring="average_precision",
14                               cv=inner, n_jobs=-1, return_train_score=True)
15        start = perf_counter()
16        search.fit(X.iloc[train_idx], y.iloc[train_idx],
17                  groups=groups.iloc[train_idx])
18        score = search.best_estimator_.predict_proba(X.iloc[valid_idx])[:, 1]
19        rows.append({
20            "fold": fold,
21            "average_precision": average_precision_score(y.iloc[valid_idx], score),
22            "best_params": search.best_params_,
23            "fit_seconds": perf_counter() - start,
24            "n_train": len(train_idx), "n_valid": len(valid_idx),
25        })
26    return pd.DataFrame(rows)

```

Interprétation. Comparer les mêmes splits externes pour les deux familles rend les différences appariées. Une moyenne légèrement supérieure avec forte dispersion et coût bien plus élevé peut ne pas justifier le modèle complexe.

Exercice 9.6 — Dossier de mise en production

Explication du problème. Faire de l'artefact entraîné un composant dont l'entrée, la version, la décision et les contrôles sont explicites.

Intuition. Un score reproductible exige plus qu'un fichier sérialisé : le schéma, les dépendances et le seuil appartiennent au même contrat. Un lot de référence détecte un changement d'environnement.

Solution directe ou force brute. Sériialiser le classifieur seul et reconstruire manuellement le prétraitement en production. L'ordre, les catégories ou l'imputation peuvent diverger silencieusement.

Solution optimisée ou structurée. Sériialiser le pipeline complet depuis une source de confiance, enregistrer manifest et empreinte, valider le schéma avant prédiction et comparer les scores sur un lot de référence après chargement. Définir monitoring et retrait avant lancement.

Pseudo-code.

```
1 geler pipeline complet et seuil choisi hors test
2 enregistrer versions, colonnes, dtypes, donnees et metriques
3 conserver lot de reference et scores attendus avec tolerance
4 au chargement: verifier origine, versions et schema
5 predire lot de reference; comparer les scores
6 en production: controler entree, score, operation et labels retardes
7 si garde critique echoue: retirer ou basculer vers baseline
```

Python.

```
1 from dataclasses import dataclass
2 import numpy as np
3
4 @dataclass(frozen=True)
5 class ModelContract:
6     columns: tuple[str, ...]
7     dtypes: dict[str, str]
8     threshold: float
9     model_version: str
10
11 def validate_batch(df, contract):
12     if tuple(df.columns) != contract.columns:
13         raise ValueError("column order or names changed")
14     actual = df.dtypes.astype(str).to_dict()
15     wrong = {c: (actual[c], contract.dtypes[c]) for c in contract.columns
16              if actual[c] != contract.dtypes[c]}
17     if wrong:
18         raise ValueError(f"dtype mismatch: {wrong}")
19
20 def compatibility_check(pipeline, reference_X, expected_scores, contract,
21                        atol=1e-10):
22     validate_batch(reference_X, contract)
23     actual = pipeline.predict_proba(reference_X)[: , 1]
24     if not np.allclose(actual, expected_scores, rtol=0, atol=atol):
25         raise RuntimeError("reference predictions changed")
26     return True
```

Monitoring. Documenter manque, catégories inconnues, plages, distribution du score, latence, taux d'action, métriques retardées, propriétaire de l'alerte et retour à une baseline. Ne charger aucun artefact sérialisé d'une source non fiable.

Corrigés du chapitre 10

Exercice 10.1 — Sélection argumentée d'un modèle

Explication du problème. Choisir un modèle compatible avec l'usage réel et identifier les informations qui manquent avant un pilote.

Intuition. Un classement par benchmark seul ignore licence, langue, mémoire, contexte, confidentialité et charge opérationnelle. La décision est multicritère et certaines contraintes sont éliminatoires.

Solution directe ou force brute. Télécharger successivement les trois modèles, lire quelques sorties puis conserver celui qui semble le meilleur. Cette approche est lente, coûteuse, non reproductible et peut tester un modèle juridiquement inutilisable.

Solution optimisée ou structurée. Appliquer d'abord les contraintes dures, puis noter les candidats restants avec une grille pondérée. Marquer explicitement les valeurs inconnues et prévoir une évaluation sur le corpus français interne avant validation.

Pseudo-code.

```
1 définir contraintes éliminatoires: licence, environnement, langue, memoire
2 pour chaque fiche:
3     extraire faits et informations absentes
4     si une contrainte dure echoue: rejeter avec raison
5     sinon calculer un score pondere et une penalite d'incertitude
6 classer les candidats restants
7 proposer un pilote sur corpus gele; ne pas conclure depuis la fiche seule
```

Python.

```
1 WEIGHTS = {"french": 4, "license": 4, "memory": 3,
2            "context": 2, "evaluation": 3, "operations": 2}
3
4 def score_model(card, requirements):
5     blockers = []
6     if card.get("license") not in requirements["allowed_licenses"]:
7         blockers.append("license")
8     if card.get("supports_french") is not True:
9         blockers.append("French support not established")
10    if card.get("memory_gb", float("inf")) > requirements["memory_gb"]:
11        blockers.append("memory")
12    unknown = [key for key in WEIGHTS if card.get(key) is None]
13    score = sum(WIGHTS[k] * bool(card.get(k)) for k in WEIGHTS)
14    score -= 2 * len(unknown)
15    return {"eligible": not blockers, "score": score,
16           "blockers": blockers, "unknown": unknown}
```

Interprétation. La grille ne transforme pas une préférence en vérité. Elle rend les critères discutables et prépare l'expérience locale qui décidera de la qualité.

Exercice 10.2 — Sortie structurée robuste

Explication du problème. Convertir une chaîne produite par un modèle en objet métier sans accepter champs inattendus, catégories inventées ou valeurs vides.

Intuition. Le modèle est une dépendance non fiable. Le parseur constitue une frontière : aucun résultat n'entre dans le reste du système avant validation.

Solution directe ou force brute. Appeler `json.loads`, puis accéder aux trois clés. Une sortie partiellement valide déclenche une exception tardive ou propage une valeur interdite.

Solution optimisée ou structurée. Distinguer syntaxe JSON, forme de l'objet, ensemble exact des clés, type des valeurs et contraintes métier. Retourner une erreur typée ; le service décide d'une réparation unique ou d'une escalade.

Pseudo-code.

```
1 essayer de decoder le JSON; sinon erreur de syntaxe
2 verifier que la racine est un objet
3 verifier exactement les trois cles attendues
4 verifier que chaque valeur est une chaine non vide
5 verifier que la categorie appartient a la liste blanche
6 construire un objet immuable
7 au plus une reparation; sinon escalade humaine
```

Python.

```
1 from dataclasses import dataclass
2 import json
3
4 ALLOWED = {"paiement", "contrat", "sinistre", "autre"}
5
6 @dataclass(frozen=True)
7 class IncidentResult:
8     category: str
9     summary: str
10    next_action: str
11
12 def parse_result(raw):
13     try:
14         obj = json.loads(raw)
15     except json.JSONDecodeError as exc:
16         raise ValueError("invalid JSON") from exc
17     expected = {"category", "summary", "next_action"}
18     if not isinstance(obj, dict) or set(obj) != expected:
19         raise ValueError("invalid schema")
20     if any(not isinstance(obj[k], str) or not obj[k].strip()
21           for k in expected):
22         raise ValueError("fields must be non-empty strings")
23     if obj["category"] not in ALLOWED:
24         raise ValueError("category outside allow-list")
25     return IncidentResult(**obj)
26
```

```

27 BAD_CASES = [
28     "not json", "[]", "{}",
29     '{"category":"urgent","summary":"x","next_action":"y"}',
30     '{"category":"autre","summary":"","next_action":"y"}',
31     '{"category":"autre","summary":"x","next_action":"y","secret":1}',
32 ]
33 for raw in BAD_CASES:
34     try:
35         parse_result(raw)
36         raise AssertionError("case should fail")
37     except ValueError:
38         pass

```

Exercice 10.3 — Recherche vectorielle exacte

Explication du problème. Trouver les k documents dont l’embedding est le plus proche d’une requête selon le cosinus.

Intuition. Après normalisation, la similarité cosinus est un produit scalaire. Tous les scores peuvent être calculés par une multiplication matrice-vecteur.

Solution directe ou force brute. Parcourir les n documents, calculer norme et produit scalaire, stocker les couples puis trier tout le tableau. Temps $O(nd + n \log n)$, espace $O(n)$.

Solution optimisée ou structurée. Normaliser en bloc, calculer docs @ q et utiliser une sélection partielle avant de trier seulement k scores. Temps $O(nd + n + k \log k)$ en moyenne pour la sélection ; espace $O(nd + n)$ si la matrice normalisée est copiée. Pour un index déjà normalisé, le coût par requête évite cette copie.

Pseudo-code.

```

1 verifier les dimensions et borner k
2 normaliser la requete avec un plancher numerique
3 normaliser chaque ligne de la matrice
4 calculer tous les produits scalaires
5 selectionner sans tri complet les indices des k meilleurs
6 trier uniquement ces k indices et retourner indice, score

```

Python.

```

1 import numpy as np
2
3 def top_k_cosine(query, documents, k=3):
4     query = np.asarray(query, dtype=float)
5     documents = np.asarray(documents, dtype=float)
6     if documents.ndim != 2 or query.shape != (documents.shape[1],):
7         raise ValueError("incompatible embedding shapes")
8     if len(documents) == 0 or k <= 0:
9         return []
10    k = min(k, len(documents))
11    q = query / max(np.linalg.norm(query), 1e-12)
12    norms = np.linalg.norm(documents, axis=1, keepdims=True)
13    docs = documents / np.clip(norms, 1e-12, None)
14    scores = docs @ q
15    candidates = np.argpartition(scores, -k)[-k:]

```

```
16 ordered = candidates[np.argsort(scores[candidates])[:-1]]
17 return [(int(i), float(scores[i])) for i in ordered]
```

Vérification. Tester $k > n$, $k = 0$, requête nulle, ligne nulle, ex aequo et comparaison avec un tri complet sur de petites matrices.

Exercice 10.4 — Banc d'évaluation

Explication du problème. Comparer des versions de manière reproductible sans coupler le banc à une API réelle.

Intuition. Chaque exemple produit une ligne de résultat. Les agrégats viennent ensuite; conserver le détail permet segments, erreurs et tests appariés.

Solution directe ou force brute. Imprimer quelques réponses et les lire. Cette démonstration ne mesure ni échec, ni latence, ni variabilité entre catégories.

Solution optimisée ou structurée. Définir un protocole `analyze`, chronométrer chaque appel, capturer l'erreur, valider la sortie et retourner une DataFrame au grain « un exemple–une version ». Agréger globalement et par segment.

Pseudo-code.

```
1 pour chaque exemple du corpus gele:
2     démarrer chronometre
3     essayer appel et validation
4     enregistrer latence, prediction, validite, exactitude, erreur
5 construire une table detaillee
6 calculer taux de validite, exactitude, erreurs, mediane et p95
7 repeter les agregats par segment avec effectif
```

Python.

```
1 from time import perf_counter
2 import numpy as np
3 import pandas as pd
4
5 def evaluate_service(service, cases):
6     rows = []
7     for case in cases:
8         start = perf_counter()
9         try:
10             result = service.analyze(case["text"])
11             error = None
12             valid = True
13             predicted = result.category
14         except Exception as exc:
15             error = type(exc).__name__
16             valid = False
17             predicted = None
18         rows.append({
19             "id": case["id"], "segment": case["segment"],
20             "expected": case["label"], "predicted": predicted,
21             "valid": valid, "correct": predicted == case["label"],
22             "error": error,
```

```

23         "latency_ms": 1000 * (perf_counter() - start),
24     })
25     detail = pd.DataFrame(rows)
26     summary = {
27         "n": len(detail),
28         "valid_rate": detail["valid"].mean(),
29         "accuracy": detail["correct"].mean(),
30         "error_rate": detail["error"].notna().mean(),
31         "latency_p50_ms": detail["latency_ms"].median(),
32         "latency_p95_ms": detail["latency_ms"].quantile(.95),
33     }
34     by_segment = (detail.groupby("segment", as_index=False)
35                     .agg(n=("id", "size"),
36                         valid_rate=("valid", "mean"),
37                         accuracy=("correct", "mean"),
38                         latency_p95_ms=("latency_ms", lambda s: s.quantile(.95)))
39     )
40     return detail, summary, by_segment

```

Interprétation. Le faux modèle rend les tests rapides et déterministes. Une suite d'intégration séparée vérifie périodiquement le fournisseur réel.

Exercice 10.5 — Revue d'architecture

Explication du problème. Réduire les risques d'une application qui mélange secret, données sensibles, dépendance réseau et action automatique.

Intuition. Placer des frontières : entrée, politique de données, client modèle, validation, autorisation d'action et observabilité. Chaque frontière refuse explicitement.

Solution directe ou force brute. Ajouter un message « sois prudent » au prompt. Le modèle ne peut protéger un jeton exposé, imposer un timeout réseau ou appliquer une permission qu'il ne contrôle pas.

Solution optimisée ou structurée. Secret dans le gestionnaire d'environnement, minimisation ou traitement local des données, timeout et reprises bornées, sortie structurée, liste blanche d'actions, confirmation humaine et logs sans contenu sensible.

Pseudo-code.

```

1 valider et minimiser l'entree
2 si donnee interdite pour service distant: utiliser modele local ou refuser
3 charger secret depuis environnement securise
4 appeler avec timeout; reprendre seulement erreurs transitoires et au plus deux fois
5 valider schema puis regles metier
6 si action sensible: demander confirmation d'un humain autorise
7 journaliser versions, duree, statut et identifiants non sensibles

```

Python.

```

1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class Policy:
5     remote_allowed: bool

```

```
6     max_chars: int = 8_000
7     max_attempts: int = 2
8
9 def safe_analyze(service, text, policy, contains_sensitive):
10     if contains_sensitive(text) and not policy.remote_allowed:
11         return {"status": "human_review", "reason": "data boundary"}
12     clean = text.strip()[:policy.max_chars]
13     last_error = None
14     for _ in range(policy.max_attempts):
15         try:
16             result = service.analyze(clean)
17             return {"status": "proposed", "result": result,
18                     "requires_confirmation": True}
19         except TimeoutError as exc:
20             last_error = exc
21     return {"status": "human_review", "reason": type(last_error).__name__}
```

Interprétation. Le résultat reste une proposition. L'autorisation et l'action sont des responsabilités déterministes extérieures au modèle.

ANNEXE A

Références et prolongements

Bibliographie

- [1] Python Software Foundation. *The Python Tutorial*. <https://docs.python.org/3/tutorial/>.
- [2] Python Software Foundation. *Data Model*. <https://docs.python.org/3/reference/datamodel.html>.
- [3] Python Software Foundation. *Collections*. <https://docs.python.org/3/library/collections.html>.
- [4] Python Software Foundation. *Data Structures and Built-in Types*. <https://docs.python.org/3/tutorial/datastructures.html>.
- [5] Google for Developers. *Python Lists*, Python Education. <https://developers.google.com/edu/python/lists>.
- [6] Ana Bell. *MIT 6.100L : Introduction to CS and Programming Using Python*, 2022. <https://ocw.mit.edu/courses/6-1001-introduction-to-cs-and-programming-using-python-fall-2022/>.
- [7] Luciano Ramalho. *Fluent Python*, 2nd edition. O'Reilly, 2022. <https://elmoukrie.com/wp-content/uploads/2022/05/luciano-ramalho-fluent-python-clear-concise-and-effective-programming-oreilly-media-2022.pdf>.
- [8] Thomas Cormen, Charles Leiserson, Ronald Rivest et Clifford Stein. *Introduction to Algorithms*, 4th edition. MIT Press, 2022.
- [9] NumPy Developers. *NumPy User Guide*. <https://numpy.org/doc/stable/user/>.
- [10] NumPy Developers. *Broadcasting*. <https://numpy.org/doc/stable/user/basics.broadcasting.html>.
- [11] NumPy Developers. *Random Generator*. <https://numpy.org/doc/stable/reference/random/generator.html>.
- [12] pandas Development Team. *pandas User Guide*. https://pandas.pydata.org/docs/user_guide/.
- [13] pandas Development Team. *Group by : split-apply-combine*. https://pandas.pydata.org/docs/user_guide/groupby.html.
- [14] pandas Development Team. *Merge, join and concatenate*. https://pandas.pydata.org/docs/user_guide/merging.html.
- [15] pandas Development Team. *Copy-on-Write*. https://pandas.pydata.org/docs/user_guide/copy_on_write.html.
- [16] SciPy Community. *Statistical functions*. <https://docs.scipy.org/doc/scipy/reference/stats.html>.
- [17] SciPy Community. *SciPy User Guide*. <https://docs.scipy.org/doc/scipy/tutorial/>.
- [18] Michael Waskom. *seaborn User Guide*. <https://seaborn.pydata.org/tutorial.html>.
- [19] statsmodels Developers. *Statistics and tools*. <https://www.statsmodels.org/stable/stats.html>.
- [20] PyMC Developers. *Introductory Overview of PyMC*. https://www.pymc.io/projects/docs/en/stable/learn/core_notebooks/pymc_overview.html.

- [21] ArviZ Developers. *Exploratory analysis of Bayesian models*. <https://python.arviz.org/en/stable/>.
- [22] scikit-learn Developers. *User Guide*. https://scikit-learn.org/stable/user_guide.html.
- [23] scikit-learn Developers. *Pipelines and composite estimators*. <https://scikit-learn.org/stable/modules/compose.html>.
- [24] Andreas Müller et Sarah Guido. *Introduction to Machine Learning with Python*. O'Reilly, 2016.
- [25] Hugging Face. *huggingface_hub : Run inference on servers*. https://huggingface.co/docs/huggingface_hub/en/guides/inference.
- [26] Hugging Face. *Transformers pipeline tutorial*. https://huggingface.co/docs/transformers/en/pipeline_tutorial.
- [27] Hugging Face. *Datasets : Load*. <https://huggingface.co/docs/datasets/en/loading>.